

Elements of Sampling Survey

R Demonstrations

Longhai Li

2026-09-12

Table of contents

Preface	5
Key Features	5
1 An Introduction to Data Analysis with R	6
1.1 Basic R Objects and Operations	6
1.2 Import a Dataset as a dataframe and Simple Operations	8
1.3 Data Manipulation and Data Table	10
1.3.1 Modern Data Manipulation using <code>dplyr</code>	10
1.3.2 Displaying Large Datasets with a Scroll Bar	11
1.3.3 Advanced Tables with <code>gt</code>	12
1.4 Graphics in R	16
1.4.1 Base R Graphics	16
1.4.2 Saving and Inserting External Plots	17
1.4.3 Advanced Visualization with <code>ggplot2</code>	18
1.5 Create your own function	19
2 Simple Random Sampling	22
2.1 Analysis of <code>agsrs.csv</code> Data	22
2.1.1 Step by step calculation without using a function	22
2.1.2 Write a function for repeated use	23
2.1.3 Comparing with true value	25
2.2 A Simulation Demonstration of SRS Inference	26
2.2.1 True Values	27
2.2.2 One SRS sampling	27
2.2.3 Repeating SRS sampling 5000 times	28
2.2.4 Empirical Coverage Rate of CIs	31
3 Stratified Random Sampling	34
3.1 Key Formulas for Stratified Sampling	34
3.2 Functions and packages for Analyzing Data	35
3.3 Analysis of “ <code>agstrat.csv</code> ” data	35
3.3.1 Importing <code>agstrat.csv</code> data	35
3.3.2 Spreadsheet calculation	36
3.3.3 Using the function “ <code>str_est_data</code> ”	38
3.3.4 Comparing with SRS estimate	39

3.4	Allocation of stratum sample size	40
3.4.1	Analyzing seals.csv collected with stratified sampling	40
3.4.2	Neyman allocation of stratum sample size	41
3.5	Simulation to Study the Efficiency of Stratified Sampling with Different Allocation	42
3.5.1	Using “Region” to Stratify	42
3.5.2	Using “acres82” to Define Strata	47
4	Ratio, Regression, and Domain Estimation	52
4.1	Functions and packages for Analyzing Data	53
4.1.1	General functions for ratio and regression estimation	53
4.2	Ratio Estimation for cherry.csv dataset	53
4.2.1	Importing data	53
4.2.2	SRS estimate	55
4.2.3	Ratio Estimation Using the Function	55
4.3	Simulation study with agpop.csv	59
4.3.1	Information of population data	59
4.3.2	Simulation Studies	61
4.4	Estimating Domain Means and Post-stratification with agsrs.csv example	62
4.4.1	Estimating Domain Means	62
4.4.2	Post-stratification Analysis	63
4.5	Estimating Domain Means and Post-stratification with teacher example	66
4.5.1	Importing Data and Calculating Contingency Table	66
4.5.2	Domain Summary	67
4.5.3	Estimating Domain Means	67
4.5.4	Post-stratification Analysis	68
4.5.5	Comparing to the Analysis with SRS	70
4.6	Regression Estimation for the cherry.csv dataset	70
4.6.1	Importing data	70
4.6.2	Regression Estimation Using the Function	71
4.7	Regression estimation for photo counts of dead trees	74
5	Cluster Sampling	79
5.1	Key Formulas for Cluster Sampling	79
5.2	Functions and packages for Analyzing Data	80
5.3	Estimating for One-stage cluster sampling: analysis of algebra.csv	80
5.4	Estimating for two-stage cluster sampling: analysis of coots.csv	83
6	Unequal Probability Sampling	87
6.1	Key Concepts and Formulas	87
6.2	Functions and Packages	87
6.3	Analysis of a dataset on student study hours collected by UPSWR	88
6.4	Analysis of <code>statepop.csv</code> collected with one-stage UPSWR	90
6.4.1	Compare with an estimate based on an SRS sample	95

6.5	Analysis of <code>studenthrs.csv</code> collected with two-stage UPSWR	95
6.6	Simulation Studies for UPSWR using <code>agpop</code> data	99
6.6.1	Visualizing Estimator Variance	100
6.7	Analysis of a dataset on student study hours collected by UPSWO	102
Appendices		104
A	Estimating Functions Source Code	104

Preface

Statistical inference in the real world almost always deals with bounded, finite populations—whether we are estimating agricultural yields, public health metrics, or economic indicators. This book explores the mathematical theory and practical mechanics of drawing reliable samples from those populations, examining how intelligent survey design directly dictates the precision and cost of our conclusions.

The chapters cover a comprehensive suite of methodologies, including simple random sampling, stratified random sampling, cluster and systematic sampling, probability proportionate to size (PPS) sampling, and powerful auxiliary-variable strategies like difference, ratio, and regression estimation.

Key Features

- **Custom R Functions:** Rather than relying exclusively on pre-built packages, this text emphasizes writing custom functions to analyze survey data, helping students learn how R works under the hood.
- **Modern R Workflows:** Data manipulation and presentation leverage the modern R ecosystem, utilizing `dplyr` for clean wrangling, `ggplot2` for advanced visualization, and `gt` for publication-ready mathematical tables.
- **Simulation-Based Learning:** Extensive simulation studies demonstrate the practical differences between sampling schemes, with a core focus on how predictive auxiliary variables improve precision and reduce variance.

1 An Introduction to Data Analysis with R

Modern workflows with dplyr, gt, and ggplot2

1.1 Basic R Objects and Operations

```
# create a vector
x <- 1:10
x <- seq(30, 3, by = -2)
a <- c(66.32, 69.87, 70.12, 90.37, 50.08, 61.20, 65.00, 57.65)
d <- a[1]
a[1] <- 85.34

mean(a)
```

```
[1] 68.70375
```

```
ma <- mean(a)

# read a vector of numbers from a file (assumes files exist)
# x <- scan("data/numbers.txt")
# x2 <- scan("data/number2.txt")

# one can also read number without saving to a file
y <- scan(text = "7 8 9 10 11 12 13 13 14 17 17 45")

# create a matrix
A <- matrix(0, 4, 2)
A <- matrix(1:8, 4, 2)
A
```

```

      [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8

```

```

D <- matrix(a, 4, 2, byrow = TRUE)
D <- matrix(1:8, 2, 4)
D

```

```

      [,1] [,2] [,3] [,4]
[1,]    1    3    5    7
[2,]    2    4    6    8

```

```

# create another matrix with all entry 0
B <- matrix(1:5000, 100, 50)

# assign a number to B
B[2,4] <- 45
head(B[1,]) # view part of the first row

```

```

[1]    1 101 201 301 401 501

```

```

head(B[,1]) # view part of the first column

```

```

[1] 1 2 3 4 5 6

```

```

B[1,] <- 1:50

# create a list
E <- list(newa = a, newA = A)
# list the names of components
names(E)

```

```

[1] "newa" "newA"

```

```

# to look at the component of E
E$newA

```

```

      [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8

```

```

E$newa <- 10:17

# create a dataframe
scores <- c(30, 45, 50)
names <- c("Peter", "John", "Alice")
stat245_scores <- data.frame(names, scores)
stat245_scores

```

```

      names scores
1 Peter      30
2 John      45
3 Alice      50

```

```
stat245_scores$names
```

```
[1] "Peter" "John" "Alice"
```

```
stat245_scores$scores[1] <- 40
```

1.2 Import a Dataset as a dataframe and Simple Operations

```

# import agpop.csv and treat -99 as NA
agpop <- read.csv("data/agpop.csv", na.strings = "-99")

# preview agpop
head(agpop)

```

```

      county state acres92 acres87 acres82 farms92 farms87 farms82
1 ALEUTIAN ISLANDS AREA AK  683533  726596  764514      26      27      28
2 ANCHORAGE AREA AK    47146   59297  256709     217     245     223
3 FAIRBANKS AREA AK   141338  154913  204568     168     175     170

```


4	JUNEAU AREA	AK	210	214	127	8	8	12
5	KENAI PENINSULA AREA	AK	50810	85712	98035	93	119	137
6	AUTAUGA COUNTY	AL	107259	116050	145044	322	388	453
	largef92	largef87	largef82	smallf92	smallf87	smallf82	region	
1	14	16	20	6	4	1	W	
2	9	10	11	41	52	38	W	
3	25	28	21	12	18	25	W	
4	0	0	0	5	4	8	W	
5	9	18	17	12	18	19	W	
6	25	32	32	8	19	17	S	

```
# look at the variable names
colnames(agpop)
```

```
[1] "county" "state" "acres92" "acres87" "acres82" "farms92"
[7] "farms87" "farms82" "largef92" "largef87" "largef82" "smallf92"
[13] "smallf87" "smallf82" "region"
```

```
# find number of cols and rows
ncol(agpop)
```

```
[1] 15
```

```
nrow(agpop)
```

```
[1] 3078
```

```
# find mean and sd of acres92
mean(agpop$acres92, na.rm = TRUE)
```

```
[1] 308582.4
```

```
sd(agpop$acres92, na.rm = TRUE)
```

```
[1] 425312.8
```

Characteristic	NC N = 1,054 ¹	NE N = 220 ¹	S N = 1,382 ¹	W N = 422 ¹
acres92	326,571 (271,188)	93,600 (78,906)	200,009 (244,132)	730,267 (836,614)
Unknown	2	7	6	4
acres87	333,754 (270,834)	102,458 (86,127)	204,202 (243,247)	746,210 (847,298)
Unknown	4	5	8	6
largef92	75 (81)	8 (11)	37 (47)	98 (97)
farms92	737 (390)	509 (441)	559 (435)	625 (777)

¹Mean (SD)

```
## Modern summary table using gtsummary
# install.packages("gtsummary") # Run this once in your console if you don't have it
library("gtsummary")
agpop |>
  select(region, acres92, acres87, largef92, farms92) |>
  tbl_summary(
    by = region,
    statistic = list(all_continuous() ~ "{mean} ({sd})")
  )
```

1.3 Data Manipulation and Data Table

1.3.1 Modern Data Manipulation using dplyr

The `dplyr` package and the native pipe (`|>`) make modifying, subsetting, and summarizing data frames much cleaner than base R.

```
library(dplyr)

# 1. Modifying an existing data frame with mutate()
# Instead of updating columns one by one with $, we can create multiple new columns at once
stat245_scores <- stat245_scores |>
  mutate(
    perc = scores / 50 * 100,
    adj = perc + 10
  )
stat245_scores
```

	names	scores	perc	adj
1	Peter	40	80	90
2	John	45	90	100
3	Alice	50	100	110

```
# 2. Filter rows based on conditions
agpop_AK <- agpop |> filter(state == "AK")
agpop_W <- agpop |> filter(region == "W")
agpop_largefarm <- agpop |> filter(largef92 > 10)

# 3. Select specific columns and slice specific rows
# This replaces base R: agpop[1:20, c("acres92", "largef92")]
agpop_subset <- agpop |>
  slice(1:20) |>
  select(acres92, largef92)

head(agpop_subset)
```

	acres92	largef92
1	683533	14
2	47146	9
3	141338	25
4	210	0
5	50810	9
6	107259	25

1.3.2 Displaying Large Datasets with a Scroll Bar

When working with large datasets, printing the entire dataframe to your HTML document takes up too much vertical space and can slow down the browser. We can use the `gt` package to create a fixed-height, scrollable window, so HTML readers can scroll through the **full** dataset. A static, non-HTML render (e.g. Typst/PDF) has no scrollbar, so there we show only the head of the data instead.

```
library(gt)

# Non-HTML output (e.g. Typst/PDF): a scrollbar doesn't exist on a static
# page, so just show the head of the dataset instead of the full table
agpop |>
  head() |>
  gt() |>
```

Agricultural Population Data

First 6 rows (see the HTML version for the full, scrollable table)

county	state	acres92	acres87	acres82	farms92	farms87	farms82	largef92
ALEUTIAN ISLANDS AREA	AK	683533	726596	764514	26	27	28	28
ANCHORAGE AREA	AK	47146	59297	256709	217	245	223	223
FAIRBANKS AREA	AK	141338	154913	204568	168	175	170	170
JUNEAU AREA	AK	210	214	127	8	8	12	12
KENAI PENINSULA AREA	AK	50810	85712	98035	93	119	137	137
AUTAUGA COUNTY	AL	107259	116050	145044	322	388	453	453

```
tab_header(  
  title = "Agricultural Population Data",  
  subtitle = "First 6 rows (see the HTML version for the full, scrollable table)"  
)
```

Output Note: The HTML version shows the full dataset in a clean, professional, 400-pixel-high scrolling container; the Typst/PDF version shows only the first six rows since a printed page cannot scroll.

1.3.3 Advanced Tables with gt

The `gt` package also allows you to create highly customized, publication-ready tables. Below, we summarize five variables of the `agpop` dataset by region, computing both the mean and the variance of each, and create a table featuring spanning headers across multiple columns and complex mathematical LaTeX symbols in the column labels.

```
# The five variables we want to summarize  
vars_of_interest <- c("acres92", "acres87", "farms92", "largef92", "smallf92")  
  
# Create a summary dataset by region: mean and variance of each variable,  
# filtering out rows where any of the five variables is missing (-99)  
ag_summary <- agpop |>  
  filter(if_all(all_of(vars_of_interest), ~ .x != -99)) |>  
  group_by(region) |>  
  summarize(  
    across(  
      all_of(vars_of_interest),  
      list(mean = ~mean(.x), var = ~var(.x)),  
    )  
  )
```

```

      .names = "{.col}_{.fn}"
    )
  )
ag_summary

# A tibble: 4 x 11
  region acres92_mean acres92_var acres87_mean acres87_var farms92_mean
  <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
1 NC          327483.  73461356635.    334064.  73319885590.    740.
2 NE          94462.   6205925458.    104387.  7358098722.    529.
3 S          200846.  59700811734.    203877.  58908068196.    563.
4 W          736031.  702941261694.    749812.  718684588595.    636.
# i 5 more variables: farms92_var <dbl>, largef92_mean <dbl>,
#   largef92_var <dbl>, smallf92_mean <dbl>, smallf92_var <dbl>

# Display the summary using a customized gt table
ag_summary |>
  gt() |>
  tab_header(
    title = "Regional Summary: Mean and Variance of Five Variables",
    subtitle = "acres92, acres87, farms92, largef92, smallf92"
  ) |>
  # Add spanning headers grouping the mean columns and the variance columns
  tab_spanner(
    label = md("**Mean**"),
    columns = ends_with("_mean")
  ) |>
  tab_spanner(
    label = md("**Variance**"),
    columns = ends_with("_var")
  ) |>
  # Use LaTeX math expressions for column names
  cols_label(
    region = md("**Region**"),
    acres92_mean = md("$\\bar{y}_{acres92}$"),
    acres87_mean = md("$\\bar{y}_{acres87}$"),
    farms92_mean = md("$\\bar{y}_{farms92}$"),
    largef92_mean = md("$\\bar{y}_{largef92}$"),
    smallf92_mean = md("$\\bar{y}_{smallf92}$"),
    acres92_var = md("$s^2_{acres92}$"),
    acres87_var = md("$s^2_{acres87}$"),

```

Regional Summary: Mean and Variance of Five Variables

acres92, acres87, farms92, largef92, smallf92

Region	Mean					Variance		
	$\bar{y}_{acres92}$	$\bar{y}_{acres87}$	$\bar{y}_{farms92}$	$\bar{y}_{largef92}$	$\bar{y}_{smallf92}$	$s^2_{acres92}$	$s^2_{acres87}$	$s^2_{farms92}$
NC	327,482.9	334,064.2	740.2	75.3	42.8	73,461,356,635	73,319,885,590	1,344,444,444
NE	94,462.2	104,387.4	529.3	8.7	49.4	6,205,925,458	7,358,098,722	1,344,444,444
S	200,845.8	203,876.7	563.2	37.1	44.0	59,700,811,734	58,908,068,196	1,344,444,444
W	736,031.4	749,811.7	636.0	99.3	121.4	702,941,261,694	718,684,588,595	6,205,925,458

```

farms92_var = md("$s^2_{farms92}$"),
largef92_var = md("$s^2_{largef92}$"),
smallf92_var = md("$s^2_{smallf92}$")
) |>
fmt_number(
  columns = ends_with("_mean"),
  decimals = 1
) |>
fmt_number(
  columns = ends_with("_var"),
  decimals = 0,
  sep_mark = ",",
) |>
tab_options(table.width = pct(100))

```

The spanners above group columns by **statistic** (all means together, all variances together). We can instead reorder the columns with `select()` and group them by **variable**, so each variable's mean and variance sit side by side under their own spanning header.

```

ag_summary |>
  select(region,
    acres92_mean, acres92_var,
    acres87_mean, acres87_var,
    farms92_mean, farms92_var,
    largef92_mean, largef92_var,
    smallf92_mean, smallf92_var) |>
  gt() |>
  tab_header(
    title = "Regional Summary: Mean and Variance of Five Variables",
    subtitle = "Grouped by variable"
  )

```

```

) |>
# One spanning header per variable, covering its mean and variance columns
tab_spanner(label = md("**acres92**"), columns = c(acres92_mean, acres92_var)) |>
tab_spanner(label = md("**acres87**"), columns = c(acres87_mean, acres87_var)) |>
tab_spanner(label = md("**farms92**"), columns = c(farms92_mean, farms92_var)) |>
tab_spanner(label = md("**largef92**"), columns = c(largef92_mean, largef92_var)) |>
tab_spanner(label = md("**smallf92**"), columns = c(smallf92_mean, smallf92_var)) |>
# The spanner already names the variable, so the column labels can stay generic
cols_label(
  region      = md("**Region**"),
  acres92_mean = md("$\\bar y$"),
  acres92_var  = md("$s^2$"),
  acres87_mean = md("$\\bar y$"),
  acres87_var  = md("$s^2$"),
  farms92_mean = md("$\\bar y$"),
  farms92_var  = md("$s^2$"),
  largef92_mean = md("$\\bar y$"),
  largef92_var  = md("$s^2$"),
  smallf92_mean = md("$\\bar y$"),
  smallf92_var  = md("$s^2$")
) |>
fmt_number(
  columns = ends_with("_mean"),
  decimals = 1
) |>
fmt_number(
  columns = ends_with("_var"),
  decimals = 0,
  sep_mark = ", "
) |>
tab_options(table.width = pct(100))

```

Output Note: Both tables show exactly the same numbers; only the column grouping differs. Grouping by variable (this view) makes it easy to compare a single variable's mean and variance; grouping by statistic (the earlier view) makes it easy to compare means (or variances) across variables.

Regional Summary: Mean and Variance of Five Variables Grouped by variable

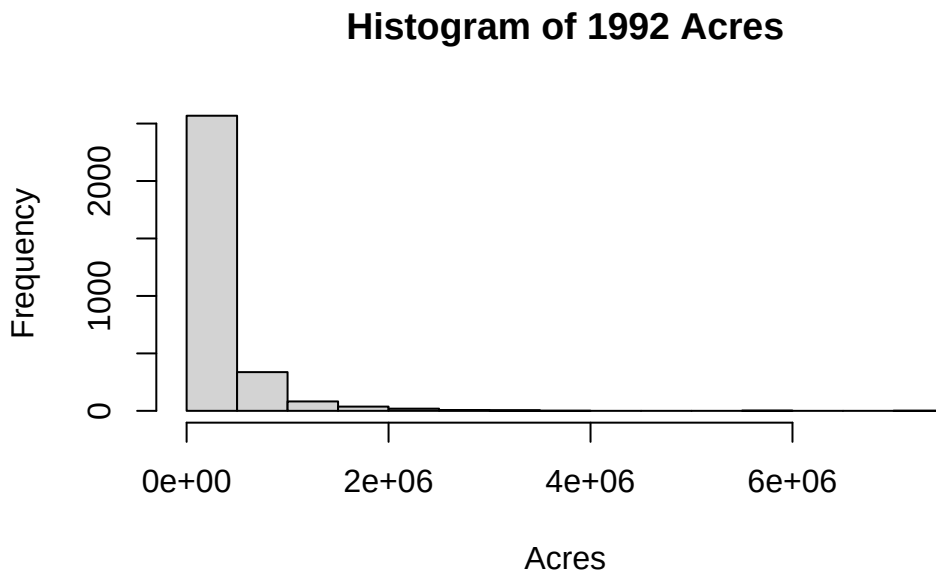
Region	acres92		acres87		farms92		largef92		s
	$\bar{y}$	s^2	$\bar{y}$	s^2	$\bar{y}$	s^2	$\bar{y}$	s^2	
NC	327,482.9	73,461,356,635	334,064.2	73,319,885,590	740.2	150,770	75.3	6,490	4
NE	94,462.2	6,205,925,458	104,387.4	7,358,098,722	529.3	191,801	8.7	126	4
S	200,845.8	59,700,811,734	203,876.7	58,908,068,196	563.2	189,097	37.1	2,241	4
W	736,031.4	702,941,261,694	749,811.7	718,684,588,595	636.0	608,924	99.3	9,455	12

1.4 Graphics in R

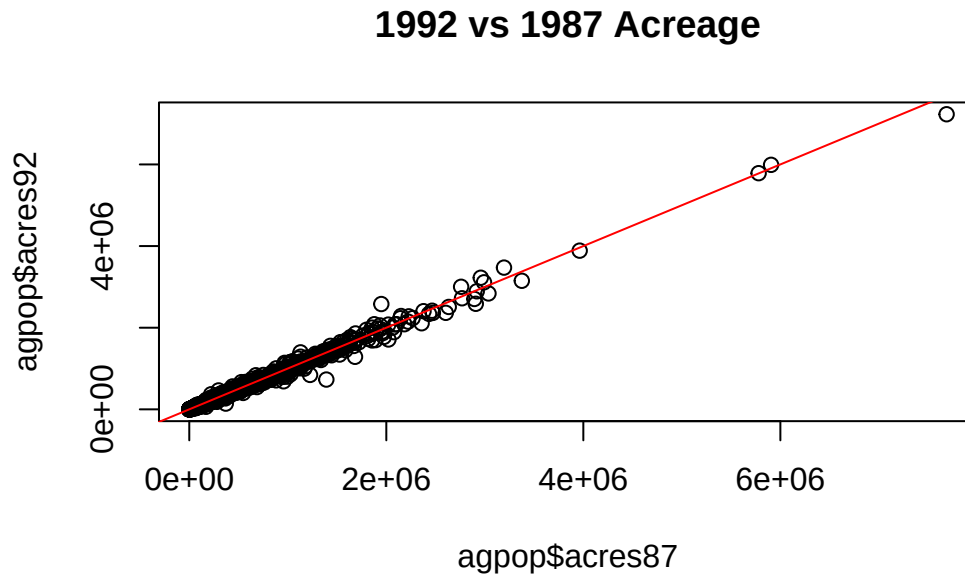
1.4.1 Base R Graphics

Base R includes built-in functions for quickly generating standard plots.

```
hist(agpop$acres92, main = "Histogram of 1992 Acres", xlab = "Acres")
```



```
plot(agpop$acres87, agpop$acres92, main = "1992 vs 1987 Acreage")
abline(a = 0, b = 1, col = "red")
```

Here is the new sub-section you can add right below your standard plotting chunk to explain this workflow to your readers.

1.4.2 Saving and Inserting External Plots

Sometimes you need to export a high-resolution plot for a journal submission or a presentation. You can save a plot directly to your computer by opening a “graphics device” (like `pdf()`, `png()`, or `jpeg()`), running your plot code, and then closing the device with `dev.off()`.

```
# 1. Open the PNG device with high resolution (res = 300)
# Note: When increasing resolution, you must also scale up the width/height in pixels or spe
png("agpop_scatter_highres.png", width = 6, height = 4, units = "in", res = 300)

plot(agpop$acres87, agpop$acres92, main = "1992 vs 1987 Acreage")
abline(a = 0, b = 1, col = "red")

dev.off()
```

Once the file is saved in your project folder, you can insert it back into your Quarto document at any time using the `knitr` package.

```
# Insert the saved PDF back into the document
knitr::include_graphics("agpop_scatter_highres.png")
```

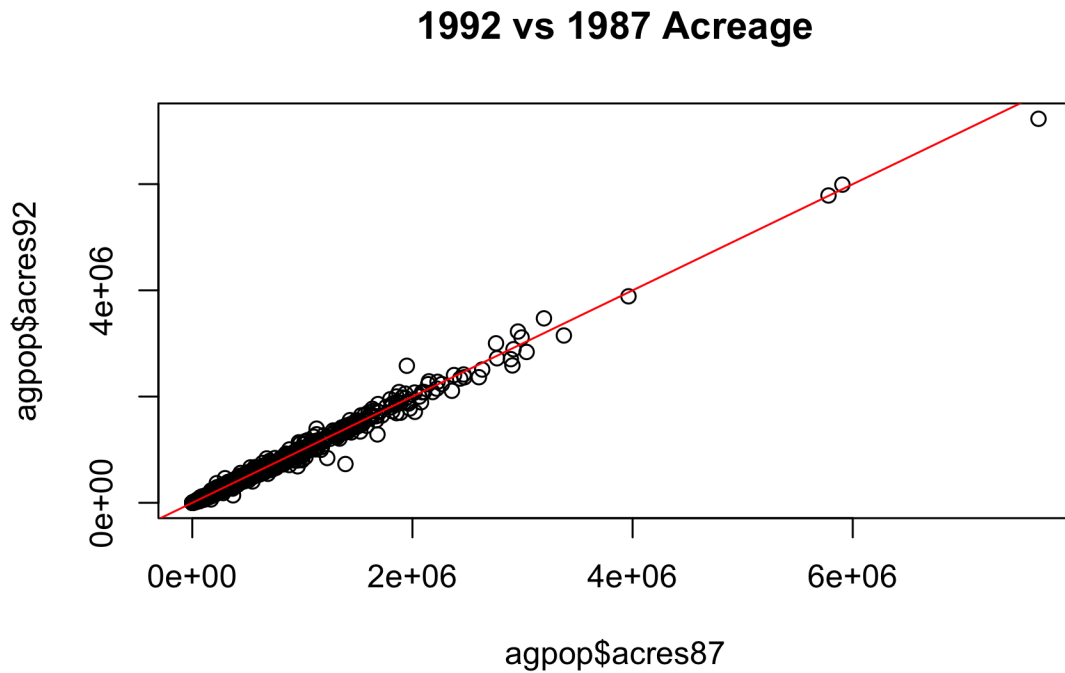


Figure 1.1: My Imported PNG Plot

1.4.3 Advanced Visualization with ggplot2

While base R graphics are useful for quick checks, `ggplot2` provides a powerful grammar of graphics for complex visuals. Below, we generate a histogram of log-transformed 1992 acreages, overlaying both an empirical density curve and a theoretical normal distribution curve.

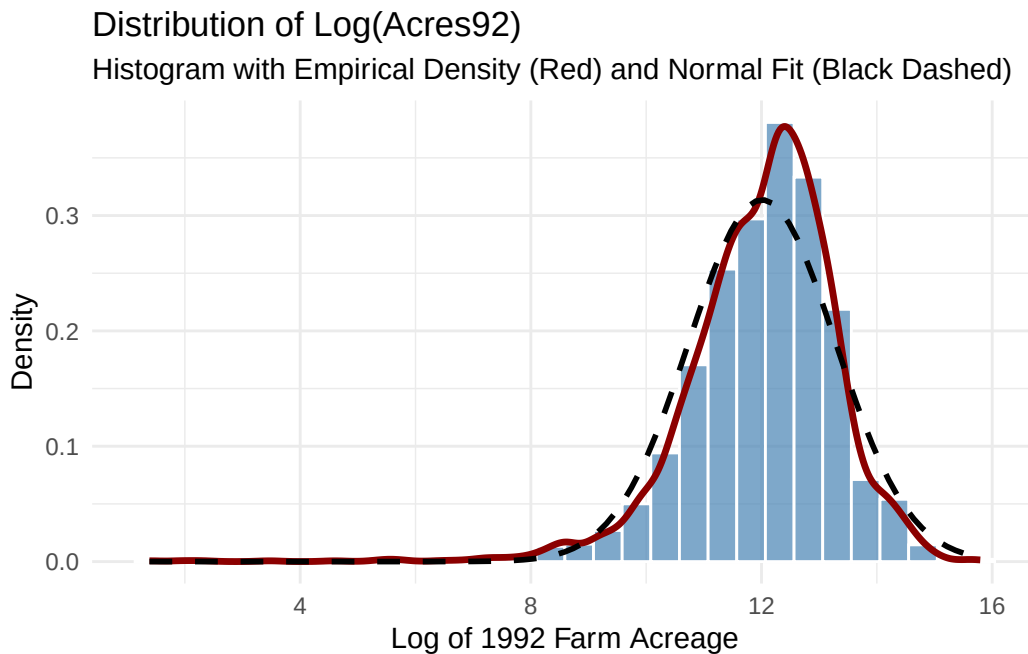
```
library(ggplot2)

# Modern data prep with pipes
ag_clean <- agpop |>
  filter(acres92 != -99, acres92 > 0) |>
  mutate(log_acres = log(acres92))

# Calculate parameters for the theoretical normal curve
mean_log <- mean(ag_clean$log_acres)
sd_log <- sd(ag_clean$log_acres)

ggplot(ag_clean, aes(x = log_acres)) +
```

```
# 1. Base Histogram mapped to density (instead of count)
geom_histogram(aes(y = after_stat(density)), bins = 30,
               fill = "steelblue", color = "white", alpha = 0.7) +
# 2. Empirical Density Curve (Dark Red)
geom_density(color = "darkred", linewidth = 1.2) +
# 3. Theoretical Normal Curve (Black, Dashed)
stat_function(fun = dnorm,
              args = list(mean = mean_log, sd = sd_log),
              color = "black", linetype = "dashed", linewidth = 1) +
labs(
  title = "Distribution of Log(Acres92)",
  subtitle = "Histogram with Empirical Density (Red) and Normal Fit (Black Dashed)",
  x = "Log of 1992 Farm Acreage",
  y = "Density"
) +
theme_minimal()
```



1.5 Create your own function

```
## data is a matrix or data.frame
means_col <- function(data)
```

```

{
  n <- ncol(data)
  cmeans <- rep(NA, n)
  for (j in 1:n)
  {
    cmeans[j] <- mean(data[,j], na.rm = TRUE)
  }
  cmeans
}

```

```

library(gt)

## apply custom function (using dplyr to select numeric columns first)
agpop_numeric <- agpop |> select(3:13)
custom_means <- means_col(agpop_numeric)

## apply R built-in function
builtin_means <- colMeans(agpop_numeric, na.rm = TRUE)

## Create a data frame to compare the results
comparison_df <- data.frame(
  Variable = colnames(agpop_numeric),
  Custom_Function = custom_means,
  Built_in = builtin_means
)

## Display the comparison as a clean table
comparison_df |>
  gt() |>
  tab_header(
    title = "Comparing Mean Calculations",
    subtitle = "Custom Function vs. Base R colMeans()"
  ) |>
  cols_label(
    Variable = md("**Variable**"),
    Custom_Function = md("**Custom `means_col()`**"),
    Built_in = md("**Base `colMeans()`**")
  ) |>
  fmt_number(
    columns = c(Custom_Function, Built_in),
    decimals = 2
  )

```

Comparing Mean Calculations

Custom Function vs. Base R colMeans()

Variable	Custom <code>means_col()</code>	Base <code>colMeans()</code>
acres92	308,582.41	308,582.41
acres87	315,373.71	315,373.71
acres82	321,972.52	321,972.52
farms92	625.50	625.50
farms87	678.28	678.28
farms82	728.06	728.06
largef92	56.18	56.18
largef87	54.86	54.86
largef82	52.62	52.62
smallf92	54.09	54.09
smallf87	59.54	59.54

Output Note: Displaying the results side-by-side confirms that our custom `means_col()` function produces the exact same output as R's highly optimized built-in `colMeans()` function.

2 Simple Random Sampling

2.1 Analysis of agsrs.csv Data

2.1.1 Step by step calculation without using a function

To evaluate a Simple Random Sample (SRS), we calculate the point estimate, the standard error (incorporating the Finite Population Correction factor), and the margin of error to construct a 95% confidence interval.

Key Formulas:

- **Sample Mean:** $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$
- **Standard Error:** $SE(\bar{y}) = \sqrt{1 - \frac{n}{N}} \left(\frac{s}{\sqrt{n}} \right)$
- **Margin of Error:** $ME = t_{\alpha/2, n-1} \times SE(\bar{y})$
- **Population Total:** $\hat{t} = N\bar{y}$

```
## read survey data
agsrs <- read.csv ("data/agsrs.csv")
head(agsrs)
```

	county	state	acres92	acres87	acres82	farms92	farms87	farms82
1	COFFEE COUNTY	AL	175209	179311	194509	760	842	944
2	COLBERT COUNTY	AL	138135	145104	161360	488	563	686
3	LAMAR COUNTY	AL	56102	59861	72334	299	362	447
4	MARENGO COUNTY	AL	199117	220526	231207	434	471	622
5	MARION COUNTY	AL	89228	105586	113618	566	658	748
6	TUSCALOOSA COUNTY	AL	96194	120542	134616	436	521	650
	largef92	largef87	largef82	smallf92	smallf87	smallf82	region	
1	29	28	21	57	47	66	S	
2	37	41	42	12	44	47	S	
3	4	4	3	16	20	30	S	
4	48	66	62	14	11	28	S	
5	7	9	9	11	23	27	S	
6	20	17	23	18	32	29	S	

```
## extract the variable of interest
sdata <- agsrs$acres92
N <- 3078

## do calculation
n <- length(sdata)
ybar <- mean(sdata)
se.ybar <- sqrt((1 - n / N)) * sd(sdata) / sqrt(n)
mem <- qt(0.975, df = n - 1) * se.ybar

## return estimate vector for pop mean
c(Est. = ybar, S.E. = se.ybar, ci.low = ybar - mem, ci.upp = ybar + mem)
```

Est.	S.E.	ci.low	ci.upp
297897.05	18898.43	260706.26	335087.84

```
## return estimate vector for pop total
c(Est. = ybar, S.E. = se.ybar, ci.low = ybar - mem, ci.upp = ybar + mem) * N
```

Est.	S.E.	ci.low	ci.upp
916927110	58169381	802453859	1031400361

2.1.2 Write a function for repeated use

2.1.2.1 A function for doing data analysis for srs sample

Wrap the previous manual calculations into a custom R function to quickly estimate means, standard errors, and confidence intervals for any variable of interest. To avoid repeating this code in every chapter, all estimating functions (and their `gt-table` formatting helpers) live in a single shared file, `samplingestimate.r`, which we source below.

2.1.2.2 Apply `srs_est` to `agsrs.csv` data

Import Data

Load the survey sample data representing US counties.

```
agsrs <- read.csv("data/agsrs.csv")
```

SRS Mean Estimation: Working Table

$n = 300$

i	y_i	$y_i - \bar{y}$	$(y_i - \bar{y})^2$ ¹
1	1.752×10^5	-1.227×10^5	1.505×10^{10}
2	1.381×10^5	-1.598×10^5	2.552×10^{10}
3	5.610×10^4	-2.418×10^5	5.846×10^{10}
4	1.991×10^5	-9.878×10^4	9.757×10^9
5	8.923×10^4	-2.087×10^5	4.354×10^{10}
6	9.619×10^4	-2.017×10^5	4.068×10^{10}
...	...	...	...
Sum	8.937×10^7	-3.900×10^{-8}	3.550×10^{13}

$$^1 s_y^2 = \sum_i (y_i - \bar{y})^2 / (n - 1) = 118716008180.4059.$$

Point estimate: $\bar{y} = \frac{\sum_i y_i}{n} = \frac{8.937 \times 10^7}{300} = 2.979 \times 10^5$

Standard error: $SE(\bar{y}) = \sqrt{1 - \frac{n}{N} \frac{s}{\sqrt{n}}} = \sqrt{1 - \frac{300}{3078} \frac{3.446 \times 10^5}{\sqrt{300}}} = 18,898.434$

$\bar{y}$	$SE(\bar{y})$	95% CI Lower	95% CI Upper
297,897.0467	18,898.4344	260,706.2569	335,087.8365

Estimating the mean of acre92

Calculate the estimated average farm acreage per county using the custom function. The function's `$table` shows the row-level working values (truncated to the first 6 rows + Sum), and `$estimate` holds the final point estimate, SE, and CI, formatted with `format_est_gt()`.

```
res_mean <- srs_est(agsrs[, "acres92"], N = 3078, estimate = "mean")
res_mean$table
```

```
res_mean$estimate |> format_est_gt("mean")
```

Estimating the total of acre92

Set `estimate = "total"` to have the function itself scale the mean and confidence interval by the population size ($N = 3078$) to estimate the total farm acreage across all counties.

$\hat{t}$	SE($\hat{t}$)	95% CI Lower	95% CI Upper
916,927,109.6400	58,169,381.1695	802,453,858.6054	1,031,400,360.6746

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
0.5100	0.0275	0.4560	0.5640

```
res_total <- srs_est(agsrs[, "acres92"], N = 3078, estimate = "total", show.details = FALSE)
res_total$estimate |> format_est_gt("total")
```

Estimating the proportion of counties with fewer than 200K acres for farming in 1992

To estimate a proportion under SRS, create an indicator variable ($y_i = 1$ if true, 0 if false). The mean of this indicator variable serves as the sample proportion ($\hat{p} = \bar{y}_{indicator}$).

```
acres92.is.fewer.200k <- as.numeric (agsrs[, "acres92"] < 200000)
head(acres92.is.fewer.200k)
```

```
[1] 1 1 1 1 1 1
```

```
res_prop <- srs_est(acres92.is.fewer.200k, N = 3078, estimate = "mean", show.details = FALSE)
res_prop$estimate |> format_est_gt("mean")
```

Estimating the total number of counties with fewer than 200K acres for farming in 1992

Set `estimate = "total"` to estimate the absolute count of counties meeting the criteria.

```
res_prop_total <- srs_est(acres92.is.fewer.200k, N = 3078, estimate = "total", show.details = FALSE)
res_prop_total$estimate |> format_est_gt("total")
```

2.1.3 Comparing with true value

Load the complete census (population) dataset to calculate the true parameters. This allows us to assess the accuracy of our previous SRS estimates against the exact population values.

$\hat{t}$	SE($\hat{t}$)	95% CI Lower	95% CI Upper
1,569.7800	84.5372	1,403.4167	1,736.1433

```
agpop <- read.csv ("data/agpop.csv", na = "-99")
#true mean
mean (agpop[, "acres92"], na.rm = T)
```

```
[1] 308582.4
```

```
# true total
sum (agpop[, "acres92"], na.rm = T)
```

```
[1] 943953599
```

```
# true proportion of counties with less than 200K acres for farming
mean (agpop[, "acres92"] < 200000, na.rm = T)
```

```
[1] 0.5145472
```

```
# true number of counties with less than 200K acres for farming
sum (agpop[, "acres92"] < 200000, na.rm = T)
```

```
[1] 1574
```

2.2 A Simulation Demonstration of SRS Inference

Load the full population data and filter out missing values to serve as a complete, known finite population for resampling experiments.

```
# read population data
agpop <- read.csv ("data/agpop.csv")
# remove those counties with na
agpop <- subset( agpop, acres92 != -99)
```

2.2.1 True Values

Define the target sample size and compute the true population mean (μ) and theoretical standard error (SE_{true}) using the known population standard deviation (S).

Theoretical Standard Error:

$$SE_{true}(\bar{y}) = \sqrt{1 - \frac{n}{N}} \left(\frac{S}{\sqrt{n}} \right)$$

```
# sample size
n <- 300
# population size
N <- nrow (agpop); N
```

```
[1] 3059
```

```
# true value of population mean
ybarU <- mean (agpop[, "acres92"]); ybarU
```

```
[1] 308582.4
```

```
# true value of deviation of sample mean
true.se.ybar <- sqrt (1- n/N) * sd (agpop[, "acres92"]) / sqrt (n); true.se.ybar
```

```
[1] 23320.29
```

2.2.2 One SRS sampling

Draw a single random sample of size $n = 300$ from the population and run our estimator function to see a single instance of survey inference.

```
##
# srs sampling
srs <- sample (1:N, n)
head(agpop [srs, ])
```

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
345,255.2267	31,081.5283	284,088.9650	406,421.4884

	county	state	acres92	acres87	acres82	farms92	farms87	farms82
1819	HAMILTON COUNTY	NE	321080	340855	336318	664	770	821
1459	PUTNAM COUNTY	MO	254493	271529	250851	581	661	715
280	TELLER COUNTY	CO	104010	83281	75437	81	67	65
391	CLAY COUNTY	GA	42678	46081	55315	49	60	94
1830	KEYA PAHA COUNTY	NE	446007	474848	456566	206	259	269
161	YAVAPAI COUNTY	AZ	2108834	2358559	2508888	463	484	474
	largef92	largef87	largef82	smallf92	smallf87	smallf82	region	
1819	60	52	55	60	51	55	NC	
1459	56	63	42	22	36	25	NC	
280	19	21	17	7	5	5	W	
391	14	14	16	3	1	0	S	
1830	124	142	144	12	15	26	NC	
161	106	110	101	102	117	96	W	

```
# get data of variable "acres92"
sdata <- agpop [srs, "acres92"]
# analysis
res_one <- srs_est(sdata, N, estimate = "mean", show.details = FALSE)
res_one$estimate |> format_est_gt("mean")
```

2.2.3 Repeating SRS sampling 5000 times

To demonstrate the Central Limit Theorem and unbiasedness of the estimator, draw 5,000 independent samples, storing the estimates from each iteration. We plot the resulting sampling distribution against the true population mean.

(Note: We use `show.details = FALSE` and pull out just `$estimate` here, so the loop skips building 5,000 working tables and stays fast.)

```
nres <- 5000 # number of repeated sampling
simulation.results <- matrix(0, nres, 4) # matrix recording repeated results
colnames(simulation.results) <- c("Est.", "S.E.", "ci.low", "ci.upp")

for (i in 1:nres)
{
  srs <- sample(N, n)
```

```

sdata <- agpop [srs, "acres92"]
simulation.results [i,] <- srs_est (sdata, N, estimate = "mean", show.details = FALSE)$est
}

head(simulation.results)

```

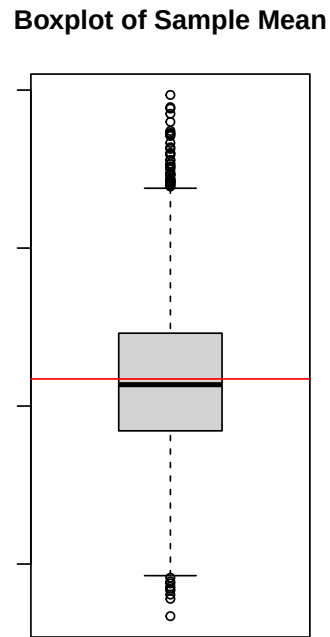
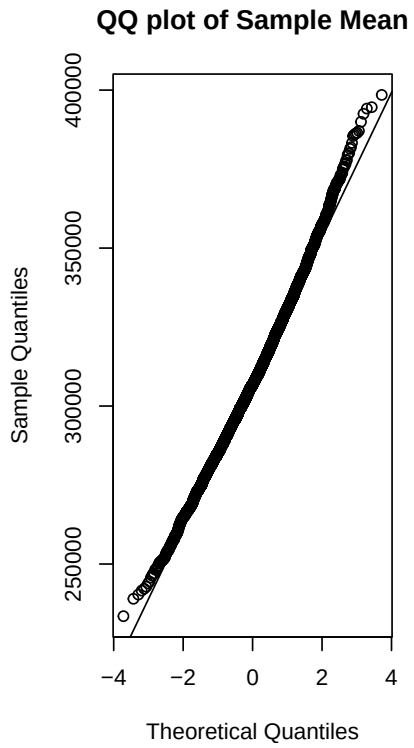
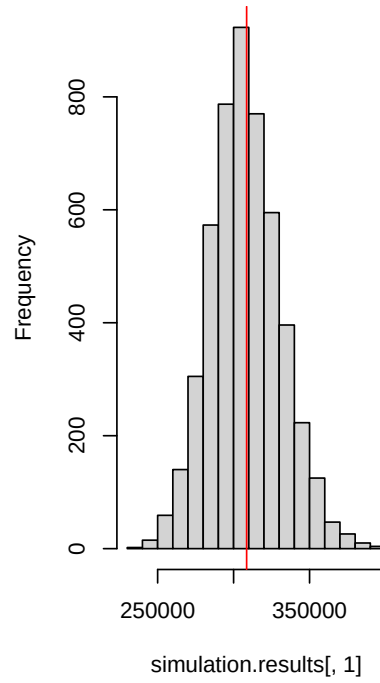
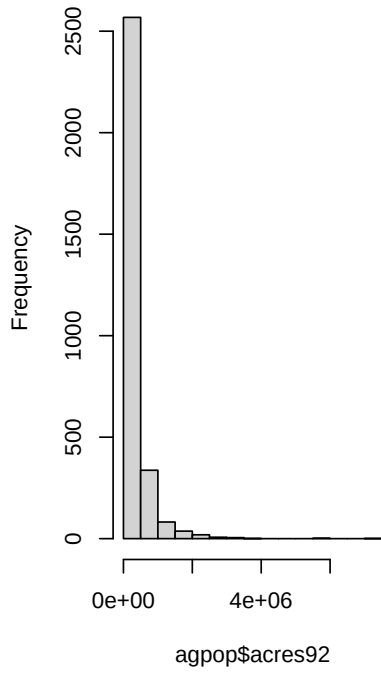
	Est.	S.E.	ci.low	ci.upp
[1,]	310499.5	22231.70	266749.0	354249.9
[2,]	290362.7	19739.07	251517.5	329207.8
[3,]	317716.8	21877.39	274663.7	360770.0
[4,]	286476.8	19692.36	247723.6	325230.0
[5,]	297487.1	20807.49	256539.4	338434.8
[6,]	317689.7	21925.18	274542.5	360836.9

```

# look at the distribution of sample mean
par (mfrow= c(2,2))
hist (agpop$acres92, main = "Population Distribution of acre92")
hist (simulation.results[,1], main = "Sampling Distribution of Sample Mean for acre92")
abline (v = ybarU, col = "red")
qqnorm (simulation.results[,1], main="QQ plot of Sample Mean"); qqline(simulation.results[,1])
boxplot (simulation.results[,1], main = "Boxplot of Sample Mean")
abline (h = ybarU, col = "red")

```

Population Distribution of acre9ling Distribution of Sample Mean f



```
mean (simulation.results[,1])
```

```
[1] 307896.2
```

```
ybarU
```

```
[1] 308582.4
```

```
sd (simulation.results [,1])
```

```
[1] 23102.36
```

```
true.se.ybar
```

```
[1] 23320.29
```

2.2.4 Empirical Coverage Rate of CIs

Verify the reliability of the 95% confidence intervals by checking whether the true population mean falls within the calculated bounds for each of the 5,000 samples. The empirical coverage rate should closely approximate 0.95. The plot highlights the first 100 intervals, flagging those that fail to capture the true mean in a distinct color.

```
library(dplyr)
library(plotrix)

# Convert to data frame (if it was a matrix) and use mutate to evaluate coverage
simulation.results <- as.data.frame(simulation.results) |>
  mutate(
    `Covered?` = as.numeric(ci.low < ybarU & ybarU < ci.upp)
  )

head(simulation.results)
```

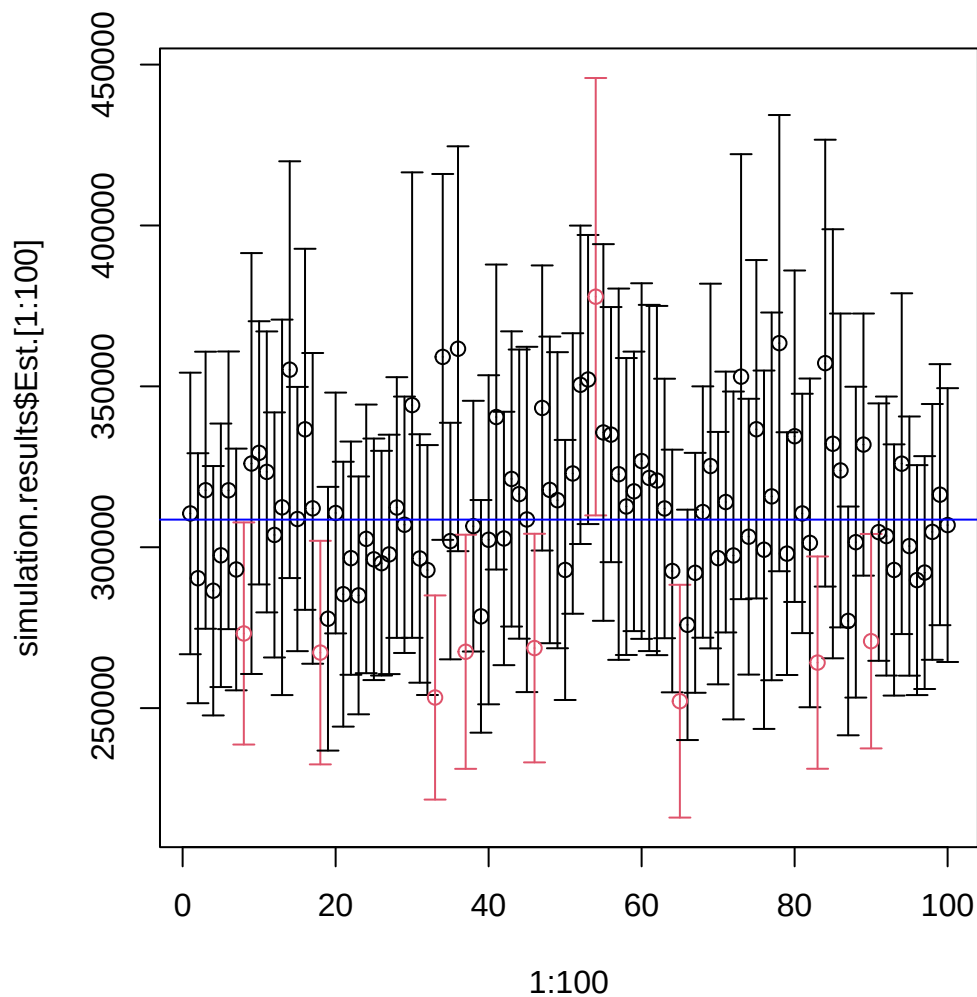
	Est.	S.E.	ci.low	ci.upp	Covered?
1	310499.5	22231.70	266749.0	354249.9	1
2	290362.7	19739.07	251517.5	329207.8	1
3	317716.8	21877.39	274663.7	360770.0	1

4	286476.8	19692.36	247723.6	325230.0	1
5	297487.1	20807.49	256539.4	338434.8	1
6	317689.7	21925.18	274542.5	360836.9	1

```

par(mfrow=c(1,1))
plotCI(
  x = 1:100,
  y = simulation.results$Est.[1:100],
  li = simulation.results$ci.low[1:100],
  ui = simulation.results$ci.upp[1:100],
  col = 2 - simulation.results$`Covered?`[1:100]
)
abline(h = ybarU, col = "blue")

```




```
# Empirical coverage rate  
mean(simulation.results$`Covered?`)
```

```
[1] 0.9324
```

3 Stratified Random Sampling

3.1 Key Formulas for Stratified Sampling

Stratified random sampling divides a population of size N into H mutually exclusive strata. A simple random sample of size n_h is then drawn from each stratum of size N_h . This design often yields higher precision than Simple Random Sampling (SRS).

1. Stratified Population Mean Estimator

The estimated population mean is a weighted average of the sample means from each stratum:

$$\bar{y}_{str} = \sum_{h=1}^H \pi_h \bar{y}_h.$$

Where $\pi_h = \frac{N_h}{N}$ is the stratum weight, and $\bar{y}_h$ is the sample mean of stratum h .

2. Variance of the Stratified Mean

The estimated variance accounts for the finite population correction in each stratum:

$$\hat{V}(\bar{y}_{str}) = \sum_{h=1}^H \pi_h^2 \left(1 - \frac{n_h}{N_h}\right) \frac{s_h^2}{n_h}.$$

Where s_h^2 is the sample variance in stratum h .

3. Proportional Allocation

Sample sizes are allocated proportional to the size of the stratum:

$$n_h = n \left(\frac{N_h}{N} \right).$$

county	state	acres92	acres87	acres82	farms92	farms87	farms82	largef92
PIERCE COUNTY	NE	297326	332862	319619	725	857	865	54
JENNINGS COUNTY	IN	124694	131481	139111	658	671	751	14
WAYNE COUNTY	OH	246938	263457	268434	1582	1734	1866	20
VAN BUREN COUNTY	MI	206781	190251	197055	1164	1278	1464	23
OZAUKEE COUNTY	WI	78772	85201	89331	448	483	527	6
CLEARWATER COUNTY	MN	210897	229537	213105	583	699	693	34

4. Neyman (Optimal) Allocation

If surveying costs are equal across strata, Neyman allocation minimizes the variance of the estimator by sampling more heavily from larger and more highly variable strata:

$$n_h = n \frac{N_h S_h}{\sum_{h=1}^H N_h S_h}$$

3.2 Functions and packages for Analyzing Data

The estimating functions used throughout this book (including `str_est`, `str_est_data`, and `srs_est`, used below to compute the stratified mean, standard error, and confidence intervals) live in a single shared file, `samplingestimate.r`, which we source below. `str_est_data` computes these directly from a dataset, while `str_est` computes them from pre-calculated summary statistics.

3.3 Analysis of “agstrat.csv” data

We will estimate the average 1992 farm acreage (`acres92`) across US counties, using geographic region (`region`) as the stratification variable.

3.3.1 Importing agstrat.csv data

```
agstrat <- read.csv("data/agstrat.csv")
head(agstrat) |> gt()
```

Output Note: The table above shows the sample survey data, including the stratum (region) and weights for each county.

	N_h	n_h	$\bar{y}_h$	s_h
NC	1054	103	300,504.16	172,099.34
NE	220	21	97,629.81	87,449.83
S	1382	135	211,315.04	231,489.71
W	422	41	662,295.51	629,433.04

3.3.2 Spreadsheet calculation

3.3.2.1 Summarizing acre92 in each stratum

We first manually compute the sample size (n_h), standard deviation (s_h), and mean ($\bar{y}_h$) for each region, matching them with the known total counties in each region (N_h).

```
nh <- tapply (agstrat[, "acres92"], agstrat[, "region"], length)
sh <- tapply (agstrat[, "acres92"], agstrat[, "region"], sd)
ybarh <- tapply (agstrat[, "acres92"], agstrat[, "region"], mean)
# create a vector with external information (true population size per stratum)
Nh <- c(NC = 1054, NE = 220, S = 1382, W = 422)

data.frame(Nh, nh, ybarh, sh) |>
  gt(rownames_to_stub = TRUE) |>
  cols_label(
    Nh = md("$N_h$"),
    nh = md("$n_h$"),
    ybarh = md("$\\bar{y}_h$"),
    sh = md("$s_h$")
  ) |>
  fmt_number(columns = c(ybarh, sh), decimals = 2)
```

Output Note: This displays the summary statistics per stratum. Note the significantly high variance (s_h) in the W region compared to the others.

3.3.2.2 Estimates

Rather than building the “spreadsheet-style” working table by hand, we call `str_est()`, whose `$table` shows the same intermediate stratum-level values (N_h , n_h , π_h , $\bar{y}_h$, s_h^2 , $\pi_h \bar{y}_h$, v_h) needed for the final stratified mean and variance, with a Total row and footnotes for π_h and v_h .

```
res_str_mean <- str_est(ybarh, sh, nh, Nh, estimate = "mean")
res_str_mean$table
```

Stratified Mean Estimation: Working Table

Stratum	N_h	n_h	π_h ¹	$\bar{y}_h$	s_h^2	$\pi_h \bar{y}_h$	
NC	1,054	103	0.34	3.01×10^5	2.96×10^{10}	1.03×10^5	3.0
NE	220	21	0.07	9.76×10^4	7.65×10^9	6.98×10^3	1.6
S	1,382	135	0.45	2.11×10^5	5.36×10^{10}	9.49×10^4	7.2
W	422	41	0.14	6.62×10^5	3.96×10^{11}	9.08×10^4	1.6
Total	3,078	300	1.00			2.96×10^5	2.68

¹ $\pi_h = N_h/N$ is the stratum weight.

² $v_h = (1 - n_h/N_h) \pi_h^2 s_h^2/n_h$ is stratum h 's contribution to $V(\bar{y}_{str}) = \sum_h v_h$.

Point estimate: $\bar{y}_{str} = \sum_h \pi_h \bar{y}_h = 2.956 \times 10^5$

Standard error: $SE(\bar{y}_{str}) = \sqrt{\sum_h v_h} = \sqrt{2.683 \times 10^8} = 16,379.873$

$\bar{y}$	$SE(\bar{y})$	95% CI Lower	95% CI Upper
295,560.7652	16,379.8727	263,456.2147	327,665.3158

Output Note: The table provides a detailed breakdown of how each individual stratum contributes to the final mean and variance.

Extracting `$estimate` gives our overall result.

```
res_str_mean$estimate |> format_est_gt("mean")
```

Output Note: The overall result indicates an estimated mean of 295,561 acres, with a 95% confidence interval of [263,456, 327,665].

To find the population total (total acreage across all US counties), set `estimate = "total"`.

```
## population total estimate
res_str_total <- str_est(ybarh, sh, nh, Nh, estimate = "total")
res_str_total$estimate |> format_est_gt("total")
```

Output Note: This scales the estimates up to the total area of 909,736,035 acres.

$\hat{t}$	$SE(\hat{t})$	95% CI Lower	95% CI Upper
909,736,035.3920	50,417,248.2519	810,918,228.8183	1,008,553,841.9656

Stratified Mean Estimation: Working Table

Stratum	N_h	n_h	π_h ¹	$\bar{y}_h$	s_h^2	$\pi_h \bar{y}_h$	
NC	1,054	103	0.34	3.01×10^5	2.96×10^{10}	1.03×10^5	3.0
NE	220	21	0.07	9.76×10^4	7.65×10^9	6.98×10^3	1.6
S	1,382	135	0.45	2.11×10^5	5.36×10^{10}	9.49×10^4	7.2
W	422	41	0.14	6.62×10^5	3.96×10^{11}	9.08×10^4	1.6
Total	3,078	300	1.00			2.96×10^5	2.68

¹ $\pi_h = N_h/N$ is the stratum weight.

² $v_h = (1 - n_h/N_h) \pi_h^2 s_h^2/n_h$ is stratum h 's contribution to $V(\bar{y}_{str}) = \sum_h v_h$.

Point estimate: $\bar{y}_{str} = \sum_h \pi_h \bar{y}_h = 2.956 \times 10^5$

Standard error: $SE(\bar{y}_{str}) = \sqrt{\sum_h v_h} = \sqrt{2.683 \times 10^8} = 16,379.873$

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
295,560.7652	16,379.8727	263,456.2147	327,665.3158

3.3.3 Using the function “str_est_data”

Instead of pre-calculating the summary stats, this function extracts everything directly from the dataset using the pre-assigned sampling weights.

```
## In the function "str_est_data", we can find the stratum size with the variable "weights":
tapply(agstrat$weight, agstrat$region, sum)
```

```
      NC      NE      S      W
1054  220 1382  422
```

Output Note: This reconstructs the total population sizes (N_h) per region.

```
## if the dataset contains a variable "weight"
res_agstrat_acres92 <- str_est_data(agstrat, y="acres92", stratum="region", weight="weight",
res_agstrat_acres92$table
```

```
res_agstrat_acres92$estimate |> format_est_gt("mean")
```

Output Note: Once again, this perfectly matches the manual calculation.

Stratified Mean Estimation: Working Table

Stratum	N_h	n_h	π_h ¹	$\bar{y}_h$	s_h^2	$\pi_h \bar{y}_h$
NC	1,054	103	0.34	44.26	1.29×10^3	15.16
NE	220	21	0.07	47.24	2.36×10^3	3.38
S	1,382	135	0.45	47.39	6.21×10^3	21.28
W	422	41	0.14	124.39	1.01×10^5	17.05
Total	3,078	300	1.00			56.86

¹ $\pi_h = N_h/N$ is the stratum weight.

² $v_h = (1 - n_h/N_h) \pi_h^2 s_h^2 / n_h$ is stratum h 's contribution to $V(\bar{y}_{str}) = \sum_h v_h$.

Point estimate: $\bar{y}_{str} = \sum_h \pi_h \bar{y}_h = 56.863$

Standard error: $SE(\bar{y}_{str}) = \sqrt{\sum_h v_h} = \sqrt{51.860} = 7.201$

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
56.8628	7.2014	42.7480	70.9776

```
## estimating the mean of the number of small farms in 1992
```

```
res_agstrat_smallf92 <- str_est_data(agstrat, y="smallf92", stratum="region", weight="weight")
res_agstrat_smallf92$table
```

```
res_agstrat_smallf92$estimate |> format_est_gt("mean")
```

Output Note: This demonstrates the function's flexibility: we can easily estimate a different variable (an average of 56.9 small farms per county).

3.3.4 Comparing with SRS estimate

To see the benefit of stratifying, we compare our stratified estimates against a Simple Random Sample approach on the same population.

```
agsrs <- read.csv("data/agsrs.csv")
res_agsrs_srs <- srs_est(agsrs[, "acres92"], N=3078, estimate = "mean", show.details = FALSE)
res_agsrs_srs$estimate |> format_est_gt("mean")
```

Output Note: The Simple Random Sample standard error is noticeably higher at 18,898.

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
297,897.0467	18,898.4344	260,706.2569	335,087.8365

```
## the ratio of estimated variance (Stratified SE / SRS SE)^2
ratio_var <- (res_agstrat_acres92$estimate["S.E."] / res_agstrs_srs$estimate["S.E."])^2
unnname(ratio_var)
```

```
[1] 0.7512239
```

Output Note: The stratified variance is approximately 75.1% of the SRS variance.

```
## percentage of reduction of variance of str estimates from that of SRS estimate
unnname(1 - ratio_var)
```

```
[1] 0.2487761
```

Output Note: Stratification by region reduced the variance by 24.9%. This is a substantial gain in precision for no extra sampling cost.

3.4 Allocation of stratum sample size

Deciding how to distribute the total sample n among the strata significantly impacts the precision of the estimator.

3.4.1 Analyzing seals.csv collected with stratified sampling

This dataset investigates ringed seal holes in different zones. The researchers used Proportional Allocation, taking 20% of the areas in each zone.

```
## load data
seals <- read.csv("data/seals.csv")

# survey data summary in each stratum
nh <- as.vector(table(seals[, "zone"]))
sh <- tapply(seals[, "holes"], seals[, "zone"], sd)
ybarh <- tapply(seals[, "holes"], seals[, "zone"], mean)
Nh <- c(68, 84, 48)
```


Stratified Mean Estimation: Working Table

Stratum	N_h	n_h	π_h ¹	$\bar{y}_h$	s_h^2	$\pi_h \bar{y}_h$
1	68	17	0.34	1.76	3.32	0.60
2	84	12	0.42	4.42	11.54	1.85
3	48	11	0.24	10.55	46.07	2.53
Total	200	40	1.00			4.99

¹ $\pi_h = N_h/N$ is the stratum weight.

² $v_h = (1 - n_h/N_h) \pi_h^2 s_h^2 / n_h$ is stratum h 's contribution to $V(\bar{y}_{str}) = \sum_h v_h$.

Point estimate: $\bar{y}_{str} = \sum_h \pi_h \bar{y}_h = 4.986$

Standard error: $SE(\bar{y}_{str}) = \sqrt{\sum_h v_h} = \sqrt{0.348} = 0.590$

$\bar{y}$	$SE(\bar{y})$	95% CI Lower	95% CI Upper
4.9859	0.5901	3.8292	6.1426

```
res_seals <- str_est(ybarh, sh, nh, Nh, estimate = "mean")
res_seals$table
```

Output Note: Zone 3 has the highest standard deviation ($s_h \approx 6.8$), indicating a high degree of variability in seal density in that region.

```
res_seals$estimate |> format_est_gt("mean")
```

Output Note: This provides the estimated mean number of seal holes per square kilometer (5).

3.4.2 Neyman allocation of stratum sample size

Because Zone 3 was highly variable, we could have achieved a lower overall variance if we had sampled it more heavily. Neyman allocation formally calculates this optimal distribution.

```
## assuming the cost of surveying each zone is the same
Ch <- rep(1, 3)
Sh <- sh ## estimate population Sh with sample sh
NhShDCh <- Nh * Sh / sqrt(Ch)
```

	C_h	N_h	S_h	$N_h S_h / \sqrt{C_h}$	L_h (Prop)
1	1	68	1.821	123.831	0.168
2	1	84	3.397	285.327	0.388
3	1	48	6.788	325.809	0.443

```
## the optimal allocation proportion (Lh) for estimating the population total or mean:
Lh <- NhShDCh / sum(NhShDCh)

data.frame(Ch, Nh, Sh, NhShDCh, Lh) |>
  gt(rownames_to_stub = TRUE) |>
  cols_label(NhShDCh = md("$N_h S_h / \\sqrt{C_h}$"), Lh = md("$L_h$ (Prop)"), Ch=md("$C_h$"),
  fmt_number(columns = c(Sh, NhShDCh, Lh), decimals = 3)
```

Output Note: This displays the optimal allocation proportions (L_h). Even though Zone 2 is a larger area ($N_h = 84$), Zone 3's high variance means we should allocate the largest share (44%) of our sampling effort there.

3.5 Simulation to Study the Efficiency of Stratified Sampling with Different Allocation

We will use the full `agpop.csv` dataset (acting as our true population) to run 2000 repeated samples. We will compare SRS, Proportional (P2S), and Optimal (Neyman) allocations.

3.5.1 Using “Region” to Stratify

3.5.1.1 Read Population Data

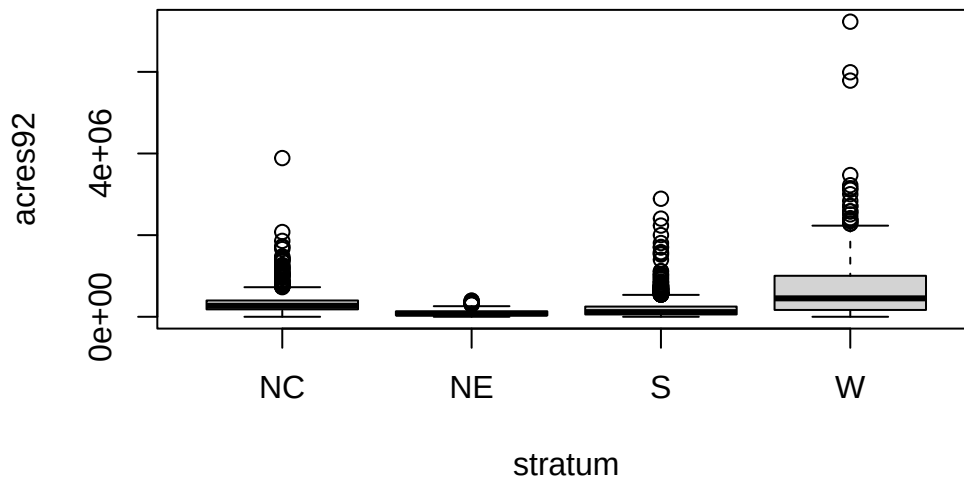
```
agpop <- read.csv("data/agpop.csv")
agpop <- agpop[agpop$acres92 != -99, ] ## remove those counties with na
N <- nrow(agpop)
```

3.5.1.2 Define Stratum Variable

```
agpop$stratum <- agpop$region
## reorder agpop for the ease of using strata of "sampling" (very important)
agpop <- agpop[order(agpop$stratum), ]
```

A stratification variable is most effective when it strongly explains the variance in the target variable. We check this via ANOVA and R-squared.

```
boxplot(acres92 ~ stratum, data = agpop)
```



```
lm_region <- lm(agpop$acres92 ~ agpop$stratum)
anova(lm_region)
```

Analysis of Variance Table

```
Response: agpop$acres92
          Df    Sum Sq   Mean Sq F value    Pr(>F)
agpop$stratum  3 1.0073e+14 3.3578e+13  226.73 < 2.2e-16 ***
Residuals    3055 4.5243e+14 1.4810e+11
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
lm_region_gt <- format_lm_gt(lm_region)
lm_region_gt$fit
```

Model Fit Summary

R^2	Adj. R^2	$\hat{\sigma}$	F-statistic	p-value
0.1821	0.1813	384,831.7694	226.7299	0.0000

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
316,484.2928	20,295.9027	276,704.3235	356,264.2621

```
r_squared_region <- summary(lm_region)$r.squared
```

Output Note: The R-squared is 0.182. The region variable only explains about 18.2% of the variance in acreage, which is helpful but not extremely strong.

3.5.1.3 Stratified Sampling with P2S allocation (Single Run)

```
# doing one stratified sampling
Nh <- tapply(1:nrow(agpop), agpop$stratum, length)
nh <- round(Nh/sum(Nh)*300)

## doing stratified sampling
strsample <- strata(agpop, "stratum", size = nh, method = "srswor")

# collecting data on sampled counties
agstrat <- agpop[strsample$ID_unit, ]
agstrat$weight <- 1/strsample$Prob

str_est_data(agstrat, "acres92", "stratum", "weight", estimate = "mean")$estimate |>
  format_est_gt("mean")
```

Output Note: This provides a single point estimate for demonstration.

3.5.1.4 Repeat stratified sampling with P2S allocation 2000 times

```
Nh <- tapply(1:nrow(agpop), agpop$stratum, length)
nh <- round(Nh/sum(Nh)*300) ## make sure the order matches Nh
```

	Nh	Sh	nh_opt
NC	1052	271,187.98	87
NE	213	78,906.20	5
S	1376	244,131.98	102
W	418	836,613.56	106

```
nres <- 2000
str_p2s_simulated <- matrix(0, nres, 4)
for (i in 1:nres) {
  strsample <- strata(agpop, "stratum", size = nh, method = "srswor")
  agstrat <- agpop[strsample$ID_unit, ]
  agstrat$weight <- 1/strsample$Prob
  str_p2s_simulated[i,] <- str_est_data(agstrat, "acres92", "stratum", "weight", estimate = )
}
```

3.5.1.5 Repeat stratified sampling with optimal allocation 2000 times

```
Nh <- tapply(agpop$acres92, agpop$stratum, length)
Sh <- tapply(agpop$acres92, agpop$stratum, sd)
nh_opt <- round((Nh*Sh)/sum(Nh*Sh) * 300)

data.frame(Nh, Sh, nh_opt) |>
  gt(rownames_to_stub = TRUE) |>
  fmt_number(columns = Sh, decimals = 2)
```

Output Note: This table compares the population sizes to the optimal sample sizes. Notice that the West region (high variance) gets a disproportionately larger sample under Neyman allocation.

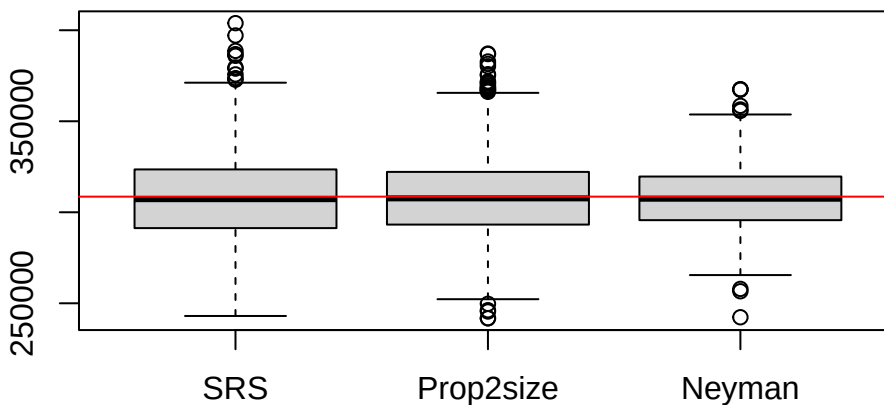
```
nres <- 2000
str_neyman_simulated <- matrix(0, nres, 4)
for (i in 1:nres) {
  strsample <- strata(agpop, "stratum", size = nh_opt, method = "srswor")
  agstrat <- agpop[strsample$ID_unit, ]
  agstrat$weight <- 1/strsample$Prob
  str_neyman_simulated[i,] <- str_est_data(agstrat, "acres92", "stratum", "weight", estimate = )
}
```

3.5.1.6 Repeat simple random sampling 2000 times

```
nres <- 2000
srs_simulated <- matrix(0, nres, 4)
for (i in 1:nres) {
  srs <- sample(sum(Nh), sum(nh))
  agsrs <- agpop[srs, ]
  srs_simulated[i,] <- srs_est(agsrs[, "acres92"], N = sum(Nh), estimate = "mean", show.de
```

3.5.1.7 Compare the efficiency of different methods

```
sim_results_str_region <- data.frame(
  "SRS" = srs_simulated[,1],
  "Prop2size" = str_p2s_simulated[,1],
  "Neyman" = str_neyman_simulated[,1]
)
boxplot(sim_results_str_region)
abline(h = mean(agpop$acres92), col = "red")
```



Output Note: The plot visually demonstrates that Neyman allocation has the tightest spread around the true mean (the red line).

```
sim_means <- sapply(sim_results_str_region, mean)
sim_var <- sapply(sim_results_str_region, var)
sim_var_relative <- sim_var / sim_var[1]
sim_var_reduction <- 1 - sim_var_relative
```

	Mean	Variance	Relative Variance	Percentage of Variance Reduction
SRS	307,988.7	554,796,280.9	1.000	0.00%
Prop2size	308,478.2	459,389,900.3	0.828	17.20%
Neyman	307,962.4	294,203,573.4	0.530	46.97%

```
data.frame(
  "Mean" = sim_means,
  "Variance" = sim_var,
  "Relative Variance" = sim_var_relative,
  "Percentage of Variance Reduction" = sim_var_reduction,
  check.names = FALSE
) |>
  gt(rownames_to_stub = TRUE) |>
  fmt_number(columns = c(Mean, Variance), decimals = 1) |>
  fmt_percent(columns = c(`Percentage of Variance Reduction`), decimals = 2) |>
  fmt_number(columns = c(`Relative Variance`), decimals = 3)
```

Output Note: The table summarizes the simulation. Proportional allocation reduces variance by 17.2%, while Neyman allocation reduces it by 47% compared to Simple Random Sampling.

3.5.2 Using “acres82” to Define Strata

What if we stratify by a variable highly correlated with our target, like the acreage from a decade prior (acres82)?

3.5.2.1 Read Population Data

```
agpop <- read.csv("data/agpop.csv")
agpop <- agpop[agpop$acres92 != -99, ]
N <- nrow(agpop)
```

3.5.2.2 Define Stratum Variable with Quantiles of “acres82”

We convert the continuous acres82 variable into four distinct categorical strata based on quantiles.

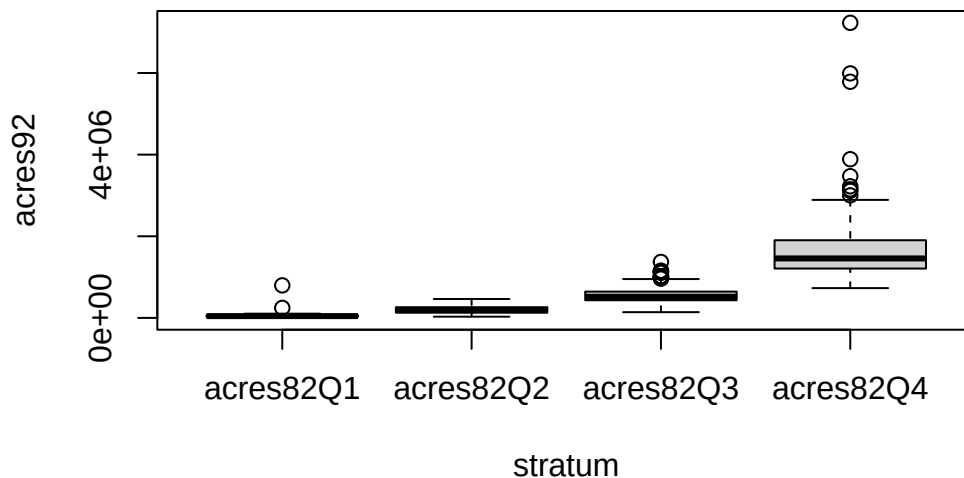
Model Fit Summary

R^2	Adj. R^2	$\hat{\sigma}$	F-statistic	p-value
0.7393	0.7390	217,264.2290	2,887.8873	0.0000

```
agpop$stratum <- cut(agpop$acres82,  
                     breaks = quantile(agpop$acres82, probs = c(0,0.25,0.75,0.95,1)),  
                     include.lowest = T)  
levels(agpop$stratum) <- paste0("acres82", c("Q1", "Q2", "Q3", "Q4"))  
agpop <- agpop[order(agpop$stratum), ]
```

Let's check the explanatory power of this new stratification variable.

```
boxplot(acres92 ~ stratum, data = agpop)
```



```
lm_acres82 <- lm(agpop$acres92 ~ agpop$stratum)  
lm_acres82_gt <- format_lm_gt(lm_acres82)  
lm_acres82_gt$fit
```

```
r_squared_acres82 <- summary(lm_acres82)$r.squared
```

Output Note: The R-squared is 0.739. The past acreage explains 73.9% of the variance in 1992 acreage, making this a vastly superior stratification variable.

3.5.2.3 Stratified sampling with P2S allocation

```
Nh <- tapply(1:nrow(agpop), agpop$stratum, length)
nh <- round(Nh/sum(Nh)*300)

nres <- 2000
str_p2s_simulated <- matrix(0, nres, 4)
for (i in 1:nres) {
  strsample <- strata(agpop, "stratum", size = nh, method = "srswor")
  agstrat <- agpop[strsample$ID_unit, ]
  agstrat$weight <- 1/strsample$Prob
  str_p2s_simulated[i,] <- str_est_data(agstrat, "acres92", "stratum", "weight", estimate = "mean", show.details = FALSE)
}
```

3.5.2.4 Repeat stratified sampling with optimal allocation 2000 times

```
Nh <- tapply(1:nrow(agpop), agpop$stratum, length)
Sh <- tapply(agpop$acres92, agpop$stratum, sd)
nh_opt <- round((Nh*Sh)/sum(Nh*Sh) * 300)

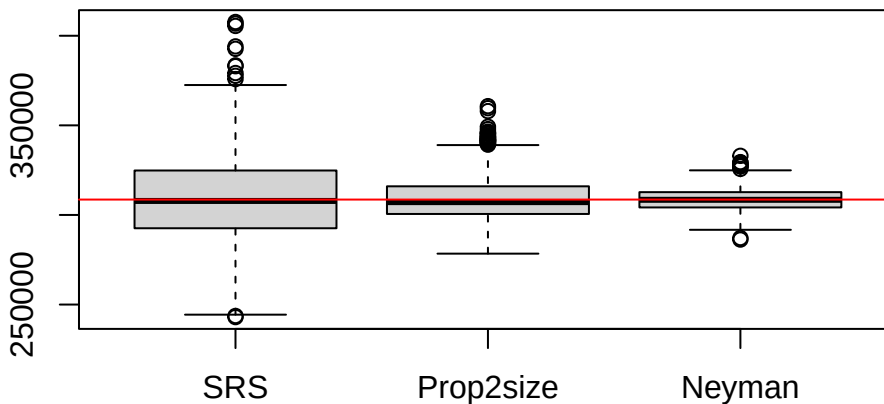
nres <- 2000
str_neyman_simulated <- matrix(0, nres, 4)
for (i in 1:nres) {
  strsample <- strata(agpop, "stratum", size = nh_opt, method = "srswor")
  agstrat <- agpop[strsample$ID_unit, ]
  agstrat$weight <- 1/strsample$Prob
  str_neyman_simulated[i,] <- str_est_data(agstrat, "acres92", "stratum", "weight", estimate = "mean", show.details = FALSE)
}
```

3.5.2.5 Repeat simple random sampling 2000 times

```
nres <- 2000
srs_simulated <- matrix(0, nres, 4)
for (i in 1:nres) {
  srs <- sample(sum(Nh), sum(nh))
  agsrs <- agpop[srs, ]
  srs_simulated[i,] <- srs_est(agsrs[, "acres92"], N = sum(Nh), estimate = "mean", show.details = FALSE)
}
```

3.5.2.6 Compare the efficiency of different methods

```
sim_results_str_acres82 <- data.frame(  
  "SRS" = srs_simulated[,1],  
  "Prop2size" = str_p2s_simulated[,1],  
  "Neyman" = str_neyman_simulated[,1]  
)  
boxplot(sim_results_str_acres82)  
abline(h = mean(agpop$acres92), col = "red")
```



Output Note: The boxplot visually highlights a massive reduction in variance compared to SRS, especially for the Neyman allocation.

```
sim_means <- sapply(sim_results_str_acres82, mean)  
sim_var <- sapply(sim_results_str_acres82, var)  
sim_var_relative <- sim_var / sim_var[1]  
sim_var_reduction <- 1 - sim_var_relative  
  
data.frame(  
  "Mean" = sim_means,  
  "Variance" = sim_var,  
  "Relative Variance" = sim_var_relative,  
  "Percentage of Variance Reduction" = sim_var_reduction,  
  check.names = FALSE  
) |>  
gt(rownames_to_stub = TRUE) |>  
fmt_number(columns = c(Mean, Variance), decimals = 1) |>  
fmt_percent(columns = c(`Percentage of Variance Reduction`), decimals = 2) |>  
fmt_number(columns = c(`Relative Variance`), decimals = 3)
```

	Mean	Variance	Relative Variance	Percentage of Variance Reduction
SRS	309,203.9	557,462,904.0	1.000	0.00%
Prop2size	308,567.8	141,549,774.8	0.254	74.61%
Neyman	308,470.4	38,760,083.2	0.070	93.05%

Output Note: The table shows dramatic results: Proportional allocation reduces variance by 74.6%, and Neyman allocation reduces it by 93%. Stratifying by a highly correlated variable is immensely powerful.

4 Ratio, Regression, and Domain Estimation

Key formulae

When auxiliary information (a variable x) is highly correlated with our target variable of interest (y), we can use **Ratio** or **Regression** estimation to improve precision without increasing the sample size. We can also use auxiliary population structures to adjust our estimates after the fact via **Post-stratification**, or estimate means for specific sub-populations (**Domains**).

1. Ratio Estimation

If the relationship between y and x passes through the origin, we estimate the ratio B and use it to adjust the mean:

- **Ratio:** $\hat{B} = \frac{\bar{y}}{\bar{x}}$
- **Mean Estimator:** $\bar{y}_r = \hat{B}\bar{x}_U$ (where $\bar{x}_U$ is the true population mean of x)
- **Estimated Variance:** $\hat{V}(\bar{y}_r) = \left(\frac{\bar{x}_U}{\bar{x}}\right)^2 \left(1 - \frac{n}{N}\right) \frac{s_e^2}{n}$, where s_e^2 is the sample variance of the residuals $e_i = y_i - \hat{B}x_i$.

2. Regression Estimation

If the relationship does not pass through the origin, a linear regression is more appropriate:

- **Mean Estimator:** $\bar{y}_{reg} = \hat{B}_0 + \hat{B}_1\bar{x}_U$ (where $\hat{B}_0$ and $\hat{B}_1$ are OLS coefficients)
- **Estimated Variance:** $\hat{V}(\bar{y}_{reg}) = \left(1 - \frac{n}{N}\right) \frac{s_e^2}{n}$, where s_e^2 is the Mean Square Error (Residuals) from the regression.

3. Domain Estimation

To estimate the mean of a subgroup (domain d) when the domain sample size n_d is a random variable:

- **Mean Estimator:** $\bar{y}_d = \frac{1}{n_d} \sum_{i \in U_d} y_i$

- **Estimated Variance:** $\hat{V}(\bar{y}_d) = (1 - \frac{n}{N}) \frac{s_d^2}{n_d}$, where n is the overall sample size, N is the population size, and s_d^2 is the sample variance of y among the n_d units observed in domain d .

4. Post-stratification

If we know the true population sizes (N_h) of strata, but we used a Simple Random Sample, we can post-stratify to reduce variance. We substitute n_h with expected proportional sample sizes:

- $n_h^{post-strat} = n \frac{N_h}{N}$

4.1 Functions and packages for Analyzing Data

We begin by loading the necessary libraries, including `gt` for formatting mathematical tables, and defining our custom functions for Ratio, Regression, Domain, and Post-stratified estimation. We also define a versatile `gt` helper to render our estimation outputs cleanly.

4.1.1 General functions for ratio and regression estimation

The estimating functions used throughout this book (including `reg_est`, `ratio_est`, `domain_est`, `srs_est`, and the `format_est_gt` table-formatting helper used below) live in a single shared file, `samplingestimate.r`, which we source below.

For post-stratification, we reuse the shared `str_est()` stratified estimator, but we plug in the expected sample sizes $n_h = n \times \frac{N_h}{N}$ instead of the actual observed random n_h .

4.2 Ratio Estimation for cherry.csv dataset

We want to estimate the volume of wood in cherry trees using their diameter as an auxiliary variable.

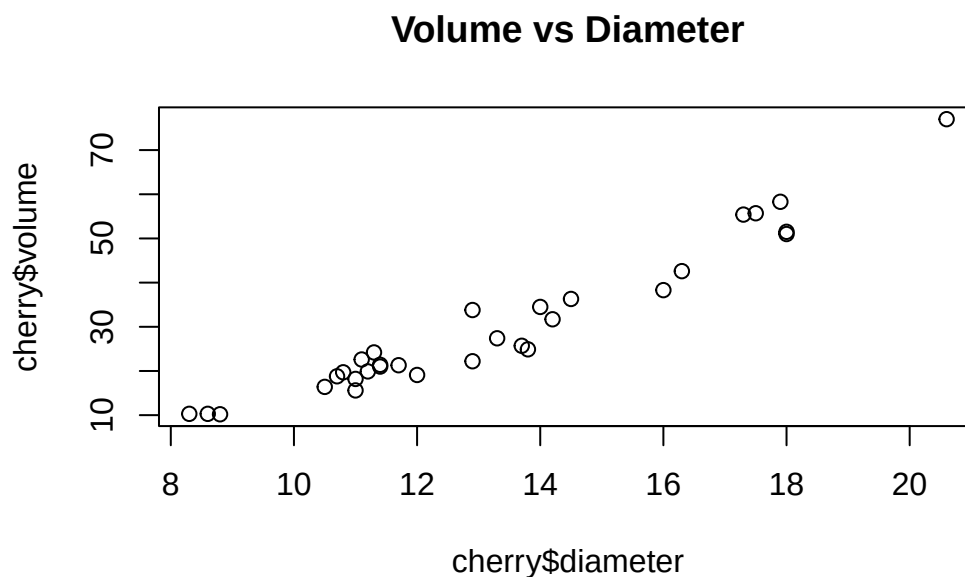
4.2.1 Importing data

First, let's load the data and verify that a strong linear relationship passing near the origin exists between volume and diameter.

diameter	height	volume
8.3	70	10.3
8.6	65	10.3
8.8	63	10.2
10.5	72	16.4
10.7	81	18.8
10.8	83	19.7

```
cherry <- read.csv("data/cherry.csv", header = T)
head(cherry) |> gt()
```

```
plot(cherry$volume ~ cherry$diameter, main = "Volume vs Diameter")
```



```
anova(lm(cherry$volume ~ 0 + cherry$diameter))
```

Analysis of Variance Table

Response: cherry\$volume

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
cherry\$diameter	1	33622	33622	373.12	< 2.2e-16 ***
Residuals	30	2703	90		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
30.1710	2.9369	24.1731	36.1688

$\hat{t}$	SE($\hat{t}$)	95% CI Lower	95% CI Upper
89,517.2613	8,713.6652	71,721.5828	107,312.9398

Output Note: The plot and the linear model output (which forces a 0 intercept) confirm a very strong relationship, making this a prime candidate for Ratio estimation.

4.2.2 SRS estimate

To establish a baseline, we first estimate the mean and total volume using standard Simple Random Sampling (ignoring the auxiliary variable).

```
N <- 2967
## estimating the mean of volume
res_srs_vol <- srs_est(cherry$volume, N = N, estimate = "mean", show.details = FALSE)
srs_mean_vol_raw <- res_srs_vol$estimate
srs_mean_vol_raw |> format_est_gt("mean")

## estimating the total of volume
res_srs_total_vol <- srs_est(cherry$volume, N = N, estimate = "total", show.details = FALSE)
srs_total_vol_raw <- res_srs_total_vol$estimate
srs_total_vol_raw |> format_est_gt("total")
```

Output Note: Under SRS, the standard error for the mean volume is 2.94.

4.2.3 Ratio Estimation Using the Function

We calculate the ratio $\hat{B} = \bar{y}/\bar{x}$; the function's `$table` shows the row-level working values x_i , y_i , fitted $\hat{y}_i = \hat{B}x_i$, residuals $e_i = y_i - \hat{y}_i$, and e_i^2 (with a Sum row and a footnote giving $\hat{B}$ and s_e^2), so we no longer need to build this table by hand.

```
ydata <- cherry$volume
xdata <- cherry$diameter
N <- 2967

res_ratio_model <- ratio_est(ydata, xdata, N = N, estimate = "model")
res_ratio_model$table
```

Ratio Estimation: Working Table

$n = 31$

i	y_i	x_i	$\hat{y}_i^1$	e_i	e_i^2
1	10.300	8.300	18.902	-8.602	73.992
2	10.300	8.600	19.585	-9.285	86.212
3	10.200	8.800	20.041	-9.841	96.836
4	16.400	10.500	23.912	-7.512	56.430
5	18.800	10.700	24.367	-5.567	30.996
6	19.700	10.800	24.595	-4.895	23.963
...	...	...	...	...	...
Sum	935.300	410.700	935.300	3.553 × 10⁻¹⁴	2,821.586

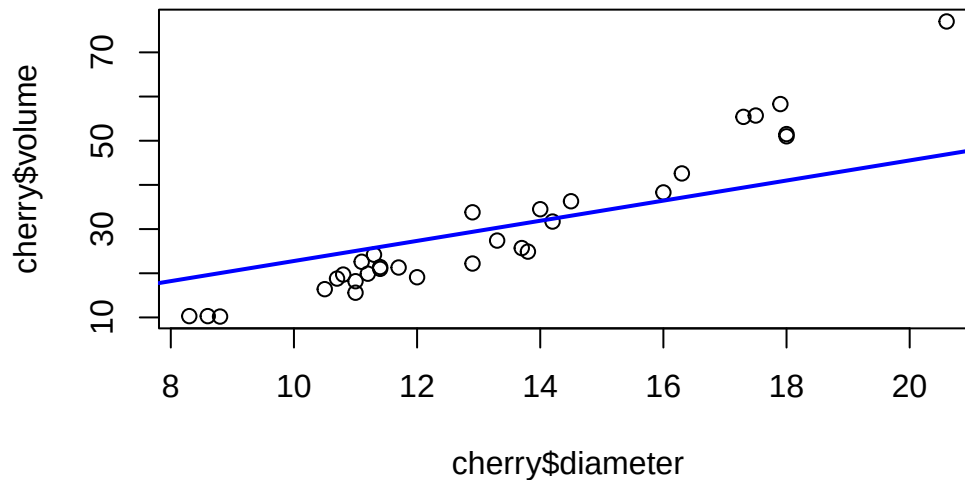
$$^1\hat{B} = \sum_i y_i / \sum_i x_i = 2.2773, \hat{y}_i = \hat{B} x_i, s_e^2 = 94.0529.$$

$$\text{Point estimate: } \hat{B} = \frac{\sum_i y_i}{\sum_i x_i} = \frac{935.300}{410.700} = 2.277$$

$$\text{Standard error: } \text{SE}(\hat{B}) = \frac{1}{\bar{x}} \sqrt{\left(1 - \frac{n}{N}\right) \frac{s_e^2}{n}} = \frac{1}{13.248} \sqrt{\left(1 - \frac{31}{2967}\right) \frac{94.053}{31}} = 0.131$$

```
plot(cherry$volume ~ cherry$diameter, main="Ratio Estimation Fit")
abline(a = 0, b = res_ratio_model$estimate["Est."], col="blue", lwd=2)
```

Ratio Estimation Fit



Output Note: The blue line represents our estimated ratio $\hat{B} \approx 2.28$. The working table

$\hat{B}$	SE($\hat{B}$)	95% CI Lower	95% CI Upper
2.2773	0.1308	2.0102	2.5444

Ratio Estimation: Working Table

$n = 31$

i	y_i	x_i	$\hat{y}_i$ ¹	e_i	e_i^2
1	10.300	8.300	18.902	-8.602	73.992
2	10.300	8.600	19.585	-9.285	86.212
3	10.200	8.800	20.041	-9.841	96.836
4	16.400	10.500	23.912	-7.512	56.430
5	18.800	10.700	24.367	-5.567	30.996
6	19.700	10.800	24.595	-4.895	23.963
...	...	...	...	...	...
Sum	935.300	410.700	935.300	3.553 × 10⁻¹⁴	2,821.586

¹ $\hat{B} = \sum_i y_i / \sum_i x_i = 2.2773$, $\hat{y}_i = \hat{B} x_i$, $s_e^2 = 94.0529$.

Point estimate: $\bar{y}_r = \hat{B} \bar{x}_U = 2.277 \times 14.100 = 32.111$

Standard error: $SE(\bar{y}_r) = SE(\hat{B}) \bar{x}_U = 0.131 \times 14.100 = 1.844$

above shows the individual residuals used to estimate s_e^2 .

```
B_v2d <- res_ratio_model$estimate
B_v2d |> format_est_gt("ratio")
```

Output Note: The ratio $\hat{B}$ is estimated at 2.277 with a standard error of 0.1308.

4.2.3.1 Estimating the mean volume of wood

If we know the true total of the population diameters is 41,835, we can derive the population mean diameter $\bar{x}_U$ and pass it as `xbarU` with `estimate = "mean"`.

```
mean_diameters <- 41835 / N
res_ratio_mean <- ratio_est(ydata, xdata, xbarU = mean_diameters, N = N, estimate = "mean")
res_ratio_mean$table
```

$\bar{y}_r$	SE($\bar{y}_r$)	95% CI Lower	95% CI Upper
32.1106	1.8441	28.3445	35.8768

Ratio Estimation: Working Table

$n = 31$

i	y_i	x_i	$\hat{y}_i$ ¹	e_i	e_i^2
1	10.300	8.300	18.902	-8.602	73.992
2	10.300	8.600	19.585	-9.285	86.212
3	10.200	8.800	20.041	-9.841	96.836
4	16.400	10.500	23.912	-7.512	56.430
5	18.800	10.700	24.367	-5.567	30.996
6	19.700	10.800	24.595	-4.895	23.963
...	...	...	...	...	...
Sum	935.300	410.700	935.300	3.553 × 10⁻¹⁴	2,821.586

¹ $\hat{B} = \sum_i y_i / \sum_i x_i = 2.2773$, $\hat{y}_i = \hat{B} x_i$, $s_e^2 = 94.0529$.

Point estimate: $\hat{t} = \hat{B} \bar{x}_U N = 2.277 \times 14.100 \times 2,967 = 95,272.159$

Standard error: $SE(\hat{t}) = SE(\hat{B}) \bar{x}_U N = 0.131 \times 14.100 \times 2,967 = 5,471.434$

```
ratio_mean_volume_raw <- res_ratio_mean$estimate
ratio_mean_volume_raw |> format_est_gt("ratio_mean")
```

4.2.3.2 Estimating the total volume of wood

Using `estimate = "total"` with the known population total diameter gives the estimated population total volume directly.

```
t_diameters <- 41835
```

```
res_ratio_total <- ratio_est(ydata, xdata, xbarU = t_diameters / N, N = N, estimate = "total")
res_ratio_total$table
```

```
ratio_total_volume <- res_ratio_total$estimate
ratio_total_volume |> format_est_gt("total")
```

$\hat{t}$	$SE(\hat{t})$	95% CI Lower	95% CI Upper
95,272.1585	5,471.4344	84,097.9988	106,446.3182

4.2.3.3 Percentage of Variance Reduction

How much did ratio estimation help compared to ignoring the auxiliary variable?

```
variance_reduction <- 1 - (ratio_mean_volume_raw["S.E."] / srs_mean_vol_raw["S.E."])^2
unnname(variance_reduction)
```

```
[1] 0.6057237
```

Output Note: Using the diameter as an auxiliary variable reduced the variance of our estimate by 60.6% compared to simple random sampling.

4.3 Simulation study with agpop.csv

Let's test this practically by drawing 2000 random samples and seeing how SRS and Ratio estimators behave under repeated sampling.

4.3.1 Information of population data

```
agpop <- read.csv("data/agpop.csv")
agpop <- agpop[agpop$acres92 != -99, ] ## remove those counties with na

# sample size
n <- 300
# population size
N <- nrow(agpop)

# true values that we want to estimate
tyU <- sum(agpop[, "acres92"])
# suppose known for ratio estimate
txU <- sum(agpop[, "acres87"])
B <- tyU / txU

plot(agpop[, "acres87"], agpop[, "acres92"], main="Acres 1992 vs Acres 1987")
abline(a = 0, b = B, col="red")
```

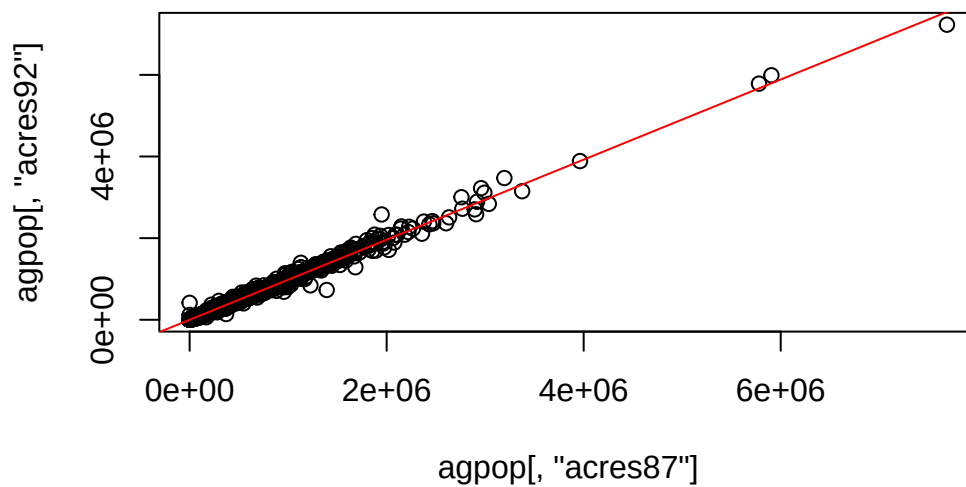
Coefficient Estimates

	Estimate	S.E.	t value	p value
(Intercept)	-2,121.4881	864.7615	-2.4533	0.0142
agpop\$acres87	0.9878	0.0016	607.3876	0.0000

Model Fit Summary

R^2	Adj. R^2	$\hat{\sigma}$	F-statistic	p-value
0.9918	0.9918	38,562.8752	368,919.6409	0.0000

Acres 1992 vs Acres 1987



```
# linear model output
lm_gt <- format_lm_gt(lm(agpop$acres92 ~ agpop$acres87))
lm_gt$coef
```

```
lm_gt$fit
```

```
# expected reduction of variance of ratio estimate to srs
exp_var_red <- 1 - var(agpop$acres92 - B * agpop$acres87) / var(agpop$acres92)
exp_var_red
```

```
[1] 0.9917355
```

Output Note: Past acreage (1987) strongly predicts current acreage (1992). The theoretically expected variance reduction is roughly 99.2%.

4.3.2 Simulation Studies

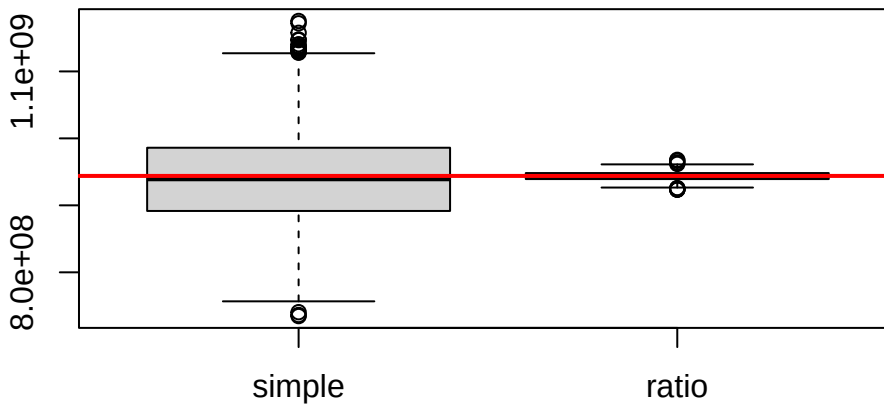
```
nsim <- 2000
sim_rat <- data.frame(simple = rep(0, nsim), ratio = rep(0, nsim), B = rep(0, nsim))

for (i in 1:nsim) {
  srs <- sample(N, n)
  y_srs <- agpop[srs, "acres92"]
  x_srs <- agpop[srs, "acres87"]

  # SRS estimate (Total)
  sim_rat$simple[i] <- mean(y_srs) * N
  # ratio estimate (Total)
  sim_rat$B[i] <- mean(y_srs) / mean(x_srs)
  sim_rat$ratio[i] <- sim_rat$B[i] * txU
}

boxplot(sim_rat[, 1:2], main="Estimator Variance: SRS vs Ratio")
abline(h = tyU, col = "red", lwd=2)
```

Estimator Variance: SRS vs Ratio



We now calculate the actual variance across our 2000 simulations.

	Variance	Relative Variance	Variance Reduction
simple	4,937,600,675,673,005	1.0000	0.00%
ratio	42,196,143,537,277	0.0085	99.15%

```
sim_var <- sapply(sim_rat[, 1:2], var)
ratio_sim_var <- sim_var / sim_var[1]
percentage_reduction <- 1 - ratio_sim_var

data.frame(sim_var, ratio_sim_var, percentage_reduction) |>
  gt(rownames_to_stub = TRUE) |>
  cols_label(
    sim_var = "Variance",
    ratio_sim_var = "Relative Variance",
    percentage_reduction = "Variance Reduction"
  ) |>
  fmt_number(columns = c(sim_var), decimals = 0) |>
  fmt_percent(columns = c(percentage_reduction), decimals = 2) |>
  fmt_number(columns = c(ratio_sim_var), decimals = 4)
```

Output Note: The boxplot dramatically illustrates the precision gain. The variance of the ratio estimator is only 0.9% of the SRS variance, confirming our theoretical expectations.

4.4 Estimating Domain Means and Post-stratification with agsrs.csv example

4.4.1 Estimating Domain Means

When estimating the mean for a specific subgroup (like a region), the sample size in that subgroup (n_d) is a random variable, which increases our uncertainty.

```
agsrs <- read.csv("data/agsrs.csv")
n <- nrow(agsrs)

## estimating domain means
region_agsrs_domain <- data.frame(rbind(
  NC = domain_est(agsrs$acres92[agsrs$region=="NC"], n=n, N = 3078),
  NE = domain_est(agsrs$acres92[agsrs$region=="NE"], n=n, N = 3078),
  S = domain_est(agsrs$acres92[agsrs$region=="S"], n=n, N = 3078),
```

	$\bar{y}_d$	SE($\bar{y}_d$)	95% CI Lower	95% CI Upper
NC	323,416.4359	30,008.3469	263,662.1833	383,170.6885
NE	106,549.0000	28,957.5582	45,453.8926	167,644.1074
S	246,185.8375	23,503.6178	199,766.2813	292,605.3937
W	518,977.6364	70,298.6900	377,206.8165	660,748.4562

	$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
NC	323,416.4359	30,395.8898	262,890.4867	383,942.3851
NE	106,549.0000	29,207.5056	44,926.5496	168,171.4504
S	246,185.8375	23,264.0052	200,239.5154	292,132.1596
W	518,977.6364	70,033.3811	377,741.8630	660,213.4097

```
W = domain_est(agsrs$acres92[agsrs$region=="W"], n=n, N = 3078)
))

region_agsrs_domain |> format_est_gt("domain_mean")
```

If we incorrectly used the standard SRS formula (pretending n_d was fixed), our standard errors would be artificially low:

```
## naively applying SRS estimation to each domain (incorrect analysis)
region_agsrs_srs <- data.frame(rbind(
  NC = srs_est(agsrs$acres92[agsrs$region=="NC"], N = 1054, estimate = "mean", show.detail=TRUE),
  NE = srs_est(agsrs$acres92[agsrs$region=="NE"], N = 220, estimate = "mean", show.detail=TRUE),
  S = srs_est(agsrs$acres92[agsrs$region=="S"], N = 1382, estimate = "mean", show.detail=TRUE),
  W = srs_est(agsrs$acres92[agsrs$region=="W"], N = 422, estimate = "mean", show.detail=TRUE)
))

region_agsrs_srs |> format_est_gt("mean")
```

Output Note: Comparing the two tables, you can see the correct `domain_mean` Standard Errors are larger, appropriately penalizing us for the uncertainty in how many counties from each region randomly fell into our sample.

4.4.2 Post-stratification Analysis

If we know the true regional population sizes (N_h), we can post-stratify our SRS data.

	$\bar{y}_h$	s_h	n_h^{obs}	π_h^{obs}	N_h	π_h^{pop}	n_h^{post}
NC	323,416.436	278,970.037	78	0.260	1054	0.342	102.729
NE	106,549.000	129,320.203	18	0.060	220	0.071	21.442
S	246,185.837	312,941.310	160	0.533	1382	0.449	134.698
W	518,977.636	490,842.046	44	0.147	422	0.137	41.131

```
nh <- tapply(agsrs[, "acres92"], agsrs[, "region"], length)
n <- sum(nh)
sh <- tapply(agsrs[, "acres92"], agsrs[, "region"], sd)
ybarh <- tapply(agsrs[, "acres92"], agsrs[, "region"], mean)
# create a vector with external information
Nh <- c(NC = 1054, NE = 220, S = 1382, W = 422)
```

Create $n_h^{\text{post-strat}} = n \times \frac{N_h}{N}$, instead of using the randomly observed n_h .

```
nh_post <- Nh / sum(Nh) * n

## regional summary for stratified sampling estimation
data.frame(
  ybarh = ybarh,
  sh = sh,
  nh = nh,
  pi_obs = nh/n,
  Nh = Nh,
  pi_pop = Nh/sum(Nh),
  nh_post = nh_post
) |>
gt(rownames_to_stub = TRUE) |>
cols_label(
  ybarh = md("$\\bar{y}_h$"),
  sh = md("$s_h$"),
  nh = md("$n_h^{\\text{obs}}$"),
  pi_obs = md("$\\pi^{\\text{obs}}_h$"),
  Nh = md("$N_h$"),
  pi_pop = md("$\\pi^{\\text{pop}}_h$"),
  nh_post = md("$n_h^{\\text{post}}$")
) |>
fmt_number(columns = c(ybarh, sh, pi_obs, pi_pop, nh_post), decimals = 3)
```

Output Note: The table above shows the difference between the sample proportion (π_h^{obs})

Stratified Mean Estimation: Working Table

Stratum	N_h	n_h	π_h ¹	$\bar{y}_h$	s_h^2	$\pi_h \bar{y}_h$	
NC	1,054	103	0.34	3.23×10^5	7.78×10^{10}	1.11×10^5	8.02
NE	220	21	0.07	1.07×10^5	1.67×10^{10}	7.62×10^3	3.60
S	1,382	135	0.45	2.46×10^5	9.79×10^{10}	1.11×10^5	1.32
W	422	41	0.14	5.19×10^5	2.41×10^{11}	7.12×10^4	9.94
Total	3,078	300	1.00			3.00×10^5	3.15

¹ $\pi_h = N_h/N$ is the stratum weight.

² $v_h = (1 - n_h/N_h) \pi_h^2 s_h^2/n_h$ is stratum h 's contribution to $V(\bar{y}_{str}) = \sum_h v_h$.

Point estimate: $\bar{y}_{str} = \sum_h \pi_h \bar{y}_h = 3.001 \times 10^5$

Standard error: $SE(\bar{y}_{str}) = \sqrt{\sum_h v_h} = \sqrt{3.154 \times 10^8} = 17,760.257$

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
300,051.6873	17,760.2566	265,241.5844	334,861.7901

and the true population proportion (π_h^{pop}). Post-stratification forces the weights to reflect the true population structure.

```
## strata mean estimate
```

```
res_post_strat <- str_est(ybarh, sh, nh_post, Nh, estimate = "mean")
res_post_strat$table
```

```
post_strat_est <- res_post_strat$estimate
post_strat_est |> format_est_gt("mean")
```

```
## compare with SRS estimate
```

```
res_srs_est <- srs_est(agsrs$acres92, N = 3087, estimate = "mean", show.details = FALSE)
srs_est_raw <- res_srs_est$estimate
srs_est_raw |> format_est_gt("mean")
```

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
297,897.0467	18,901.4092	260,700.4027	335,093.6907

```
## true mean check
mean(agpop$acres92)
```

```
[1] 308582.4
```

Output Note: The post-stratified estimate gives a Standard Error of 1.77603×10^4 , which is more precise than the unadjusted SRS standard error of 1.89014×10^4 .

4.5 Estimating Domain Means and Post-stratification with teacher example

4.5.1 Importing Data and Calculating Contingency Table

```
teacher <- read.csv("data/college_teacher.csv")
table(teacher[, 2:1])
```

```
      teacher
gender  0    1
  1 156   84
  2 120   40
```

```
addmargins(table(teacher[, 2:1]), 2)
```

```
      teacher
gender  0    1 Sum
  1 156   84 240
  2 120   40 160
```

```
# Proportions of students who want to become a teacher in two domains (female and male)
prop.table(table(teacher[, 2:1]), margin = "gender")
```

```
      teacher
gender  0    1
  1 0.65 0.35
  2 0.75 0.25
```

	$\bar{y}_h$	s_h	n_h^{obs}	π_h^{obs}
1	0.350	0.478	240.000	0.600
2	0.250	0.434	160.000	0.400

4.5.2 Domain Summary

```
n <- nrow(teacher)
ybarh <- tapply(teacher$teacher, INDEX = teacher$gender, FUN = mean)
sh <- tapply(teacher$teacher, INDEX = teacher$gender, FUN = sd)
nh <- tapply(teacher$teacher, INDEX = teacher$gender, FUN = length)
prop_h_obs <- nh / sum(nh)

data.frame(
  ybarh = ybarh,
  sh = sh,
  nh = nh,
  pi_obs = prop_h_obs
) |>
gt(rownames_to_stub = TRUE) |>
cols_label(
  ybarh = md("$\\bar{y}_h$"),
  sh = md("$s_h$"),
  nh = md("$n^{\\text{obs}}_h$"),
  pi_obs = md("$\\pi^{\\text{obs}}_h$")
) |>
fmt_number(columns = everything(), decimals = 3)
```

4.5.3 Estimating Domain Means

```
Nh <- c(3000, 1000)
n <- nrow(teacher)
domain_teacher_est <- data.frame(rbind(
  female = domain_est(subset(teacher, gender==1)$teacher, n=n, N=4000),
  male = domain_est(subset(teacher, gender==2)$teacher, n=n, N=4000)
))

domain_teacher_est |> format_est_gt("domain_mean")
```

	$\bar{y}_d$	SE($\bar{y}_d$)	95% CI Lower	95% CI Upper
female	0.3500	0.0293	0.2923	0.4077
male	0.2500	0.0326	0.1857	0.3143

4.5.4 Post-stratification Analysis

```
Nh <- c(3000, 1000)
prop_h_pop <- Nh / sum(Nh)
prop_h_pop
```

```
[1] 0.75 0.25
```

Create $n_h^{\text{post-strat}} = n \times \frac{N_h}{N}$

```
nh_post <- Nh / sum(Nh) * n

## show the differences
data.frame(
  phat = ybarh,
  sh = sh,
  nh = nh,
  pi_obs = nh/n,
  Nh = Nh,
  pi_pop = Nh/sum(Nh),
  nh_post = nh_post
) |>
gt(rownames_to_stub = TRUE) |>
cols_label(
  phat = md("$\\hat{p}_h$"),
  sh = md("$s_h$"),
  nh = md("$n_h$"),
  pi_obs = md("$\\pi^{\\text{obs}}_h$"),
  Nh = md("$N_h$"),
  pi_pop = md("$\\pi^{\\text{pop}}_h$"),
  nh_post = md("$n_h^{\\text{post-strat}}$")
) |>
fmt_number(columns = c(phat, sh, pi_obs, pi_pop, nh_post), decimals = 3)
```

	$\hat{p}_h$	s_h	n_h	π_h^{obs}	N_h	π_h^{pop}	$n_h^{\text{post-strat}}$
1	0.350	0.478	240	0.600	3000	0.750	300.000
2	0.250	0.434	160	0.400	1000	0.250	100.000

Stratified Mean Estimation: Working Table

Stratum	N_h	n_h	π_h^1	$\bar{y}_h$	s_h^2	$\pi_h \bar{y}_h$	
1	3,000	300	0.75	0.35	0.23	0.26	3.86
2	1,000	100	0.25	0.25	0.19	0.06	1.06
Total	4,000	400	1.00			0.32	4.92

¹ $\pi_h = N_h/N$ is the stratum weight.

² $v_h = (1 - n_h/N_h) \pi_h^2 s_h^2/n_h$ is stratum h 's contribution to $V(\bar{y}_{str}) = \sum_h v_h$.

Point estimate: $\bar{y}_{str} = \sum_h \pi_h \bar{y}_h = 0.325$

Standard error: $SE(\bar{y}_{str}) = \sqrt{\sum_h v_h} = \sqrt{4.916 \times 10^{-4}} = 0.022$

Calculate the post-stratification estimation of the mean:

```
## poststratification estimation of mean
res_teacher_post_strat <- str_est(ybarh, sh, nh_post, Nh, estimate = "mean")
res_teacher_post_strat$table
```

```
teacher_post_strat <- res_teacher_post_strat$estimate
teacher_post_strat |> format_est_gt("mean")
```

This post-stratification estimate explicitly corrects for the sampling imbalance using the true strata sizes:

```
# 0.35 * 0.75 + 0.25 * 0.25
(ybarh[1] * prop_h_pop[1]) + (ybarh[2] * prop_h_pop[2])
```

```
1
0.325
```

$\bar{y}$	$SE(\bar{y})$	95% CI Lower	95% CI Upper
0.3250	0.0222	0.2815	0.3685

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
0.3100	0.0220	0.2668	0.3532

4.5.5 Comparing to the Analysis with SRS

If we didn't post-stratify, the simple mean is heavily weighted toward males because they were over-sampled by chance (40% in sample vs 25% in population).

```
res_teacher_srs <- srs_est(teacher$teacher, N=sum(Nh), estimate = "mean", show.details = FALSE)
res_teacher_srs$table
```

NULL

```
teacher_srs <- res_teacher_srs$estimate
teacher_srs |> format_est_gt("mean")
```

This equals the naive sample proportion:

```
# 0.35 * 0.6 + 0.25 * 0.4
mean(teacher$teacher == 1)
```

```
[1] 0.31
```

4.6 Regression Estimation for the cherry.csv dataset

When the relationship between y and x doesn't pass exactly through the origin, we use Regression Estimation.

4.6.1 Importing data

```
cherry <- read.csv("data/cherry.csv", header = T)
ydata <- cherry$volume
xdata <- cherry$diameter
t_diameters <- 41835
xbarU <- t_diameters / 2967
N <- 2967
```

Coefficient Estimates

	Estimate	S.E.	t value	p value
(Intercept)	-36.9435	3.3651	-10.9783	0.0000
xdata	5.0659	0.2474	20.4783	0.0000

Model Fit Summary

R^2	Adj. R^2	$\hat{\sigma}$	F-statistic	p-value
0.9353	0.9331	4.2520	419.3603	0.0000

4.6.2 Regression Estimation Using the Function

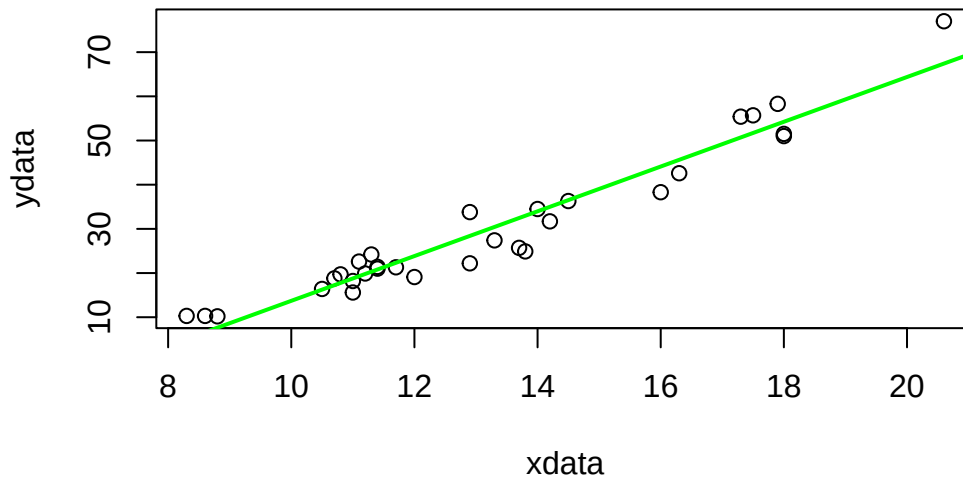
4.6.2.1 Fitting a linear regression model

```
n <- length(ydata)
lmfit <- lm(ydata ~ xdata)
lmfit_gt <- format_lm_gt(lmfit)
lmfit_gt$coef
```

```
lmfit_gt$fit
```

```
plot(xdata, ydata, main="Linear Regression: Volume vs Diameter")
abline(lmfit, col="green", lwd=2)
```

Linear Regression: Volume vs Diameter



The function's `$table` shows the row-level working values x_i , y_i , fitted $\hat{y}_i = \hat{B}_0 + \hat{B}_1 x_i$, residuals $e_i = y_i - \hat{y}_i$, and e_i^2 (with a Sum row and a footnote giving $\hat{B}_0$, $\hat{B}_1$, and s_e^2):

```
res_reg_model <- reg_est(ydata, xdata, N = N, estimate = "model")
res_reg_model$table
```

4.6.2.2 Estimate the mean

We compute the regression estimate $\bar{y}_{reg}$ by passing `xbarU` with `estimate = "mean"`.

```
res_reg_mean <- reg_est(ydata, xdata, xbarU = xbarU, N = N, estimate = "mean")
res_reg_mean$table
```

```
output_reg <- res_reg_mean$estimate
output_reg |> format_est_gt("reg_mean")
```

Output Note: The Regression estimate for the mean volume is 34.486 with a standard error of 0.7597.

4.6.2.3 Estimate the total

Regression Estimation: Working Table

$n = 31$

i	y_i	x_i	$\hat{y}_i^1$	e_i	e_i^2
1	10.300	8.300	5.103	5.197	27.007
2	10.300	8.600	6.623	3.677	13.521
3	10.200	8.800	7.636	2.564	6.574
4	16.400	10.500	16.248	1.520×10^{-1}	0.023
5	18.800	10.700	17.261	1.539	2.368
6	19.700	10.800	17.768	1.932	3.733
...	...	...	...	...	...
Sum	935.300	410.700	935.300	-3.553 $\times$ 10^{-15}	524.303

$$^1\hat{B}_0 = -36.9435, \hat{B}_1 = 5.0659, \hat{y}_i = \hat{B}_0 + \hat{B}_1 x_i, s_e^2 = 18.0794.$$

$$\text{Point estimate: } \hat{B}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{1,496.644}{295.437} = 5.066$$

$$\text{Standard error: } \text{SE}(\hat{B}_1) = \sqrt{\frac{s_e^2}{\sum_i (x_i - \bar{x})^2}} = \sqrt{\frac{18.079}{295.437}} = 0.247$$

Regression Estimation: Working Table

$n = 31$

i	y_i	x_i	$\hat{y}_i^1$	e_i	e_i^2
1	10.300	8.300	5.103	5.197	27.007
2	10.300	8.600	6.623	3.677	13.521
3	10.200	8.800	7.636	2.564	6.574
4	16.400	10.500	16.248	1.520×10^{-1}	0.023
5	18.800	10.700	17.261	1.539	2.368
6	19.700	10.800	17.768	1.932	3.733
...	...	...	...	...	...
Sum	935.300	410.700	935.300	-3.553 $\times$ 10^{-15}	524.303

$$^1\hat{B}_0 = -36.9435, \hat{B}_1 = 5.0659, \hat{y}_i = \hat{B}_0 + \hat{B}_1 x_i, s_e^2 = 18.0794.$$

$$\text{Point estimate: } \bar{y}_{reg} = \hat{B}_0 + \hat{B}_1 \bar{x}_U = -36.943 + 5.066 \times 14.100 = 34.486$$

$$\text{Standard error: } \text{SE}(\bar{y}_{reg}) = \sqrt{\left(1 - \frac{n}{N}\right) \frac{s_e^2}{n}} = \sqrt{\left(1 - \frac{31}{2967}\right) \frac{18.079}{31}} = 0.760$$

$\bar{y}_{reg}$	SE($\bar{y}_{reg}$)	95% CI Lower	95% CI Upper
34.4856	0.7597	32.9319	36.0393

$\hat{t}_{reg}$	SE($\hat{t}_{reg}$)	95% CI Lower	95% CI Upper
102,318.8602	2,253.9690	97,708.9761	106,928.7444

```
reg_total_volume_raw <- reg_est(ydata, xdata, xbarU = xbarU, N = N, estimate = "total")$estimator
reg_total_volume_raw |> format_est_gt("reg_total")
```

4.6.2.4 Percentage of Variance Reduction

```
reg_mean_volume_raw <- output_reg
var_reduction_reg <- 1 - (reg_mean_volume_raw["S.E."] / srs_mean_vol_raw["S.E."])^2
unnname(var_reduction_reg)
```

```
[1] 0.9330895
```

Output Note: The variance reduction for the regression estimator is 93.3%, which is similar to the ratio estimator since the true intercept is very close to zero.

4.7 Regression estimation for photo counts of dead trees

To estimate the number of dead trees in an area, we divide the area into 100 square plots and count dead trees on photographs. We select an SRS of 25 plots for exact field counts. We know the true mean photo count is 11.3.

```
photocounts <- c(10, 12, 7, 13, 13, 6, 17, 16, 15, 10, 14, 12, 10, 5, 12, 10,
                10, 9, 6, 11, 7, 9, 11, 10, 10)

fieldcounts <- c(15, 14, 9, 14, 8, 5, 18, 15, 13, 15, 11, 15, 12, 8, 13,
                9, 11, 12, 9, 12, 13, 11, 10, 9, 8)

lmfit <- lm(fieldcounts ~ photocounts)
lmfit_gt <- format_lm_gt(lmfit)
lmfit_gt$coef
```

Coefficient Estimates

	Estimate	S.E.	t value	p value
(Intercept)	5.0593	1.7635	2.8689	0.0087
photocounts	0.6133	0.1601	3.8316	0.0009

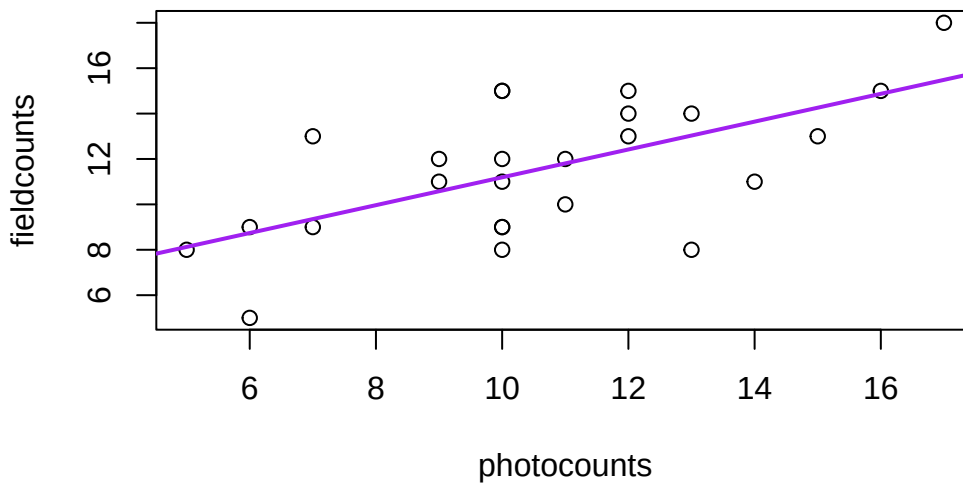
Model Fit Summary

R^2	Adj. R^2	$\hat{\sigma}$	F-statistic	p-value
0.3896	0.3631	2.4062	14.6815	0.0009

```
lmfit_gt$fit
```

```
plot(photocounts, fieldcounts, main="Field Counts vs Photo Counts")
abline(lmfit, col="purple", lwd=2)
```

Field Counts vs Photo Counts



Let's compare the three different estimation methods to see which provides the lowest Standard Error for the population mean.

```
# 1. Regression estimate for the mean
res_dt_reg <- reg_est(ydata = fieldcounts, xdata = photocounts, xbarU = 11.3, N = 100, estim
res_dt_reg$table
```

Regression Estimation: Working Table

$n = 25$

i	y_i	x_i	$\hat{y}_i$ ¹	e_i	e_i^2
1	15.000	10.000	11.192	3.808	14.501
2	14.000	12.000	12.419	1.581	2.501
3	9.000	7.000	9.352	-3.522×10^{-1}	0.124
4	14.000	13.000	13.032	9.681×10^{-1}	0.937
5	8.000	13.000	13.032	-5.032	25.320
6	5.000	6.000	8.739	-3.739	13.980
...	...	...	...	...	...
Sum	289.000	265.000	289.000	-2.665×10^{-15}	133.160

¹ $\hat{B}_0 = 5.0593$, $\hat{B}_1 = 0.6133$, $\hat{y}_i = \hat{B}_0 + \hat{B}_1 x_i$, $s_e^2 = 5.7896$.

Point estimate: $\bar{y}_{reg} = \hat{B}_0 + \hat{B}_1 \bar{x}_U = 5.059 + 0.613 \times 11.300 = 11.989$

Standard error: $SE(\bar{y}_{reg}) = \sqrt{\left(1 - \frac{n}{N}\right) \frac{s_e^2}{n}} = \sqrt{\left(1 - \frac{25}{100}\right) \frac{5.790}{25}} = 0.417$

$\bar{y}_{reg}$	$SE(\bar{y}_{reg})$	95% CI Lower	95% CI Upper
11.9893	0.4168	11.1272	12.8514

```
dt_reg_raw <- res_dt_reg$estimate
dt_reg_raw |> format_est_gt("reg_mean")
```

```
# (And estimating the total)
reg_est(ydata = fieldcounts, xdata = photocounts, xbarU = 11.3, N = 100, estimate = "total")
format_est_gt("reg_total")
```

```
# 2. Simple random sampling estimate for the mean (Ignoring photo counts)
res_dt_srs <- srs_est(sdata = fieldcounts, N = 100, estimate = "mean", show.details = FALSE)
dt_srs_raw <- res_dt_srs$estimate
dt_srs_raw |> format_est_gt("mean")
```

$\hat{t}_{reg}$	$SE(\hat{t}_{reg})$	95% CI Lower	95% CI Upper
1,198.9292	41.6758	1,112.7163	1,285.1422

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
11.5600	0.5222	10.4822	12.6378

Ratio Estimation: Working Table

$n = 25$

i	y_i	x_i	$\hat{y}_i$ ¹	e_i	e_i^2
1	15.000	10.000	10.906	4.094	16.764
2	14.000	12.000	13.087	9.132×10^{-1}	0.834
3	9.000	7.000	7.634	1.366	1.866
4	14.000	13.000	14.177	-1.774×10^{-1}	0.031
5	8.000	13.000	14.177	-6.177	38.160
6	5.000	6.000	6.543	-1.543	2.382
...	...	...	...	...	...
Sum	289.000	265.000	289.000	5.329×10^{-15}	184.645

¹ $\hat{B} = \sum_i y_i / \sum_i x_i = 1.0906$, $\hat{y}_i = \hat{B} x_i$, $s_e^2 = 7.6935$.

Point estimate: $\bar{y}_r = \hat{B} \bar{x}_U = 1.091 \times 11.300 = 12.323$

Standard error: $SE(\bar{y}_r) = SE(\hat{B}) \bar{x}_U = 0.045 \times 11.300 = 0.512$

```
# 3. Ratio estimate for the mean
```

```
res_dt_ratio <- ratio_est(ydata = fieldcounts, xdata = photocounts, xbarU = 11.3, N = 100, es
res_dt_ratio$table
```

```
dt_ratio_raw <- res_dt_ratio$estimate
dt_ratio_raw |> format_est_gt("ratio_mean")
```

Output Note:

- **SRS Mean:** SE = 0.522

$\bar{y}_r$	SE($\bar{y}_r$)	95% CI Lower	95% CI Upper
12.3234	0.5121	11.2664	13.3804

- **Ratio Mean:** $SE = 0.512$
- **Regression Mean:** $SE = 0.417$

Because the intercept of the true line is non-zero, the **Regression estimator** provides the most precise estimate in this scenario.

5 Cluster Sampling

5.1 Key Formulas for Cluster Sampling

Cluster sampling divides the population into N clusters, out of which a simple random sample of n clusters is selected. This design is highly practical when a sampling frame of individual elements is unavailable, or when individuals are geographically grouped.

1. One-Stage Cluster Sampling

In a one-stage design, every element M_i within the sampled cluster i is surveyed.

- **Total in Cluster i :** $t_i = \sum_{j=1}^{M_i} y_{ij}$
- **Ratio Estimator of the Population Mean:** The ratio estimator leverages cluster sizes to estimate the overall element mean:

$$\bar{y}_r = \frac{\sum_{i=1}^n t_i}{\sum_{i=1}^n M_i}.$$

- **Estimated Variance of the Mean:**

$$\hat{V}(\bar{y}_r) = \left(1 - \frac{n}{N}\right) \frac{1}{n\bar{M}^2} \frac{\sum_{i=1}^n (t_i - \bar{y}_r M_i)^2}{n-1}.$$

2. Two-Stage Cluster Sampling

In a two-stage design, we first sample n clusters, then take a subsample of m_i elements from the M_i elements within each selected cluster. Because we don't observe the true cluster total t_i , we estimate it:

- **Estimated Total in Cluster i :** $\hat{t}_i = M_i \bar{y}_i$, where $\bar{y}_i = \frac{1}{m_i} \sum_{j=1}^{m_i} y_{ij}$
- **Ratio Estimator of the Population Mean:**

$$\bar{y}_r = \frac{\sum_{i=1}^n \hat{t}_i}{\sum_{i=1}^n M_i}$$

5.2 Functions and packages for Analyzing Data

The estimating functions used throughout this book (including `cluster_ratio`, used below for one-stage and two-stage cluster samples) live in a single shared file, `samplingestimate.r`, which we source below.

5.3 Estimating for One-stage cluster sampling: analysis of algebra.csv

Consider a population of 187 high school algebra classes in a city. An investigator takes an SRS of 12 of those classes (the clusters). Because this is a one-stage sample, every student in the 12 sampled classes takes the test to evaluate their function knowledge.

We read the raw data and display it using `gt`'s interactive pagination feature to keep the document clean.

```
algebra <- read.csv("data/algebra.csv")

algebra |>
  gt() |>
  tab_header(title = "Raw Data: Algebra Classes") |>
  cols_label(Mi = html("M<sub>i</sub>")) |>
  opt_interactive(use_pagination = TRUE, page_size_default = 10)
```

Output Note: The table above displays the raw, unaggregated survey data. The interactive pagination allows you to flip through the dataset compactly.

Next, we run our updated function to compute both the cluster-level working table and the overall ratio estimate for the average student score.

```
alg_res <- cluster_ratio(data = algebra,
                        cname = "class",
                        csize = "Mi",
                        yvar = "score",
                        N = 187)

# Display the cluster-level working table
alg_res$table
```


Raw Data: Algebra Classes

class	M _i	score
23	20	57
23	20	90
23	20	56
23	20	57
23	20	46
23	20	55
23	20	62
23	20	66
23	20	78
23	20	76
23	20	57
23	20	84
23	20	27
23	20	70
23	20	49
23	20	82
23	20	52
23	20	82
23	20	59
23	20	25
37	26	57
37	26	34
37	26	68
37	26	99
37	26	24
37	26	83
37	26	40
37	26	98
37	26	90
37	26	50
37	26	60
37	26	64
37	26	58
37	26	100
37	26	68
37	26	52
37	26	77
37	26	69
37	26	64
37	26	64
37	26	73
37	26	46
37	26	44
37	26	95
37	26	28
37	26	65
38	24	41
38	24	66

Ratio Estimation: Working Table

$n = 12$

i	$\bar{y}_i$	$\hat{t}_i$ ¹	M_i	$\hat{B}M_i$ ²	e_i	e_i^2
1	61.500	1,230.000	20.000	1,251.371	-2.137×10^1	4.567×10^2
2	64.231	1,670.000	26.000	1,626.783	4.322×10^1	1.868×10^3
3	58.417	1,402.000	24.000	1,501.645	-9.965×10^1	9.929×10^3
4	58.000	1,972.000	34.000	2,127.331	-1.553×10^2	2.413×10^4
5	58.000	1,508.000	26.000	1,626.783	-1.188×10^2	1.411×10^4
6	64.857	1,816.000	28.000	1,751.920	6.408×10^1	4.106×10^3
...	...	...	...	...	...	...
Sum	...	18,708.000	299.000	18,708.000	1.364×10^{-12}	1.948×10^5

$$^1\hat{t}_i = \bar{y}_i M_i.$$

$$^2\hat{B} = \sum_i \hat{t}_i / \sum_i M_i = 62.5686, \hat{B}M_i = \hat{B} M_i, s_e^2 = 17711.5489.$$

$$\text{Point estimate: } \hat{B} = \frac{\sum_i \hat{t}_i}{\sum_i M_i} = \frac{18,708.000}{299.000} = 62.569$$

$$\text{Standard error: } \text{SE}(\hat{B}) = \frac{1}{\bar{x}} \sqrt{\left(1 - \frac{n}{N}\right) \frac{s_e^2}{n}} = \frac{1}{24.917} \sqrt{\left(1 - \frac{12}{187}\right) \frac{17,711.549}{12}} = 1.492$$

$\bar{y}_r$	SE($\bar{y}_r$)	95% CI Lower	95% CI Upper
62.5686	1.4916	59.2856	65.8515

Output Note: The working table above displays, for each of the 12 randomly selected classes, the class size (M_i), the total class score (t_i), the fitted value $\hat{B}M_i$, and the residual and squared residual used to estimate s_e^2 .

Finally, we extract the ratio estimate from the function output list to assess the overall population mean.

```
# Display the overall ratio estimate
alg_res$estimate |> format_est_gt("ratio_mean")
```

Output Note: Using the ratio estimator, the estimated mean score for a student across all 187 algebra classes is 62.57, with a 95% confidence interval of [59.29, 65.85]. The standard error accounts for the clustering effect, which typically increases variance compared to a pure SRS of students.

5.4 Estimating for two-stage cluster sampling: analysis of coots.csv

In two-stage cluster sampling, we sample clusters, and then subsample elements within them. In this dataset, researcher Arnold investigated coot eggs. He first selected a random sample of clutches (the clusters), and then measured the volume of a random subsample of eggs within each selected clutch.

```
coots <- read.csv("data/coots.csv")

coots |>
  gt() |>
  tab_header(title = "Raw Data: Coots") |>
  tab_options(
    container.height = px(400), # Adjust height as needed
    container.overflow.y = TRUE # Forces the vertical scroll bar
  )
```

We apply the exact same ratio estimator formulation, though mathematically, the cluster total t_i is now an *estimated* total $\hat{t}_i$.

Raw Data: Coots

clutch	csize	length	breadth	volume	tmt
1	13	44.30	31.10	3.7957569	1
1	13	45.90	32.70	3.9328497	1
2	13	49.20	34.40	4.2156036	1
2	13	48.70	32.70	4.1727621	1
3	6	51.05	34.25	0.9317646	0
3	6	49.35	34.40	0.9007362	0
4	11	49.20	31.55	3.0182724	1
4	11	48.55	33.10	2.9783969	1
5	10	49.40	34.55	2.5045800	1
5	10	49.05	34.95	2.4868350	1
6	13	46.35	33.10	3.9714070	1
6	13	46.65	33.30	3.9971119	1
7	9	48.15	33.95	1.9773760	1
7	9	45.70	32.65	1.8767619	1
8	11	47.45	34.10	2.9109152	1
8	11	49.10	33.80	3.0121377	1
9	12	47.50	33.50	3.4678800	1
9	12	47.30	33.30	3.4532784	1
10	11	50.00	33.45	3.0673500	1
10	11	46.55	31.60	2.8557029	1
11	12	48.50	33.05	3.5408880	0
11	12	47.35	34.00	3.4569288	0
12	11	48.75	34.50	2.9906662	1
12	11	49.05	34.35	3.0090704	1
13	12	49.95	33.70	3.6467496	1
13	12	47.75	33.05	3.4861320	1
14	11	49.30	35.30	3.0244071	1
14	11	48.05	35.80	2.9477233	1
15	11	49.00	33.55	3.0060030	1
15	11	48.25	33.30	2.9599928	1
16	10	48.10	34.10	2.4386700	0
16	10	46.85	33.15	2.3752950	0
17	9	48.55	35.35	1.9938029	0
17	9	49.35	34.85	2.0266564	0
18	10	48.15	33.60	2.4412050	0
18	10	48.00	32.85	2.4336000	0
19	11	47.80	33.45	2.9323866	1
19	11	47.60	33.50	2.9201172	1
20	11	50.40	32.95	3.0918888	1
20	11	45.70	33.25	2.8035579	1
21	10	50.70	36.00	2.5704900	1
21	10	49.40	34.70	2.5045800	1
22	13	50.90	34.00	4.3612647	1
22	13	48.75	34.55	4.1770463	1
23	12	52.15	35.70	3.8073672	0
23	12	51.00	35.75	3.7234080	0
24	10	50.00	33.00	2.5350000	0
24	10	51.15	33.20	2.5933050	0

Ratio Estimation: Working Table

$n = 184$

i	$\bar{y}_i$	$\hat{t}_i$ ¹	M_i	$\hat{B}M_i$ ²	e_i	e_i^2
1	3.864	50.236	13.000	32.378	1.786×10^1	318.923
2	4.194	54.524	13.000	32.378	2.215×10^1	490.483
3	0.916	5.498	6.000	14.943	-9.446	89.226
4	2.998	32.982	11.000	27.396	5.585	31.196
5	2.496	24.957	10.000	24.906	5.129×10^{-2}	0.003
6	3.984	51.795	13.000	32.378	1.942×10^1	377.053
...	...	...	...	...	...	...
Sum	...	4,375.947	1,757.000	4,375.947	4.476×10^{-13}	11,439.579

¹ $\hat{t}_i = \bar{y}_i M_i$.

² $\hat{B} = \sum_i \hat{t}_i / \sum_i M_i = 2.4906$, $\hat{B}M_i = \hat{B} M_i$, $s_e^2 = 62.5114$.

Point estimate: $\hat{B} = \frac{\sum_i \hat{t}_i}{\sum_i M_i} = \frac{4,375.947}{1,757.000} = 2.491$

Standard error: $SE(\hat{B}) = \frac{1}{\bar{x}} \sqrt{\frac{s_e^2}{n}} = \frac{1}{9.549} \sqrt{\frac{62.511}{184}} = 0.061$

```
coots_res <- cluster_ratio(data = coots,
                           cname = "clutch",
                           csize = "csize",
                           yvar = "volume")

coots_res$table
```

Output Note: Unlike the previous example, we only measured a subset of eggs per clutch. The function automatically estimates the total volume of each clutch ($\hat{t}_i$) by multiplying the true clutch size (M_i) by the observed subsample mean ($\bar{y}_i$); the working table (truncated to the first 6 rows + Sum) shows these estimated totals alongside the ratio fit and residuals.

```
# Display the overall ratio estimate
coots_res$estimate |> format_est_gt("ratio_mean")
```

Output Note: The estimated overall average egg volume is 2.491 units. Because the total number of clutches in the population (N) is not explicitly provided (set to `Inf` by default), the finite population correction factor is safely omitted from the standard error calculation.

$\bar{y}_r$	$SE(\bar{y}_r)$	95% CI Lower	95% CI Upper
2.4906	0.0610	2.3701	2.6110

6 Unequal Probability Sampling

6.1 Key Concepts and Formulas

Unequal Probability Sampling (UPS) improves estimation precision by sampling units with probabilities proportional to their size (PPS). If a cluster or county is larger, it has a higher chance of being included in the sample. This approach prevents extreme variance when cluster sizes vary drastically.

1. UPS With Replacement (UPSWR) When sampling with replacement, we draw n units. A single unit i has a selection probability ψ_i on each draw.

- **Total Estimator (Hansen-Hurwitz):** $\hat{t}_{pr} = \frac{1}{n} \sum_{i=1}^n \frac{y_i}{\psi_i}$
- **Ratio Estimator (Mean):** $\hat{y}_r = \frac{\sum_{i=1}^n \frac{y_i}{\psi_i}}{\sum_{i=1}^n \frac{M_i}{\psi_i}}$

2. UPS Without Replacement (UPSWO) When sampling without replacement, the inclusion probability for unit i across all draws is π_i .

- **Total Estimator (Horvitz-Thompson):** $\hat{t}_{HT} = \sum_{i=1}^n \frac{y_i}{\pi_i}$
-

6.2 Functions and Packages

The estimating functions used throughout this book (including `ratio_est`, `reg_est`, `srs_est`, `upswr_total`, `upswr_ratio`, `cluster_upswr_ratio`, `cluster_upswr_ratio`, and the `format_est_gt` table-formatting helper used below) live in a single shared file, `samplingestimate.r`, which we source below.

M_i	ψ_i	Total Hours (y_i)
24	0.037	75
100	0.155	203
100	0.155	203
76	0.117	191
44	0.068	168

6.3 Analysis of a dataset on student study hours collected by UPSWR

We first manually define a small dataset describing hours spent studying, where the probability of sampling a specific student group (ψ_i) was proportional to the number of students in that group (M_i).

```
# 1. Define the raw data
psi <- c(24, 100, 100, 76, 44) / 647
Mi <- c(24, 100, 100, 76, 44)
hrs <- c(75, 203, 203, 191, 168)

# Display the data cleanly
data.frame(Mi, psi, hrs) |>
  gt() |>
  cols_label(
    Mi = md("$M_i$"),
    psi = md("$\\psi_i$"),
    hrs = md("**Total Hours ($y_i$)**")
  ) |>
  fmt_number(columns = psi, decimals = 3)
```

```
plot(hrs ~ Mi, main = "Study Hours vs Number of Students",
     xlab = "Number of Students (Mi)", ylab = "Study Hours (hrs)")
abline(lm(hrs ~ Mi), col = "blue")
```


SRS Mean Estimation: Working Table

$n = 5$

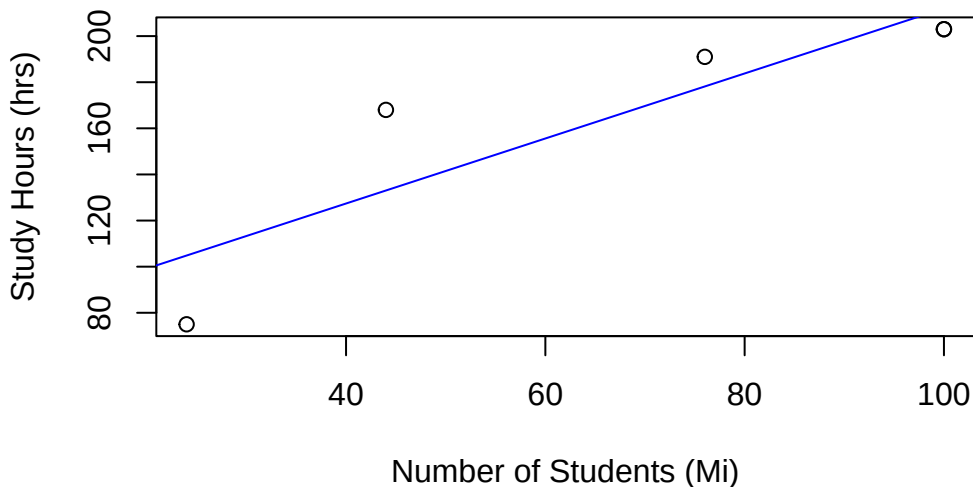
i	t_i/ψ_i	$t_i/\psi_i - \bar{y}$	$(t_i/\psi_i - \bar{y})^2$ ¹
1	2,021.875	2.729×10^2	7.445×10^4
2	1,313.410	-4.356×10^2	1.898×10^5
3	1,313.410	-4.356×10^2	1.898×10^5
4	1,626.013	-1.230×10^2	1.513×10^4
5	2,470.364	7.213×10^2	5.203×10^5
Sum	8,745.072	-4.547×10^{-13}	9.894×10^5

$$^1s_y^2 = \sum_i (t_i/\psi_i - \bar{y})^2 / (n - 1) = 247357.3300.$$

Point estimate: $\bar{y} = \frac{\sum_i t_i/\psi_i}{n} = \frac{8,745.072}{5} = 1,749.014$

Standard error: $SE(\bar{y}) = \frac{s}{\sqrt{n}} = \frac{497.350}{\sqrt{5}} = 222.422$

Study Hours vs Number of Students



Estimate the total and mean study hours:

```
# Estimate the total hours spent by all students
res_hrs_total <- upswr_total(total = hrs, psi = psi, show.details = TRUE)
res_hrs_total$table
```

$\hat{t}$	SE($\hat{t}$)	95% CI Lower	95% CI Upper
1,749.0144	222.4218	1,131.4724	2,366.5563

$\bar{y}$	SE($\bar{y}$)	95% CI Lower	95% CI Upper
2.7033	0.3438	1.7488	3.6577

```
res_hrs_total$estimate |> format_est_gt("total")
```

```
# Estimate the mean hours spent by each student (SE and CI scale down too)
(res_hrs_total$estimate / 647) |> format_est_gt("mean")
```

```
# Estimate the mean hours spent by each student
res_hrs_ratio <- upswr_ratio(total = hrs, M = Mi, psi = psi, estimate = "mean")
res_hrs_ratio$table
```

```
res_hrs_ratio$estimate |> format_est_gt("ratio_mean")
```

6.4 Analysis of statepop.csv collected with one-stage UPSWR

The file `statepop.csv` contains data from an unequal-probability sample of 100 U.S. counties. Counties were chosen with probabilities proportional to their populations. The total U.S. population is $M_0 = 255,077,536$. Because sampling was done **with replacement**, large counties like Los Angeles occur multiple times in the sample.

```
statepop <- read.csv("data/statepop.csv")

# Constants
M0 <- 255077536
popn <- statepop$popn
psi <- popn / M0
phys <- statepop$phys

# Display a subset of the raw data in a scrollable box
statepop |>
  select(state, county, popn, phys) |>
```

Ratio Estimation: Working Table

$n = 5$

i	t_i/ψ_i	M_i/ψ_i	$\hat{B} M_i/\psi_i$ ¹	e_i	e_i^2
1	2,021.875	647.000	1,749.014	2.729×10^2	7.445×10^4
2	1,313.410	647.000	1,749.014	-4.356×10^2	1.898×10^5
3	1,313.410	647.000	1,749.014	-4.356×10^2	1.898×10^5
4	1,626.013	647.000	1,749.014	-1.230×10^2	1.513×10^4
5	2,470.364	647.000	1,749.014	7.213×10^2	5.203×10^5
Sum	8,745.072	3,235.000	8,745.072	-4.547×10^{-13}	9.894×10^5

$$^1\hat{B} = \sum_i t_i/\psi_i / \sum_i M_i/\psi_i = 2.7033, \hat{B} M_i/\psi_i = \hat{B} M_i/\psi_i, s_e^2 = 247357.3300.$$

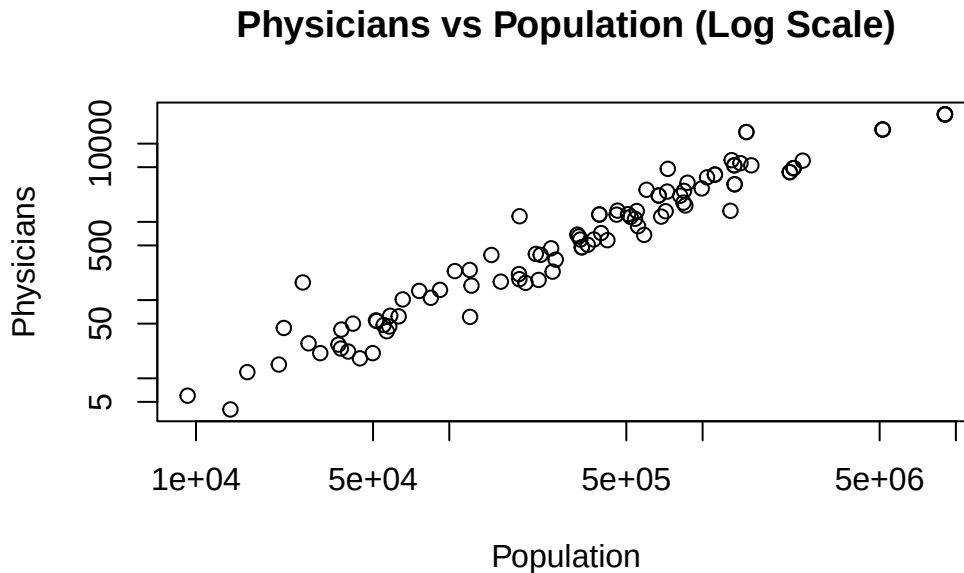
$$\text{Point estimate: } \hat{B} = \frac{\sum_i t_i/\psi_i}{\sum_i M_i/\psi_i} = \frac{8,745.072}{3,235.000} = 2.703$$

$$\text{Standard error: } \text{SE}(\hat{B}) = \frac{1}{\bar{x}} \sqrt{\frac{s_e^2}{n}} = \frac{1}{647.000} \sqrt{\frac{2.474 \times 10^5}{5}} = 0.344$$

$\bar{y}_r$	$\text{SE}(\bar{y}_r)$	95% CI Lower	95% CI Upper
2.7033	0.3438	1.7488	3.6577

```
mutate(psi = popn / M0) |>
gt() |>
tab_header(title = "U.S. County Data (Sample)") |>
tab_options(container.height = px(300), container.overflow.y = TRUE)
```

```
plot(popn, phys, log = "xy",
     main = "Physicians vs Population (Log Scale)",
     xlab = "Population", ylab = "Physicians")
```



Estimate the Total and Per Capita Physicians:

```
# Estimate the total number of physicians in the USA
res_phys_total <- upswr_total(phys, psi, show.details = FALSE)
res_phys_total$estimate |> format_est_gt("total")
```

```
# Estimate the number of physicians per capita (Ratio)
res_phys_ratio <- upswr_ratio(phys, popn, psi, estimate = "mean")
res_phys_ratio$table
```

```
res_phys_ratio$estimate |> format_est_gt("ratio_mean")
```

Output Note: The total estimator heavily leverages the selection probabilities to estimate that there are roughly 5.70304×10^5 physicians in the US.

U.S. County Data (Sample)

state	county	popn	phys	psi
AL	Wilcox	13672	4	5.359939e-05
AZ	Maricopa	2209567	4320	8.662335e-03
AZ	Maricopa	2209567	4320	8.662335e-03
AZ	Pinal	120786	61	4.735266e-04
AR	Garland	76100	131	2.983407e-04
AR	Mississippi	55060	48	2.158559e-04
CA	Contra_Costa	840585	1761	3.295410e-03
CA	Kern	587680	682	2.303927e-03
CA	Los_Angeles	9053645	23677	3.549370e-02
CA	Los_Angeles	9053645	23677	3.549370e-02
CA	Los_Angeles	9053645	23677	3.549370e-02
CA	Los_Angeles	9053645	23677	3.549370e-02
CA	Merced	189107	185	7.413707e-04
CA	Orange	2484789	6062	9.741309e-03
CA	Riverside	1288435	1385	5.051150e-03
CA	San_Francisco	728921	4761	2.857645e-03
CA	Ventura	686560	1168	2.691574e-03
CO	Arapahoe	420882	583	1.650016e-03
FL	Alachua	189409	1180	7.425546e-04
FL	Lee	352051	509	1.380172e-03
FL	Orange	714579	1367	2.801419e-03
FL	Pinellas	854976	1620	3.351828e-03
GA	Forsyth	49859	21	1.954661e-04
IL	Cook	5139341	15153	2.014815e-02
IL	Cook	5139341	15153	2.014815e-02
IL	Cook	5139341	15153	2.014815e-02
IL	DuPage	816116	2157	3.199482e-03
IL	Kane	333626	473	1.307940e-03
IL	Kane	333626	473	1.307940e-03
IL	Lake	541047	1093	2.121108e-03
IL	St._Clair	263124	329	1.031545e-03
IL	Williamson	58449	63	2.291421e-04
IN	Bartholomew	65498	102	2.567768e-04
KS	Douglas	84538	107	3.314208e-04
KY	Boone	63107	62	2.474032e-04
KY	Jefferson	670837	2171	2.629934e-03
KY	Jefferson	670837	2171	2.629934e-03
LA	Jefferson	457738	1237	1.794505e-03
LA	St._Charles	44372	18	1.739549e-04
MD	Frederick	159738	172	6.262331e-04
MD	Worcester	37306	24	1.462536e-04
MI	Oakland	1118611	4020	4.385376e-03
MI	Oakland	1118611	4020	4.385376e-03
MN	Hennepin	1041332	3706	4.082414e-03
MN	Otter_Tail	51362	55	2.013584e-04
MS	Leflore	37477	42	1.469240e-04
MS	Pearl_River	39879	22	1.563407e-04
MO	Jasper	91895	135	3.602630e-04

$\hat{t}$	SE($\hat{t}$)	95% CI Lower	95% CI Upper
570,304.2951	41,401.2299	488,155.2729	652,453.3173

Ratio Estimation: Working Table $n = 100$

i	t_i/ψ_i	M_i/ψ_i	$\hat{B} M_i/\psi_i^1$	e_i	e_i^2
1	7.463×10^4	2.551×10^8	5.703×10^5	-4.957×10^5	2.457×10^{11}
2	4.987×10^5	2.551×10^8	5.703×10^5	-7.159×10^4	5.126×10^9
3	4.987×10^5	2.551×10^8	5.703×10^5	-7.159×10^4	5.126×10^9
4	1.288×10^5	2.551×10^8	5.703×10^5	-4.415×10^5	1.949×10^{11}
5	4.391×10^5	2.551×10^8	5.703×10^5	-1.312×10^5	1.722×10^{10}
6	2.224×10^5	2.551×10^8	5.703×10^5	-3.479×10^5	1.211×10^{11}
...	...	...	...	...	...
Sum	5.703×10^7	2.551×10^{10}	5.703×10^7	-2.095×10^{-9}	1.697×10^{13}

$^1\hat{B} = \sum_i t_i/\psi_i / \sum_i M_i/\psi_i = 0.0022$, $\hat{B} M_i/\psi_i = \hat{B} M_i/\psi_i$, $s_e^2 = 171406183906.6202$.

Point estimate: $\hat{B} = \frac{\sum_i t_i/\psi_i}{\sum_i M_i/\psi_i} = \frac{5.703 \times 10^7}{2.551 \times 10^{10}} = 0.002$

Standard error: $SE(\hat{B}) = \frac{1}{\bar{x}} \sqrt{\frac{s_e^2}{n}} = \frac{1}{2.551 \times 10^8} \sqrt{\frac{1.714 \times 10^{11}}{100}} = 1.623 \times 10^{-4}$

$\bar{y}_r$	SE($\bar{y}_r$)	95% CI Lower	95% CI Upper
0.0022	0.0002	0.0019	0.0026

$\hat{t}$	$SE(\hat{t})$	95% CI Lower	95% CI Upper
933,410.9700	491,982.7871	-42,789.6161	1,909,611.5561

6.4.1 Compare with an estimate based on an SRS sample

Let's compare our UPSWR results against a naive Simple Random Sample of 100 counties.

```
# Read SRS sample, treating -99 as NA
counties <- read.csv("data/counties.csv", na.strings = "-99")
totpop <- 255077536
N <- 3141 # Total number of counties

# 1. Naive SRS estimate for total
res_counties_srs <- srs_est(counties$physician, N = N, estimate = "total", show.details = FALSE)
res_counties_srs$estimate |> format_est_gt("total")
```

```
# 2. Ratio estimate for total using SRS data
res_counties_ratio <- ratio_est(counties$physician, counties$totpop, xbarU = totpop/N, N = N, estimate = "total")
res_counties_ratio$table
```

```
res_counties_ratio$estimate |> format_est_gt("total")
```

```
# 3. Regression estimate for total using SRS data
res_counties_reg <- reg_est(counties$physician, counties$totpop, xbarU = totpop/N, N = N, estimate = "total")
res_counties_reg$table
```

```
res_counties_reg$estimate |> format_est_gt("reg_total")
```

Output Note: The true value is **532,638**. Notice how poorly the naive SRS estimate performs (predicting over 800,000) because it ignores county sizes. Both the SRS Ratio and SRS Regression methods perform much better by incorporating auxiliary population data.

6.5 Analysis of studenthrs.csv collected with two-stage UPSWR

Here, we sample classes (Primary Sampling Units) with replacement based on class size, and then subsample students within them to ask about their study hours.

Ratio Estimation: Working Table

$n = 100$

i	y_i	x_i	$\hat{y}_i^1$	e_i	e_i^2
1	24.000	3.602×10^4	90.313	-6.631×10^1	4.397×10^3
2	44.000	7.352×10^4	184.332	-1.403×10^2	1.969×10^4
3	7.000	6.408×10^3	16.066	-9.066	8.218×10^1
4	11.000	1.926×10^4	48.289	-3.729×10^1	1.390×10^3
5	3.000	7.649×10^3	19.177	-1.618×10^1	2.617×10^2
6	327.000	1.884×10^5	472.281	-1.453×10^2	2.111×10^4
...	...	...	...	...	...
Sum	29,717.000	1.185×10^7	29,717.000	-2.675×10^{-12}	1.705×10^7

$^1\hat{B} = \sum_i y_i / \sum_i x_i = 0.0025$, $\hat{y}_i = \hat{B} x_i$, $s_e^2 = 172267.8740$.

Point estimate: $\hat{t} = \hat{B} \bar{x}_U N = 0.003 \times 81,209.021 \times 3,141 = 6.395 \times 10^5$

Standard error: $SE(\hat{t}) = SE(\hat{B}) \bar{x}_U N = 3.445 \times 10^{-4} \times 81,209.021 \times 3,141 = 87,885.265$

$\hat{t}$	$SE(\hat{t})$	95% CI Lower	95% CI Upper
639,506.0284	87,885.2651	465,122.5955	813,889.4613

Regression Estimation: Working Table

$n = 100$

i	y_i	x_i	$\hat{y}_i$ ¹	e_i	e_i^2
1	24.000	3.602×10^4	52.564	-2.856×10^1	8.159×10^2
2	44.000	7.352×10^4	163.740	-1.197×10^2	1.434×10^4
3	7.000	6.408×10^3	-35.234	4.223×10^1	1.784×10^3
4	11.000	1.926×10^4	2.871	8.129	6.609×10^1
5	3.000	7.649×10^3	-31.555	3.455×10^1	1.194×10^3
6	327.000	1.884×10^5	504.237	-1.772×10^2	3.141×10^4
...	...	...	...	...	...
Sum	29,717.000	1.185×10^7	29,717.000	6.963×10^{-13}	1.135×10^7

¹ $\hat{B}_0 = -54.2313$, $\hat{B}_1 = 0.0030$, $\hat{y}_i = \hat{B}_0 + \hat{B}_1 x_i$, $s_e^2 = 115813.8922$.

Point estimate: $\hat{t}_{reg} = N\bar{y}_{reg} = 3,141 \times 186.524 = 5.859 \times 10^5$

Standard error: $SE(\hat{t}_{reg}) = N \times SE(\bar{y}_{reg}) = 3,141 \times 33.485 = 1.052 \times 10^5$

$\hat{t}_{reg}$	$SE(\hat{t}_{reg})$	95% CI Lower	95% CI Upper
585,870.5992	105,177.4184	377,149.4353	794,591.7630

Ratio Estimation: Working Table

$n = 5$

i	t_i/ψ_i	M_i/ψ_i	$\hat{B} M_i/\psi_i^1$	e_i	e_i^2
1	2,393.900	647.000	1,617.500	776.400	6.028×10^5
2	1,811.600	647.000	1,617.500	194.100	3.767×10^4
3	1,552.800	647.000	1,617.500	-64.700	4.186×10^3
4	1,035.200	647.000	1,617.500	-582.300	3.391×10^5
5	1,294.000	647.000	1,617.500	-323.500	1.047×10^5
Sum	8,087.500	3,235.000	8,087.500	0.000	1.088×10^6

$^1\hat{B} = \sum_i t_i/\psi_i / \sum_i M_i/\psi_i = 2.5000$, $\hat{B} M_i/\psi_i = \hat{B} M_i/\psi_i$, $s_e^2 = 272095.8500$.

Point estimate: $\hat{B} = \frac{\sum_i t_i/\psi_i}{\sum_i M_i/\psi_i} = \frac{8,087.500}{3,235.000} = 2.500$

Standard error: $SE(\hat{B}) = \frac{1}{\bar{x}} \sqrt{\frac{s_e^2}{n}} = \frac{1}{647.000} \sqrt{\frac{2.721 \times 10^5}{5}} = 0.361$

```

studenthrs <- read.table("data/studentshrs.txt", header = TRUE)

# Calculate sampling probability (cpsi) based on known total students (647)
studenthrs <- studenthrs |>
  mutate(cpsi = class_size / 647)

# Run the 2-stage UPSWR function
res_hrs <- cluster_upswr_ratio(
  data = studenthrs,
  sid = "sid",
  csize = "class_size",
  cpsi = "cpsi",
  yvar = "hrs",
  estimate = "mean"
)

# Display the cluster-level working table
res_hrs$table

```

Estimate the overall mean study hours:

$\bar{y}_r$	SE($\bar{y}_r$)	95% CI Lower	95% CI Upper
2.5000	0.3606	1.4989	3.5011

```
res_hrs$estimate |> format_est_gt("ratio_mean")
```

6.6 Simulation Studies for UPSWR using agpop data

Let's simulate 2000 random samples to see how UPSWR empirically compares to Simple Random Sampling.

```
# Read population data
agpop <- read.csv("data/agpop.csv", na.strings = "-99")
agpop <- agpop |> filter(!is.na(acres92), !is.na(acres87))

N <- nrow(agpop)
total_acres92 <- sum(agpop$acres92)
total_acres87 <- sum(agpop$acres87)
acres87 <- agpop$acres87

# Storage vectors for simulation
est_srs300 <- rep(0, 2000)
est_srs300_reg_acres87 <- rep(0, 2000)
est_ups300_acres87 <- rep(0, 2000)

for (i in 1:2000) {
  # SRS Simulation
  srs300 <- sample(1:N, size = 300)
  acres92_srs300 <- agpop$acres92[srs300]
  acres87_srs300 <- agpop$acres87[srs300]

  est_srs300[i] <- srs_est(acres92_srs300, N = N, estimate = "total", show.details = FALSE)$
  est_srs300_reg_acres87[i] <- reg_est(acres92_srs300, acres87_srs300,
                                       xbarU = total_acres87/N, N = N, estimate = "total")

  # UPSWR Simulation (Probability proportional to acres87)
  ups300 <- sample(1:N, size = 300, prob = acres87, replace = TRUE)
  acres92_ups300 <- agpop$acres92[ups300]
```

```

psi <- (acres87 / sum(acres87))[ups300]
est_ups300_acres87[i] <- upswr_total(acres92_ups300, psi, show.details = FALSE)$estimate[1,]
}

```

6.6.1 Visualizing Estimator Variance

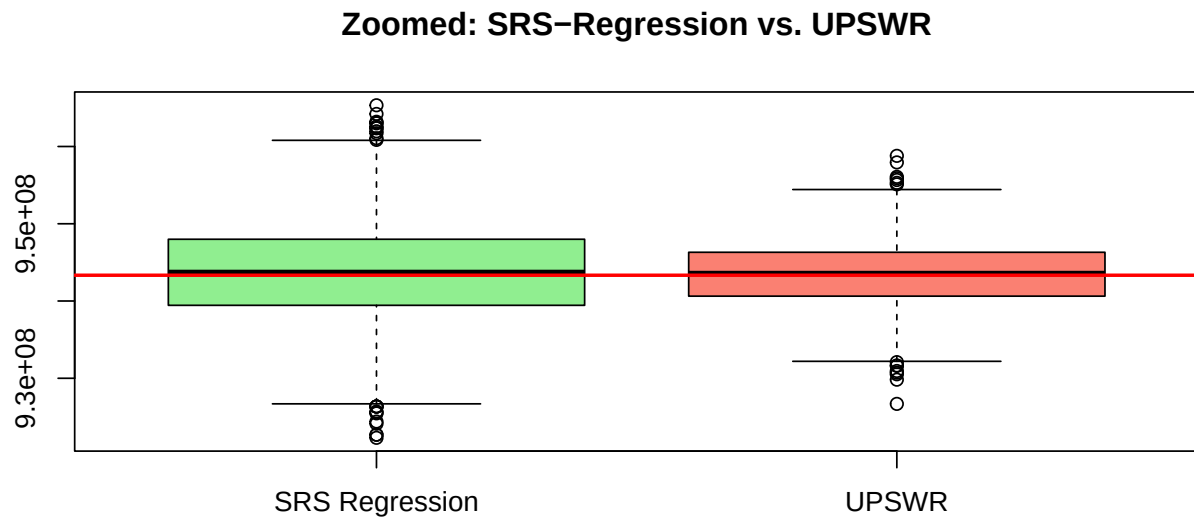
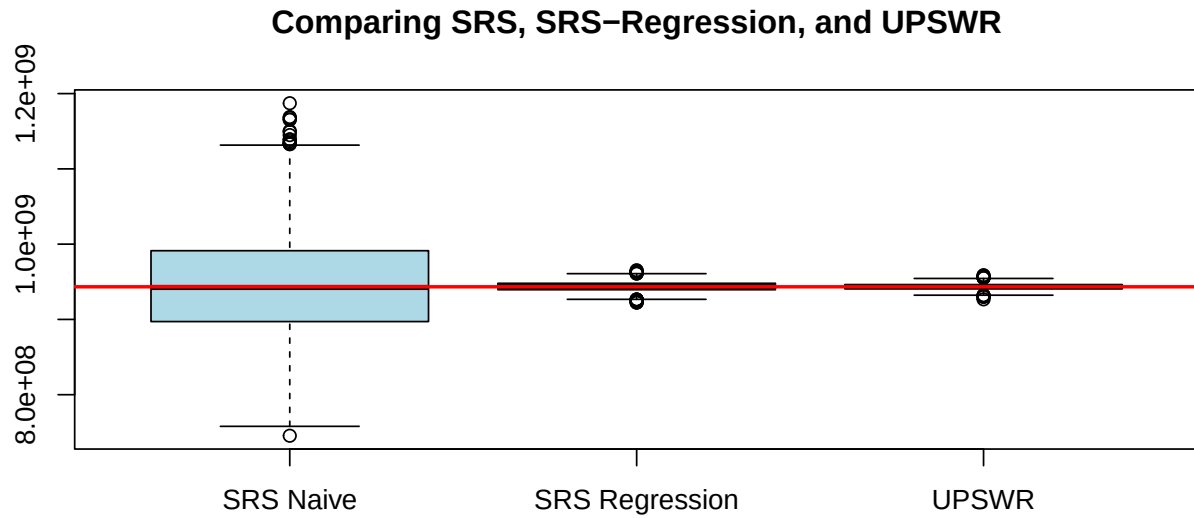
```

par(mfrow = c(2,1))

# Compare Naive SRS, Regression SRS, and UPSWR
boxplot(data.frame(est_srs300, est_srs300_reg_acres87, est_ups300_acres87),
        main = "Comparing SRS, SRS-Regression, and UPSWR",
        names = c("SRS Naive", "SRS Regression", "UPSWR"),
        col = c("lightblue", "lightgreen", "salmon"))
abline(h = total_acres92, col = "red", lwd = 2)

# Zoom in on the two high-precision estimators
boxplot(data.frame(est_srs300_reg_acres87, est_ups300_acres87),
        main = "Zoomed: SRS-Regression vs. UPSWR",
        names = c("SRS Regression", "UPSWR"),
        col = c("lightgreen", "salmon"))
abline(h = total_acres92, col = "red", lwd = 2)

```



Output Note: The red line represents the true total acreage. The naive SRS estimator has massive variance. Both SRS Regression and UPSWR drastically reduce variance, centering tightly around the true mean.

Ratio Estimation: Working Table

$n = 5$

i	$\hat{t}_i/\pi_i$	M_i/π_i	$\hat{B} M_i/\pi_i$ ¹	e_i	e_i^2
1	3.500	1.000	3.450	5.000×10^{-2}	0.002
2	5.000	1.000	3.450	1.550	2.402
3	3.625	1.000	3.450	1.750×10^{-1}	0.031
4	3.125	1.000	3.450	-3.250×10^{-1}	0.106
5	2.000	1.000	3.450	-1.450	2.103
Sum	17.250	5.000	17.250	-8.882×10^{-16}	4.644

¹ $\hat{B} = \sum_i \hat{t}_i/\pi_i / \sum_i M_i/\pi_i = 3.4500$, $\hat{B} M_i/\pi_i = \hat{B} M_i/\pi_i$, $s_e^2 = 1.1609$.

Point estimate: $\hat{B} = \frac{\sum_i \hat{t}_i/\pi_i}{\sum_i M_i/\pi_i} = \frac{17.250}{5.000} = 3.450$

Standard error: $SE(\hat{B}) = \frac{1}{\bar{x}} \sqrt{\frac{s_e^2}{n}} = \frac{1}{1.000} \sqrt{\frac{1.161}{5}} = 0.482$

6.7 Analysis of a dataset on student study hours collected by UPSWO

When sampling **Without Replacement** with unequal probabilities, we rely on the inclusion probabilities (π_k) rather than single-draw probabilities (ψ_i).

```
classpps <- read.csv("data/classpps.csv")

res_upswo <- cluster_upswo_ratio(
  data = classpps,
  cname = "class",
  csize = "clssize",
  cpik = "clssize", # Assuming inclusion prob is proportional to class size
  yvar = "hours",
  estimate = "mean"
)

# Display the cluster-level working table
res_upswo$table
```

Estimate the overall mean study hours (Horvitz-Thompson ratio style):

$\bar{y}_r$	SE($\bar{y}_r$)	95% CI Lower	95% CI Upper
3.4500	0.4819	2.1121	4.7879

```
res_upsw0$estimate |> format_est_gt("ratio_mean")
```

A Estimating Functions Source Code

The R functions used throughout this book to compute point estimates, standard errors, and confidence intervals — along with their `gt`-table formatting helpers — are defined in a single shared file, `samplingestimate.r`, which every chapter sources. Its full source is listed below for reference.

```
# =====
# Shared estimation functions for the Elements of Sampling Survey
# book. Source this file from any chapter that needs point/SE/CI
# estimators or their gt-table formatting helpers, e.g.:
#
#   source("samplingestimate.r")
#
# =====

suppressPackageStartupMessages({
  library(gt)
})

# -----
# Shared gt-table helpers
# -----

## Formats a gt table column as ordinary fixed-point, UNLESS its values are
## too small (< `small` in absolute value) or too large (>= `large`), in
## which case it switches that column to scientific notation (rendered as
## "m x 10^n" via gt's exp_style = "x10n") instead.
fmt_auto <- function(gt_tbl, data, columns, decimals = 3, small = 1e-3, large = 1e5)
{
  for (col in columns)
  {
    x <- data[[col]]
    x <- x[is.finite(x) & x != 0]
    if (length(x) == 0) next
    if (any(abs(x) < small) || any(abs(x) >= large))
      gt_tbl <- fmt_scientific(gt_tbl, columns = tidyselect::all_of(col), decimals =
```



```

      else
        gt_tbl <- fmt_number (gt_tbl, columns = tidyselect::all_of (col), decimals = dec)
      }
    gt_tbl
  }

## Formats a single scalar for inline formula display (fixed-point with
## thousands separators, switching to "m \times 10^{n}" LaTeX scientific
## notation for very small or very large magnitudes, mirroring fmt_auto()'s
## use of exp_style = "x10n") --- used to show worked-example steps such as
## \bar y = sum/n with the actual numbers plugged in.
fmt_num <- function (x, decimals = 3, small = 1e-3, large = 1e5)
{
  use_sci <- is.finite (x) & x != 0 & (abs (x) < small | abs (x) >= large)
  exps    <- ifelse (x == 0, 0, floor (log10 (abs (x))))
  mant    <- ifelse (x == 0, 0, x / 10^exps)
  ifelse (use_sci,
    sprintf ("%s \\times 10^{%d}", formatC (mant, format = "f", digits = decimals),
    formatC (x, format = "f", digits = decimals, big.mark = ","))
  )
}

## Truncates a row-level working data.frame whose LAST row is a summary row
## (e.g. "Sum"/"Total"): if there are more than head_n data rows (excluding
## that summary row), keeps only the first head_n data rows, inserts one
## "... " placeholder row, then the summary row --- so long working tables
## stay readable. id_col --- name of the row-label column.
truncate_working <- function (working, id_col, head_n = 6)
{
  n <- nrow (working) - 1
  if (n <= head_n + 1) return (working)

  dots <- working[1, ]
  dots[1, ] <- NA
  dots[1, id_col] <- "... "

  rbind (working[seq_len (head_n), ], dots, working[nrow (working), ])
}

# Helper function to format estimation outputs with appropriate mathematical headers
format_est_gt <- function(est_data, est_type = c("mean", "total", "ratio", "reg_mean", "reg_t
  est_type <- match.arg(est_type)

```

```

sym_map <- list(
  "mean"      = c(est = "$\\bar{y}$",      se = "$\\mathrm{SE}(\\bar{y})$"),
  "total"     = c(est = "$\\hat{t}$",      se = "$\\mathrm{SE}(\\hat{t})$"),
  "ratio"     = c(est = "$\\hat{B}$",      se = "$\\mathrm{SE}(\\hat{B})$"),
  "reg_mean"  = c(est = "$\\bar{y}_{reg}$", se = "$\\mathrm{SE}(\\bar{y}_{reg})$"),
  "reg_total" = c(est = "$\\hat{t}_{reg}$", se = "$\\mathrm{SE}(\\hat{t}_{reg})$"),
  "ratio_mean" = c(est = "$\\bar{y}_r$",    se = "$\\mathrm{SE}(\\bar{y}_r)$"),
  "domain_mean" = c(est = "$\\bar{y}_d$",    se = "$\\mathrm{SE}(\\bar{y}_d)$")
)

est_sym <- sym_map[[est_type]]["est"]
se_sym  <- sym_map[[est_type]]["se"]

# Convert to data frame. Use unname() to strip hidden lm() coefficient names
if (is.vector(est_data) && !is.list(est_data)) {
  df <- as.data.frame(t(unname(est_data)))
} else {
  df <- as.data.frame(est_data)
}

# Force standard column names so gt() never gets confused
colnames(df) <- c("Est.", "S.E.", "ci.low", "ci.upp")

# Check if we should use rownames as a stub
use_stub <- !is.null(rownames(df)) && !all(rownames(df) == as.character(1:nrow(df)))

df |>
  gt(rownames_to_stub = use_stub) |>
  cols_label(
    Est.    = md(est_sym),
    S.E.    = md(se_sym),
    ci.low  = md("95% CI Lower"),
    ci.upp  = md("95% CI Upper")
  ) |>
  fmt_number(
    columns = everything(),
    decimals = 4
  ) |>
  tab_options(table.width = pct(70))
}

## Formats an lm object as two gt tables, replacing the plain-text

```

```

## summary(lmfit) output:
##   $coef --- coefficient table (Estimate, S.E., t value, p value)
##   $fit  --- one-row table of model fit statistics, including R^2
## returns list($coef, $fit)
format_lm_gt <- function (lmfit)
{
  s <- summary (lmfit)

  coef_df <- as.data.frame (s$coefficients)
  colnames (coef_df) <- c ("Estimate", "S.E.", "t value", "p value")

  coef_gt <- coef_df |>
    gt (rownames_to_stub = TRUE) |>
    fmt_number (columns = c ("Estimate", "S.E.", "t value"), decimals = 4) |>
    fmt_number (columns = "p value", decimals = 4) |>
    tab_header (title = "Coefficient Estimates") |>
    tab_options (table.width = pct(70))

  fstat <- s$fstatistic
  fit_df <- data.frame (
    R.squared      = s$r.squared,
    Adj.R.squared  = s$adj.r.squared,
    Sigma          = s$sigma,
    F.statistic    = unname (fstat["value"]),
    p.value        = unname (pf (fstat["value"], fstat["numdf"], fstat["dendf"], lower.tail = FALSE))
  )

  fit_gt <- fit_df |>
    gt () |>
    cols_label (
      R.squared      = md ("R^2"),
      Adj.R.squared  = md ("Adj. R^2"),
      Sigma          = md ("σ"),
      F.statistic    = "F-statistic",
      p.value        = "p-value"
    ) |>
    fmt_number (columns = everything (), decimals = 4) |>
    tab_header (title = "Model Fit Summary") |>
    tab_options (table.width = pct(70))

  list (coef = coef_gt, fit = fit_gt)
}

```

```

# -----
# Base Estimators
# -----

## sdata --- a vector of original survey data
## N --- population size
## estimate --- "mean" (default) returns the population MEAN estimate;
##               "total" returns the population TOTAL estimate (N * mean,
##               with SE/CI scaled accordingly); requires finite N
## show.details --- if TRUE (default), also return a gt table of the
##                  row-level working values  $y_i$ ,  $y_i - \bar{y}$ ,  $(y_i - \bar{y})^2$ ,
##                  with a Sum row and a footnote showing  $s_y^2$ ; NULL otherwise
## returns list ($estimate = c(Est., S.E., ci.low, ci.upp), $table)
srs_est <- function (sdata, N = Inf, estimate = c("mean", "total"),
                    show.details = TRUE, col_labels = list(y = "y_i"))
{
  estimate <- match.arg (estimate)

  n <- length (sdata)
  ybar <- mean (sdata)
  dev <- sdata - ybar
  s2y <- sum (dev^2) / (n - 1)
  se.ybar <- sqrt((1 - n / N)) * sd (sdata) / sqrt(n)

  if (estimate == "total")
  {
    if (!is.finite (N))
      stop ("N must be finite to compute estimate = \"total\"")
    est <- N * ybar
    se.est <- N * se.ybar
  }
  else
  {
    est <- ybar
    se.est <- se.ybar
  }
  mem <- qt (0.975, df = n - 1) * se.est
  estimate_vec <- c (Est. = est, S.E. = se.est, ci.low = est - mem, ci.upp = est + mem)

  if (!show.details)
    return (list (estimate = estimate_vec, table = NULL))
}

```

```

working <- data.frame (
  i    = c (seq_len (n), "Sum"),
  y    = c (sdata, sum (sdata)),
  dev  = c (dev, sum (dev)),
  dev2 = c (dev^2, sum (dev^2))
)
working <- truncate_working (working, id_col = "i")

table <- gt (working) |>
  tab_header (title = "SRS Mean Estimation: Working Table", subtitle = md (sprintf ("%N", n)),
  cols_label (
    i    = md ("i"),
    y    = md (sprintf ("%s", col_labels$y)),
    dev  = md (sprintf ("%s-\\bar y", col_labels$y)),
    dev2 = md (sprintf ("%s-\\bar y)^2", col_labels$y))
  ) |>
  cols_width (i ~ px (40), everything () ~ px (110)) |>
  fmt_auto (data = working, columns = c ("y", "dev", "dev2"), decimals = 3) |>
  sub_missing (missing_text = "...") |>
  tab_style (
    style      = cell_text (weight = "bold"),
    locations  = cells_body (rows = i == "Sum")
  ) |>
  tab_footnote (
    footnote   = md (sprintf ("%s_y^2 = \\sum_i (%s-\\bar y)^2/(n-1) = %.4f$.", col_labels$y, sum (dev2))),
    locations  = cells_column_labels (columns = dev2)
  ) |>
  tab_source_note (
    source_note = md (if (estimate == "total")
      sprintf ("**Point estimate:** $\\hat t = N\\bar y = %s \\times %s = %s$",
        fmt_num (N, 0), fmt_num (ybar), fmt_num (est))
    else
      sprintf ("**Point estimate:** $\\bar y = \\dfrac{\\sum_i %s}{n} = \\dfrac{\\sum %s}{n}$",
        col_labels$y, fmt_num (sum (sdata)), n, fmt_num (ybar)))
  ) |>
  tab_source_note (
    source_note = md (if (estimate == "total")
      sprintf ("**Standard error:** $\\mathrm{SE}(\\hat t) = N \\times \\mathrm{SE}(\\bar y)$",
        fmt_num (N, 0), fmt_num (se.ybar), fmt_num (se.est))
    else if (is.finite (N))
      sprintf ("**Standard error:** $\\mathrm{SE}(\\bar y) = \\sqrt{1-\\dfrac{n}{N}} \\times \\mathrm{SE}(\\hat t)$",
        n, N, fmt_num (sqrt (s2y)), n, fmt_num (se.ybar))
  )

```

```

        else
            sprintf ("**Standard error:**  $\mathrm{SE}(\bar{y}) = \frac{s}{\sqrt{n}}$  =
                    fmt_num (sqrt (s2y)), n, fmt_num (se.ybar)))
    )

    list (estimate = estimate_vec, table = table)
}

## ydata --- observations of the variable of interest
## xdata --- observations of the auxiliary variable
## xbarU --- population mean of the auxiliary variable; required when
##           estimate is "mean" or "total"; ignored when estimate = "model"
## N --- population size
## estimate --- "mean" (default) returns the ratio estimate of the
##              population MEAN of y, Bhat*xbarU; "total" returns the ratio
##              estimate of the population TOTAL of y (requires finite N);
##              "model" returns the raw ratio coefficient Bhat = ybar/xbar
##              itself (with its own SE)
## show.details --- if TRUE (default), also return a gt table of the
##                  row-level working values y_i, x_i, fitted yhat_i =
##                  Bhat*x_i, residual e_i, and e_i^2, with a Sum row and a
##                  footnote showing Bhat and s_e^2; NULL otherwise
## col_labels --- named list (y, x, yhat) of RAW latex for those
##                working-table columns, so this function can be reused
##                (e.g. by cluster_ratio(), upsur_ratio()) with
##                context-appropriate labels
## extra_col --- optional list (label, values, formula) adding one extra
##                RAW-latex-labelled column of per-unit `values` right before
##                the y column of the working table (e.g. the within-cluster
##                means  $\bar{y}_i$  used by cluster_ratio() to build  $\hat{t}_i = \bar{y}_i M_i$ ); `formula` (optional) is shown as a
##                footnote on the y column documenting that relationship
## returns list ($estimate = c(Est., S.E., ci.low, ci.upp), $table)
ratio_est <- function (ydata, xdata, xbarU = NULL, N = Inf,
                      estimate = c ("mean", "total", "model"), show.details = TRUE,
                      col_labels = list (y = "y_i", x = "x_i", yhat = "\\hat{y}_i"),
                      extra_col = NULL)
{
    estimate <- match.arg (estimate)

    n <- length (xdata)
    xbar <- mean (xdata)

```

```

ybar <- mean (ydata)
B_hat <- ybar / xbar
yhat <- B_hat * xdata
e <- ydata - yhat
var_e <- sum (e^2) / (n - 1)
sd_B_hat <- sqrt ((1 - n/N) * var_e / n) / xbar

if (estimate == "model")
{
  est <- B_hat
  sd_est <- sd_B_hat
}
else
{
  if (is.null (xbarU))
    stop ("xbarU (population mean of x) is required when estimate is \"mean\" or \"total\"")
  if (estimate == "total")
  {
    if (!is.finite (N))
      stop ("N must be finite to compute estimate = \"total\"")
    est <- B_hat * xbarU * N
    sd_est <- sd_B_hat * xbarU * N
  }
  else
  {
    est <- B_hat * xbarU
    sd_est <- sd_B_hat * xbarU
  }
}

mem <- qt (0.975, df = n - 1) * sd_est
estimate_vec <- c (Est. = est, S.E. = sd_est, ci.low = est - mem, ci.upp = est + mem)

if (!show.details)
  return (list (estimate = estimate_vec, table = NULL))

working_cols <- list (i = c (seq_len (n), "Sum"))
if (!is.null (extra_col))
  working_cols$extra <- c (extra_col$values, NA)
working_cols$y <- c (ydata, sum (ydata))
working_cols$x <- c (xdata, sum (xdata))
working_cols$yhat <- c (yhat, sum (yhat))

```

```

working_cols$e      <- c (e, sum (e))
working_cols$e2     <- c (e^2, sum (e^2))
working <- as.data.frame (working_cols, check.names = FALSE)
working <- truncate_working (working, id_col = "i")

fmt_cols <- c ("y", "x", "yhat", "e", "e2")
if (!is.null (extra_col)) fmt_cols <- c ("extra", fmt_cols)

label_args <- list (i = md ("i"))
if (!is.null (extra_col))
  label_args$extra <- md (sprintf ("%s$", extra_col$label))
label_args$y      <- md (sprintf ("%s$", col_labels$y))
label_args$x      <- md (sprintf ("%s$", col_labels$x))
label_args$yhat   <- md (sprintf ("%s$", col_labels$yhat))
label_args$e      <- md ("%e_i$")
label_args$e2     <- md ("%e_i^2$")

table <- gt (working) |>
  tab_header (title = "Ratio Estimation: Working Table", subtitle = md (sprintf ("%n = %",
table <- do.call (cols_label, c (list (table), label_args))
table <- table |>
  cols_width (i ~ px (40), everything () ~ px (100)) |>
  fmt_auto (data = working, columns = fmt_cols, decimals = 3) |>
  sub_missing (missing_text = "...") |>
  tab_style (
    style      = cell_text (weight = "bold"),
    locations  = cells_body (rows = i == "Sum")
  ) |>
  tab_footnote (
    footnote   = md (sprintf ("$\hat{B} = \sum_i %s / \sum_i %s = %.4f$, %s = \hat{B}$",
    col_labels$y, col_labels$x, B_hat, col_labels$yhat, col_
    locations  = cells_column_labels (columns = yhat)
  )
if (!is.null (extra_col) && !is.null (extra_col$formula))
  table <- table |>
    tab_footnote (
      footnote   = md (sprintf ("%s$", extra_col$formula)),
      locations  = cells_column_labels (columns = y)
    )
table <- table |>
  tab_source_note (
    source_note = md (if (estimate == "model")

```



```

        sprintf ("**Point estimate:**  $\hat{B} = \frac{\sum_i y_i}{\sum_i x_i} = \frac{\text{col\_labels$y}}{\text{col\_labels$x}}$ , fmt_num (sum (ydata)), fmt_num (sum (xdata))
    else if (estimate == "total")
        sprintf ("**Point estimate:**  $\hat{t} = \hat{B} \cdot \bar{x}_U$ , N = %s  $\times$  %s", fmt_num (B_hat), fmt_num (xbarU), fmt_num (N, 0), fmt_num (est))
    else
        sprintf ("**Point estimate:**  $\bar{y}_r = \hat{B} \cdot \bar{x}_U$  = %s  $\times$  %s", fmt_num (B_hat), fmt_num (xbarU), fmt_num (est)))
) |>
tab_source_note (
  source_note = md (if (estimate == "model")
    (if (is.finite (N))
      sprintf ("**Standard error:**  $\text{SE}(\hat{B}) = \frac{1}{\sqrt{n}}$   $\sqrt{\text{var}_e}$ , n, N, fmt_num (var_e), n, fmt_num (sd_B_hat))
    else
      sprintf ("**Standard error:**  $\text{SE}(\hat{B}) = \frac{1}{\sqrt{n}}$   $\sqrt{\text{var}_e}$ , n, fmt_num (sd_B_hat)))
  else if (estimate == "total")
    sprintf ("**Standard error:**  $\text{SE}(\hat{t}) = \text{SE}(\hat{B}) \cdot \sqrt{\bar{x}_U^2}$ , fmt_num (sd_B_hat), fmt_num (xbarU), fmt_num (N, 0), fmt_num (sd_est))
  else
    sprintf ("**Standard error:**  $\text{SE}(\bar{y}_r) = \text{SE}(\hat{B}) \cdot \sqrt{\bar{x}_U^2}$ , fmt_num (sd_B_hat), fmt_num (xbarU), fmt_num (sd_est)))
)

list (estimate = estimate_vec, table = table)
}

## ydata --- observations of the variable of interest
## xdata --- observations of the auxiliary variable
## xbarU --- population mean of the auxiliary variable; required when
##           estimate is "mean" or "total"; ignored when estimate = "model"
## N --- population size
## estimate --- "mean" (default) returns the regression estimate of the
##              population MEAN of y, Bhat0 + Bhat1*xbarU; "total" returns
##              the regression estimate of the population TOTAL of y
##              (requires finite N); "model" returns the raw fitted slope
##              Bhat1 itself (with its own SE from the lm fit)
## show.details --- if TRUE (default), also return a gt table of the
##                  row-level working values y_i, x_i, fitted yhat_i, e_i,
##                  e_i^2, with a Sum row and a footnote showing Bhat0,
##                  Bhat1, and s_e^2; NULL otherwise

```

```

## returns list ($estimate = c(Est., S.E., ci.low, ci.upp), $table)
reg_est <- function (ydata, xdata, xbarU = NULL, N = Inf,
                    estimate = c("mean", "total", "model"), show.details = TRUE)
{
  estimate <- match.arg (estimate)

  n <- length (ydata)
  lmfit <- lm (ydata ~ xdata)
  Bhat <- lmfit$coefficients
  yhat <- lmfit$fitted.values
  e <- lmfit$residuals
  SSe <- sum (e^2) / (n - 2)

  if (estimate == "model")
  {
    est <- unname (Bhat[2])
    sd_est <- summary (lmfit)$coefficients[2, "Std. Error"]
  }
  else
  {
    if (is.null (xbarU))
      stop ("xbarU (population mean of x) is required when estimate is \"mean\" or \"total\"")
    yhat_reg <- unname (Bhat[1] + Bhat[2] * xbarU)
    se_yhat_reg <- sqrt ((1-n/N) * SSe / n)
    if (estimate == "total")
    {
      if (!is.finite (N))
        stop ("N must be finite to compute estimate = \"total\"")
      est <- yhat_reg * N
      sd_est <- se_yhat_reg * N
    }
    else
    {
      est <- yhat_reg
      sd_est <- se_yhat_reg
    }
  }

  mem <- qt (0.975, df = n - 2) * sd_est
  estimate_vec <- c (Est. = est, S.E. = sd_est, ci.low = est - mem, ci.upp = est + mem)

  if (!show.details)

```

```

    return (list (estimate = estimate_vec, table = NULL))

Sxx <- sum ((xdata - mean (xdata))^2)
Sxy <- sum ((xdata - mean (xdata)) * (ydata - mean (ydata)))

working <- data.frame (
  i    = c (seq_len (n), "Sum"),
  y    = c (ydata, sum (ydata)),
  x    = c (xdata, sum (xdata)),
  yhat = c (yhat, sum (yhat)),
  e    = c (e, sum (e)),
  e2   = c (e^2, sum (e^2))
)
working <- truncate_working (working, id_col = "i")

table <- gt (working) |>
  tab_header (title = "Regression Estimation: Working Table", subtitle = md (sprintf ("%s",
  cols_label (
    i    = md ("i"),
    y    = md ("y_i"),
    x    = md ("x_i"),
    yhat = md ("ŷ_i"),
    e    = md ("e_i"),
    e2   = md ("e_i^2")
  ) |>
  cols_width (i ~ px (40), everything () ~ px (100)) |>
  fmt_auto (data = working, columns = c ("y", "x", "yhat", "e", "e2"), decimals = 3) |>
  sub_missing (missing_text = "...") |>
  tab_style (
    style      = cell_text (weight = "bold"),
    locations = cells_body (rows = i == "Sum")
  ) |>
  tab_footnote (
    footnote = md (sprintf ("ŷ_0 = %.4f$, ŷ_1 = %.4f$, ŷ_i = ŷ_i - Bhat[1] + Bhat[2], SSe)),
    locations = cells_column_labels (columns = yhat)
  ) |>
  tab_source_note (
    source_note = md (if (estimate == "model")
      sprintf ("**Point estimate:** ŷ_1 = \\dfrac{\\sum_i(x_i-\\bar x)(y_i-\\bar y)}{\\sum_i(x_i-\\bar x)^2} = %s",
        fmt_num (Sxy), fmt_num (Sxx), fmt_num (unnamed (Bhat[2])))
      else if (estimate == "total")

```

```

        sprintf ("**Point estimate:**  $\hat{t}_{reg} = N\bar{y}_{reg} = %s \times %s = %s$ "
                fmt_num (N, 0), fmt_num (yhat_reg), fmt_num (est))
    else
        sprintf ("**Point estimate:**  $\bar{y}_{reg} = \hat{B}_0 + \hat{B}_1\bar{x}_U = %s + %s\bar{x}_U = %s$ "
                fmt_num (Bhat[1]), fmt_num (Bhat[2]), fmt_num (xbarU), fmt_num (yhat_reg))
    ) |>
    tab_source_note (
        source_note = md (if (estimate == "model")
            sprintf ("**Standard error:**  $\mathrm{SE}(\hat{B}_1) = \sqrt{\frac{s_e^2}{S_{xx}}} = %s$ "
                    fmt_num (SSe), fmt_num (Sxx), fmt_num (sd_est))
            else if (estimate == "total")
            sprintf ("**Standard error:**  $\mathrm{SE}(\hat{t}_{reg}) = N \times \mathrm{SE}(\bar{y}_{reg}) = %s$ "
                    fmt_num (N, 0), fmt_num (se_yhat_reg), fmt_num (sd_est))
            else if (is.finite (N))
            sprintf ("**Standard error:**  $\mathrm{SE}(\bar{y}_{reg}) = \sqrt{\frac{s_e^2}{n}} = %s$ "
                    n, N, fmt_num (SSe), n, fmt_num (se_yhat_reg))
            else
            sprintf ("**Standard error:**  $\mathrm{SE}(\bar{y}_{reg}) = \sqrt{\frac{s_e^2}{n}} = %s$ "
                    fmt_num (SSe), n, fmt_num (se_yhat_reg)))
    )

    list (estimate = estimate_vec, table = table)
}

## sdata --- a vector of original survey data in a domain
## N --- population size
## n --- total sample size (not the sample size in the domain)
## to find total, multiply domain size N_d to the estimate returned by this function
domain_est <- function (sdata, n, N = Inf)
{
    n_d <- length (sdata)
    ybar <- mean (sdata)
    se.ybar <- sqrt((1 - n / N)) * sd (sdata) / sqrt(n_d)
    mem <- qt (0.975, df = n_d - 1) * se.ybar
    c (Est. = ybar, S.E. = se.ybar, ci.low = ybar - mem, ci.upp = ybar + mem)
}

# -----
# Stratified Estimator
# -----

## Given per-stratum summaries, compute the stratified (or post-stratified)

```

```

## mean or total.
## estimate --- "mean" (default) returns the stratified MEAN estimate;
##               "total" returns the stratified TOTAL estimate
## show.details --- if TRUE (default), also return a gt table showing the
##                   stratum-by-stratum working, with a Total row
## for poststratification, use nh = n * Nh/N
## returns list ($estimate = c(Est., S.E., ci.low, ci.upp), $table)
str_est <- function (ybarh, sh, nh, Nh, estimate = c("mean", "total"), show.details = TRUE)
{
  estimate <- match.arg (estimate)

  N      <- sum (Nh)
  Pi_h   <- Nh / N
  v_h    <- (1 - nh / Nh) * Pi_h^2 * sh^2 / nh

  ybar    <- sum (ybarh * Pi_h)
  seybar  <- sqrt (sum (v_h))

  if (estimate == "total")
  {
    est      <- N * ybar
    se.est   <- N * seybar
  }
  else
  {
    est      <- ybar
    se.est   <- seybar
  }
  mem          <- 1.96 * se.est
  estimate_vec <- c (Est. = est, S.E. = se.est, ci.low = est - mem, ci.upp = est + mem)

  if (!show.details)
    return (list (estimate = estimate_vec, table = NULL))

  stratum <- names (Nh)
  if (is.null (stratum)) stratum <- seq_along (Nh)

  working <- data.frame (
    Stratum      = c (stratum, "Total"),
    Nh           = c (Nh, N),
    nh           = c (nh, sum (nh)),
    Pi_h         = c (Pi_h, sum (Pi_h)),

```

```

ybarh      = c (ybarh, NA),
sh2         = c (sh^2, NA),
weighted_mean = c (Pi_h * ybarh, ybar),
v_h         = c (v_h, seybar^2)
)

table <- gt (working) |>
  tab_header (title = "Stratified Mean Estimation: Working Table") |>
  cols_label (
    Stratum      = md ("Stratum"),
    Nh           = md ("N_h$"),
    nh           = md ("n_h$"),
    Pi_h         = md ("$\pi_h$"),
    ybarh        = md ("$\bar{y}_h$"),
    sh2          = md ("s_h^2$"),
    weighted_mean = md ("$\pi_h \bar{y}_h$"),
    v_h          = md ("v_h$")
  ) |>
  cols_width (c (Nh, nh) ~ px (60), everything () ~ px (110)) |>
  fmt_number (columns = c (Nh, nh), decimals = 0) |>
  fmt_number (columns = Pi_h, decimals = 2) |>
  fmt_auto (data = working, columns = c ("ybarh", "sh2", "weighted_mean", "v_h"), decimals = 2) |>
  sub_missing (missing_text = "") |>
  tab_style (
    style      = cell_text (weight = "bold"),
    locations = cells_body (rows = Stratum == "Total")
  ) |>
  tab_footnote (
    footnote = md ("$\pi_h = N_h/N$ is the stratum weight."),
    locations = cells_column_labels (columns = Pi_h)
  ) |>
  tab_footnote (
    footnote = md ("v_h = (1-n_h/N_h)\pi_h^2, s_h^2/n_h$ is stratum $h$'s variance"),
    locations = cells_column_labels (columns = v_h)
  ) |>
  tab_source_note (
    source_note = md (if (estimate == "total")
      sprintf ("**Point estimate:** $\hat{t}_{\text{str}} = N \bar{y}_{\text{str}} = \%s \times \%s$ \n",
        fmt_num (N, 0), fmt_num (ybar), fmt_num (est))
    else
      sprintf ("**Point estimate:** $\bar{y}_{\text{str}} = \sum_h \pi_h \bar{y}_h = \%s$")
    )
  ) |>

```

```

    tab_source_note (
      source_note = md (if (estimate == "total")
        sprintf ("**Standard error:**  $\mathrm{SE}(\hat{t}_{\text{str}}) = N \times \mathrm{SE}(\bar{y}_{\text{str}})$ "
          fmt_num (N, 0), fmt_num (seybar), fmt_num (se.est))
      else
        sprintf ("**Standard error:**  $\mathrm{SE}(\bar{y}_{\text{str}}) = \sqrt{\sum_h v_h}$ "
          fmt_num (sum (v_h)), fmt_num (seybar)))
    )

  list (estimate = estimate_vec, table = table)
}

## this function finds statistical estimates given a dataset with sampling weight
# stratdata --- data.frame containing stratified sample
# y --- name of variable for which we want to estimate population mean
# stratum --- name of variable that will be used as stratum variable
# weight --- name of variable indicating sampling weight
# note: from weights we can find Nh (see the code for formula)
# returns the same list ($estimate, $table) as str_est(), which
# this function calls after deriving ybarh/sh/nh/Nh from the raw data
str_est_data <- function (stratdata, y, stratum, weight, estimate = c ("mean", "total"), show.details = FALSE)
{
  ## compute stratum-wise data
  sh <- tapply (stratdata[, y], stratdata[,stratum], sd)
  ybarh <- tapply (stratdata[, y], stratdata[,stratum], mean)
  ## find population stratum size using sampling weight included in the data set
  Nh <- tapply (stratdata[, weight], stratdata[,stratum], sum)
  nh <- tapply (1:nrow(stratdata), stratdata[,stratum], length)

  str_est (ybarh, sh, nh, Nh, estimate = estimate, show.details = show.details)
}

# -----
# Cluster (ratio-to-size) Estimator
# -----

## Cluster ratio estimation (one-stage if  $M_i = m_i$ , i.e. every element in the
## sampled cluster is measured; two-stage/ratio-to-size if only  $m_i < M_i$ 
## elements are subsampled, so  $\hat{t}_i = M_i * \bar{y}_i$  is an ESTIMATED total).
## data --- original data frame for holding data
## cname --- variable recording cluster (psu) identity
## csize --- variable recording cluster (psu) population size (not sample size)

```

```

## yvar --- variable of interest
## N --- total number of clusters (psus) in the population
## estimate --- "mean" (default) or "model" both return the raw ratio
##           Bhat = sum(t_hat_i)/sum(Mi) --- for this ratio-to-size
##           cluster estimator, that raw B IS the population MEAN per
##           element, so the two coincide; "total" returns the
##           population TOTAL of y, Bhat*Mtotal_U (requires Mtotal_U,
##           the population total of the cluster-size variable)
## Mtotal_U --- population total of the cluster-size variable; required
##           when estimate = "total"
## show.details --- if TRUE, also return a gt table of the cluster-level
##           working values (t_hat_i, M_i, fitted, e_i, e_i^2), via
##           ratio_est()'s own working table
## returns list ($estimate = c(Est., S.E., ci.low, ci.upp), $table)
cluster_ratio <- function (data, cname, csize, yvar, N = Inf,
                           estimate = c("mean", "total", "model"),
                           show.details = TRUE, Mtotal_U = NULL)
{
  estimate <- match.arg (estimate)

  clust <- data[, cname]
  ydata <- data[, yvar]

  ybari <- tapply (ydata, clust, mean)
  Mi <- tapply (data[, csize], clust, function (x) x[1])
  ## same as the cluster total if all Mi elements were measured (mi = Mi)
  t_hat_cls <- ybari * Mi

  ybar_col <- list (label = "\\bar y_i", values = ybari, formula = "\\hat t_i = \\bar y_i \\")

  if (estimate == "total")
  {
    if (is.null (Mtotal_U))
      stop ("Mtotal_U (population total of the cluster-size variable) is required when es")
    ratio_est (t_hat_cls, Mi, xbarU = Mtotal_U, N = N, estimate = "mean", show.details = sh
              col_labels = list (y = "\\hat t_i", x = "M_i", yhat = "\\hat B M_i"),
              extra_col = ybar_col)
  }
  else
  {
    ratio_est (t_hat_cls, Mi, N = N, estimate = "model", show.details = show.details,
              col_labels = list (y = "\\hat t_i", x = "M_i", yhat = "\\hat B M_i"),

```



```

        extra_col = ybar_col)
    }
}

# -----
# UPS Estimators
# -----

## Total estimation for datasets collected with UPS with Replacement
## (psi-estimator / Hansen-Hurwitz).
## estimate --- "total" (default) is this function's whole purpose, computed
##               as the SAMPLE MEAN of the expanded pseudo-values  $t_i/\psi_i$ 
##               (a with-replacement HT-type property); "mean" instead
##               divides that total by N (requires finite N)
upswr_total <- function (total, psi, N = Inf, estimate = c ("total", "mean"), show.details =
{
  estimate <- match.arg (estimate)
  res <- srs_est (total/psi, N = N, estimate = "mean", show.details = show.details,
                 col_labels = list (y = "t_i/\psi_i"))
  if (estimate == "mean")
  {
    if (!is.finite (N)) stop ("N must be finite to compute estimate = \"mean\" (mean total
    res$estimate <- res$estimate / N
  }
  res
}

## Ratio estimation for datasets collected with UPS with Replacement.
## estimate --- "mean" (default) or "model" both return the raw ratio
##                $Bhat = \sum(t_i/\psi_i)/\sum(M_i/\psi_i)$  --- that raw B IS the
##               population MEAN per element; "total" returns the population
##               TOTAL of y,  $Bhat * Mtotal\_U$  (requires Mtotal_U, the population
##               total of the size variable)
upswr_ratio <- function (total, M, psi, N = Inf, estimate = c ("mean", "total", "model"),
                        show.details = TRUE, Mtotal_U = NULL)
{
  estimate <- match.arg (estimate)

  if (estimate == "total")
  {
    if (is.null (Mtotal_U))
      stop ("Mtotal_U (population total of the size variable) is required when estimate =

```

```

    ratio_est (total/psi, M/psi, xbarU = Mtotal_U, N = N, estimate = "mean", show.details =
      col_labels = list (y = "t_i/\psi_i", x = "M_i/\psi_i", yhat = "\hat B\,l
  }
  else
  {
    ratio_est (total/psi, M/psi, N = N, estimate = "model", show.details = show.details,
      col_labels = list (y = "t_i/\psi_i", x = "M_i/\psi_i", yhat = "\hat B\,l
  }
}

## Cluster UPS with Replacement: aggregates to the cluster level (ybari, Mi,
## psi per sampled cluster), then delegates to upswr_ratio(), so the
## working table and estimate/total logic are shared with the base function.
## data --- data frame holding the sample
## sid --- variable recording cluster (psu) identity
## csize --- variable recording cluster (psu) population size
## cpsi --- variable recording the cluster's selection probability psi_i
## yvar --- variable of interest (element level)
## estimate, Mtotal_U --- see upswr_ratio()
cluster_upswr_ratio <- function (data, sid, csize, cpsi, yvar, N = Inf,
  estimate = c ("mean", "total", "model"),
  show.details = TRUE, Mtotal_U = NULL)
{
  estimate <- match.arg (estimate)

  clust <- data[, sid]
  ydata <- data[, yvar]

  ybari <- tapply (ydata, clust, mean)
  Mi <- tapply (data[,csize], clust, function (x) x[1])
  psi <- tapply (data[,cpsi], clust, function (x) x[1])
  t_hat_cls <- ybari * Mi

  upswr_ratio (t_hat_cls, Mi, psi, N = N, estimate = estimate,
    show.details = show.details, Mtotal_U = Mtotal_U)
}

## Cluster UPS without Replacement: same idea as cluster_upswr_ratio(),
## but dividing by the cluster's inclusion probability pik instead of psi.
## data --- data frame holding the sample
## cname --- variable recording cluster identity
## csize --- variable recording cluster population size

```

```

## cpik --- variable recording the cluster's inclusion probability  $\pi_k$ 
## yvar --- variable of interest (element level)
cluster_upswo_ratio <- function (data, cname, csize, cpik, yvar, N = Inf,
                                estimate = c ("mean", "total", "model"),
                                show.details = TRUE, Mtotal_U = NULL)
{
  estimate <- match.arg (estimate)

  clust <- data[, cname]
  ydata <- data[, yvar]

  ybari <- tapply (ydata, clust, mean)
  Mi <- tapply (data[,csize], clust, function (x) x[1])
  pik <- tapply (data[,cpik], clust, function (x) x[1])
  t_hat_cls <- ybari * Mi

  if (estimate == "total")
  {
    if (is.null (Mtotal_U))
      stop ("Mtotal_U (population total of the size variable) is required when estimate = 'total'")
    ratio_est (t_hat_cls/pik, Mi/pik, xbarU = Mtotal_U, N = N, estimate = "mean", show.details = show.details,
              col_labels = list (y = "\\hat t_i/\\pi_i", x = "M_i/\\pi_i", yhat = "\\hat t_i"))
  }
  else
  {
    ratio_est (t_hat_cls/pik, Mi/pik, N = N, estimate = "model", show.details = show.details,
              col_labels = list (y = "\\hat t_i/\\pi_i", x = "M_i/\\pi_i", yhat = "\\hat t_i"))
  }
}

```